Contents

1 Choose Your Destiny Tutorial

- Choose Your Destiny Tutorial

Figure 1: logo

1.1 Introduction

ChooseYourDestiny (or **CYD** for short) is an environment that will allow you to create the gamebook experience on your Spectrum. The tool consists of a compiler that, using a simple language on a ready-made text, allows you to add interactivity and visual and sound effects to "play" this type of adventure.

When designing this type of tool there are different approaches. You can design a simple tool, easy to use and understand for anyone, but at the cost of reducing its flexibility. Or you can take a compiler of a general language for Spectrum such as **ZxBasic** from Boriel or **Z88DK**, where you already have to have advanced concepts of the machine and get into it, if necessary, even with assembler. For **CYD** I decided to take a middle ground approach: a simple, markup-based programming language, yet flexible enough to be able to create almost anything you want, but simplified enough to focus on the creative aspects rather than the technical ones.

Therefore, this tutorial has been created to introduce concepts that will allow you to understand how it works and start making your own adventures. This document is constantly being revised and updated but may be a bit out of date with respect to the reference distribution, so the absolute reference for information will always be the manual, which is always updated with the tool. I recommend that you always have it at hand.

In addition, there are also examples in the distribution's `examples` folder, which you can play with and learn from, and which will surely give you many ideas for your own creations. To try them out, simply copy the contents of each example folder and copy it to the root directory of the distribution, overwriting the files. If you have something already done that you want to keep, **remember to copy it somewhere else first**.

With this, I hope it helps you create adventures that we can all enjoy.

1.2 Installation

1.2.1 Windows

To install on Windows 10 (64-bit) or higher, download the `ChooseYourDestiny.Win_x64.zip` file from the Releases section of the repository and unzip it into a folder called Tutorial, which you can create wherever you see fit. The script to build the adventure is called `make_adv.cmd`, you will need to run it to compile the adventure. I recommend doing this from the command line.

1.2.2 Linux, BSDs, etc. (Experimental)

You must first meet the prerequisites, for this see the relevant section of the manual.

Then download the `ChooseYourDestiny.Linux_x64.zip` file from the Releases section of the repository and unzip it into a folder called Tutorial, which you can create wherever you see fit. The script to build the adventures is called `make_adv.cmd`, you will need to run it to compile the adventure. I recommend doing this from the command line.

1.2.3 Highlighter for VSCode

To start using the extension with Visual Studio Code, download the file chooseyourdestiny-highlighter-x.x.x.vsix from Releases. On VsCode, go to the extensions screen, and on the ... button, it will open a new menu. From that, select the Install from VSIX option and open the previous file.

Otherwise, download this repository and copy the folder into the `<user home>/.vscode/extensions` folder and restart Code. If your Code installation is portable, it must be copied on the folder `data/extensions/` inside of the VSCode folder.

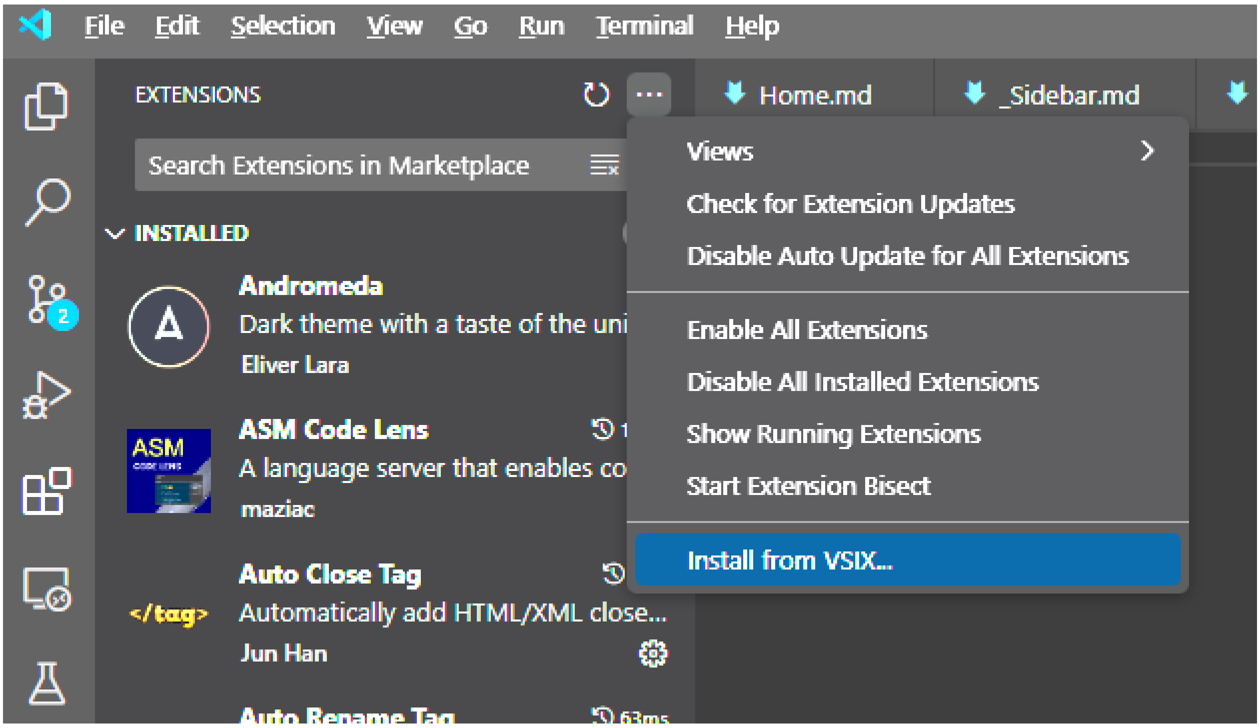


Figure 2: Resaltador

1.3 Preparing our first adventure

The first thing we are going to do is change a couple of things so that we can generate a custom adventure.

If you are using Windows, open the file `make_adv.cmd` with any text editor and you will see this at the beginning:

```
REM ---- Configuration variables ----------

REM Name of the game
SET GAME=test
REM This name will be used as:
REM   - The file to compile will be test.cyd with this example
REM   - The name of the TAP file or +3 disk image

REM Target for the compiler (48k, 128k for TAP, plus3 for DSK)
SET TARGET=48k

REM Number of lines used on SCR files at compressing
SET IMGLINES=192

REM Loading screen
SET LOAD_SCR="LOAD.scr"

REM Parameters for compiler
SET CYDC_EXTRA_PARAMS=
```

```
REM ------
```

The first thing we are going to do is give a name to the adventure we are going to create. For example, we will call it `Tutorial`:

```
REM ---- Configuration variables ----------

REM Name of the game
SET GAME=Tutorial
REM This name will be used as:
REM   - The file to compile will be test.cyd with this example
REM   - The name of the TAP file or +3 disk image

REM Target for the compiler (48k, 128k for TAP, plus3 for DSK)
SET TARGET=48k

REM Number of lines used on SCR files at compressing
SET IMGLINES=192

REM Loading screen
SET LOAD_SCR="LOAD.scr"

REM Parameters for compiler
SET CYDC_EXTRA_PARAMS=

REM ------
```

If you use another operating system, the process is similar with `make_adv.sh`:

```
# ---- Configuration variables ----------
# Name of the game
GAME="Tutorial"
# This name will be used as:
#   - The file to compile will be test.cyd with this example
#   - The name of the TAP file or +3 disk image
#
# Target for the compiler (48k, 128k for TAP, plus3 for DSK)
TARGET="48k"
#
# Number of lines used on SCR files at compressing
IMGLINES="192"
#
# Loading screen
LOAD_SCR="./LOAD.scr"
#
# Parameters for compiler
CYDC_EXTRA_PARAMS=
# -------------------------------------
```

We save the file and now create a new text file, called `tutorial.Cyd`.In this file, we write this:

```
Hello World[[WAITKEY]]
```

And we keep it in the same place as `Make_adv.cmd` and`make_adv.sh`. We execute the batch file and if everything goes well, you will have created a tape file called tutorial.TAP, which you can run with your favorite emulator.

1.4 Source File Format

When launching the resulting Tap file with an emulator, this comes out:

Figure 3: Screen 1

Let's analyze what happens…

Apart from `GAME`, another important variable is `TARGET`, which indicates the Spectrum model and the type of output file to use. For now we will use the value `48k`, which we will later change when we want more advanced features.

Returning to the code of the adventure, we see that the text *Hello World* is painted and then a kind of cursor appears. If we press the `Enter` or `Space` key, the program is restarted. If we return to the code:

```
Hello World[[WAITKEY]]
```

There are two different parts, one is the *Hello World*, and then *[[WAITKEY]]*. The second part is a command that is sent to the interpreter to bring up that animated cursor and wait for a key to be pressed. This is the fundamental basis for understanding how the compiler works: **Everything between `[[` and `]]` is considered code or commands for the engine and everything outside is considered printable text**.

To understand this better, let's do an experiment. Let's put a line break after the two open brackets, like this:

```
Hello World[[
WAITKEY]]
```

If we compile and load the game, we see that the same thing happens. Now, we delete the line break we put and put it **before** the double brackets:

```
Hello World
[[WAITKEY]]
```

If we compile and load again, we see that now the icon is in the following line:

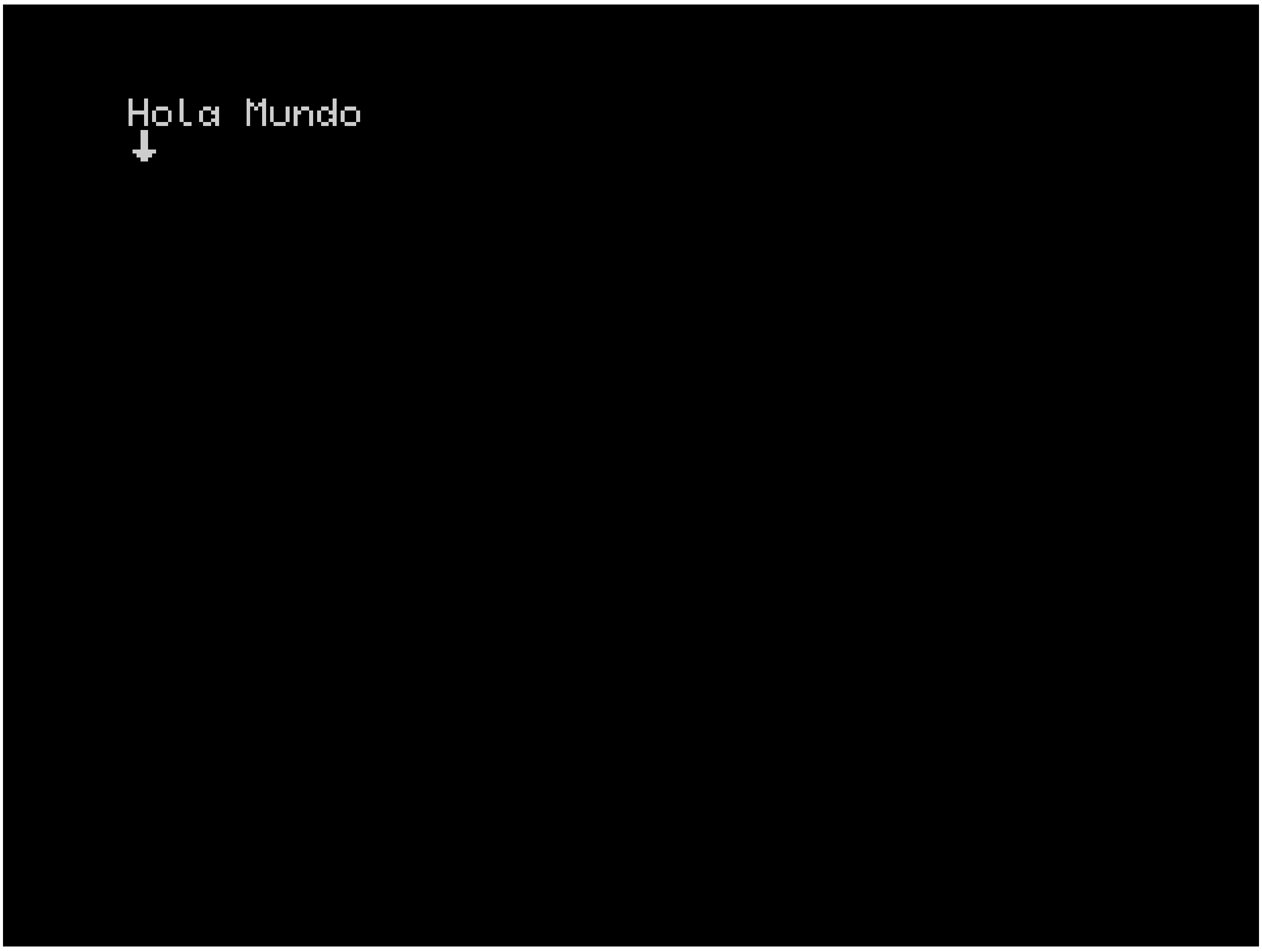


Figure 4: Screen 2

That is because the line jump, being out of the double brackets, is considered printable text and, therefore, the waiting icon passes to the next line. Keep these situations when you write the adventure.

Let's leave this as I was and we will add more commands. Type this within `tutorial.CyD`:

```
[[CLEAR]]Hello World[[WAITKEY]]
```

Now we have put a command ahead of the text. If we compile and execute, we get this:

Figure 5: Screen 3

With the CLEAR command we delete the printable area that, for the moment, is the full screen. As the screen is automatically deleted at the beginning of the interpreter, we will not see anything at the moment, but with this command we will have the clean screen to print from the beginning.You have a complete reference of the commands in the [Manual](https://github.com/cronomantic/ChooseYourDestiny/blob/main/manual_es.md).

Now we are going to change the color of the text.For this we are going to use the INK N command, where n is a number from 0 to 7 that corresponds to the colors of the spectrum.By default it is white, so we are going to put it with a cyan color, which is number 5, which would be ink 5. We will put it before Clear, such as: like this:

```
[[INK 5]][[CLEAR]]Hello World[[WAITKEY]]
```

The result is …

But the color is a bit dark … let's bright it! To do this we use the Bright 1 command, (again, look at the reference in the manual), this way:

```
[[INK 5]][[BRIGHT 1]][[CLEAR]]Hello World[[WAITKEY]]
```

This is already better.

But both bracket can be quite confusing and unpleasant to view. As I have already indicated, every

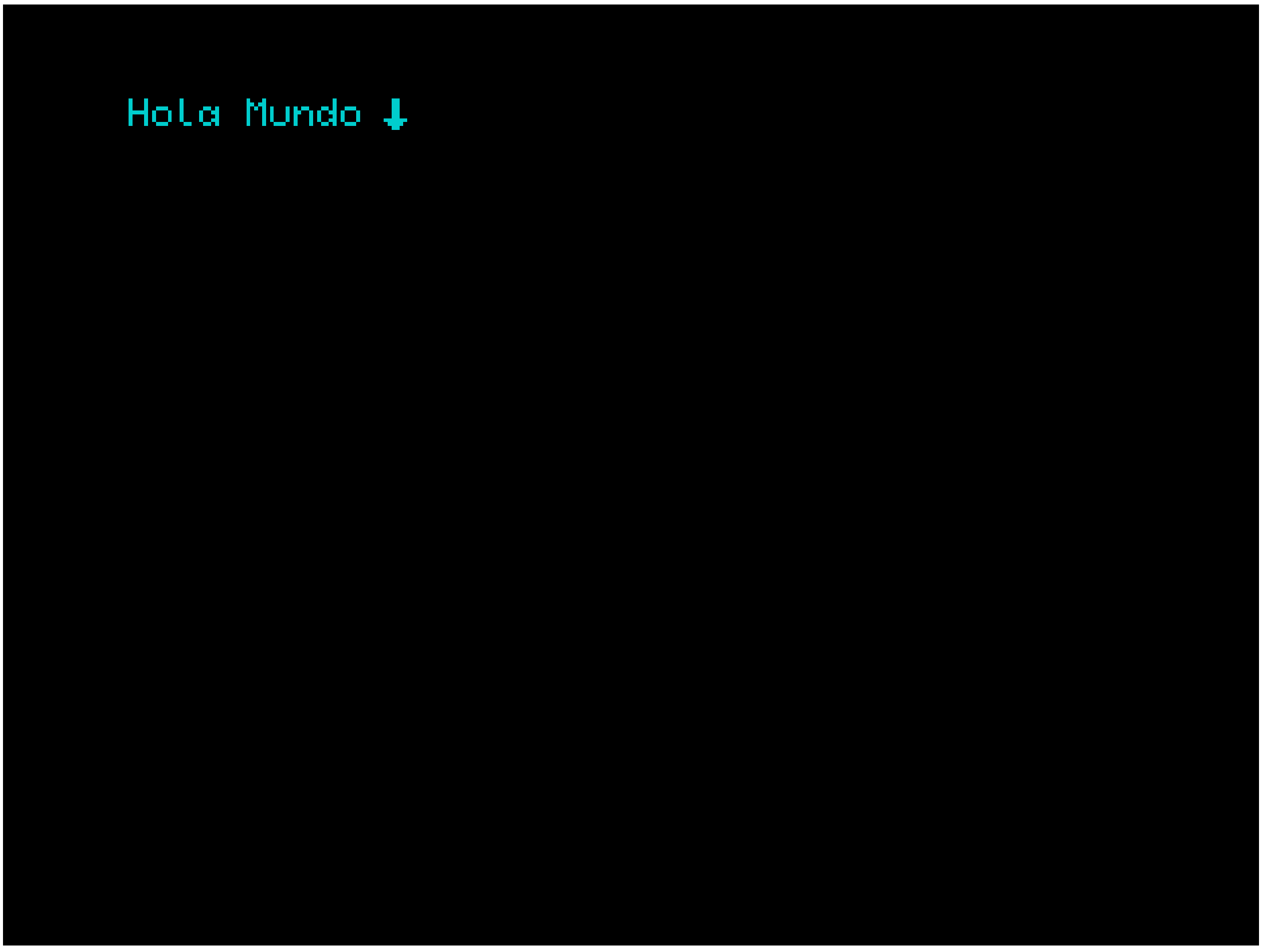


Figure 6: Screen 4

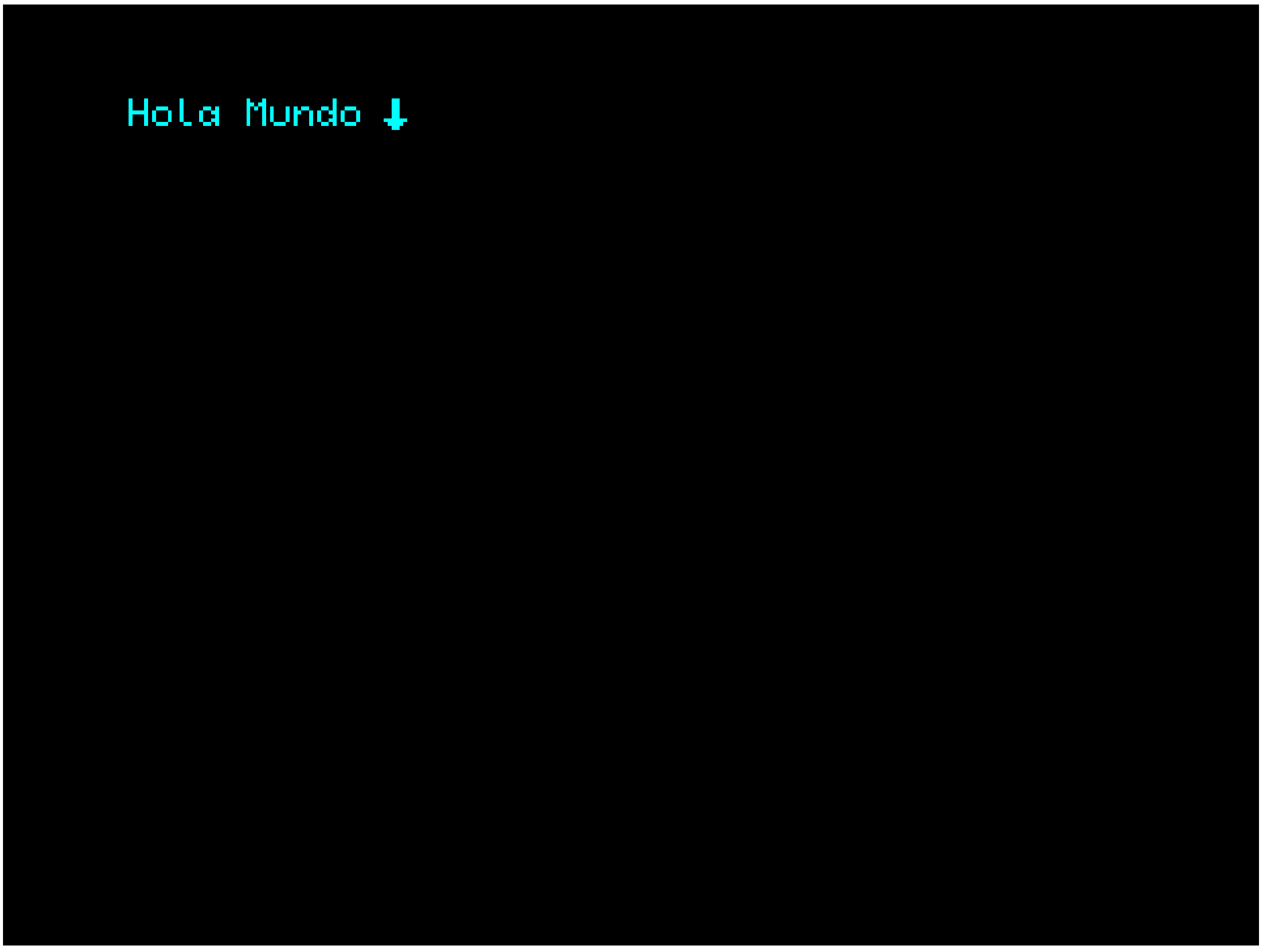


Figure 7: Screen 5

time it is `[[`, the compiler interprets that the following are commands. How can we chain them without being opening and closing brackets? Well… there are two ways:

- Through line jumps:

```
[[
    INK 5
    BRIGHT 1
    CLEAR
]]Hello World[[WAITKEY]]
```

- Through two points on the same line:

```
[[ INK 5 : BRIGHT 1 : CLEAR ]]Hello World[[ WAITKEY ]]
```

The two previous variants will produce the same result and are equivalent.

One last point is the comments. Within the code we can put comments by surrounding them with `/*` and `*/`:

```
[[
    INK 5     /* Print in cyan color */
    BRIGHT 1  /* Enable brightness */
    CLEAR     /* Clear the screen */
]]Hello World[[
  WAITKEY     /* Wait for a key press */
]]
```

With this you should have a good notion of how the CyD source code works.

1.5 Jumps and labels

In the example from the previous chapter, you may have noticed that when we press the select key, the Spectrum is reset. This is because when we press the validation key (while waiting with WAITKEY), it reaches the end of the file. When this happens, the Spectrum is reset. We can do the same thing by using the END command anywhere in the code.

But we don't want it to do that. We want it to start over again. To do this, copy the following into the source file:

```
[[
  LABEL principio
  INK 5
  BRIGHT 1
  CLEAR
]]Hello World[[
  WAITKEY
  GOTO principio
]]
```

When you compile and run it, it won't look very nice, but you'll notice that when you press the validate key, it doesn't reset, but rather clears the screen and prints the text and waits again.

The two additions to the code are `LABEL beginning` and `GOTO beginning`. The first command is not really a command, but a label, which puts a marker at that point with an identifier named *beginning*; and the second tells the interpreter to jump to where the label *beginning* is located.

The result is that, when you press the validate key with WAITKEY, the `GOTO beginning` is found and jumps to where the label *beginning* is declared, which, being the beginning, what it does is re-execute all the subsequent commands and print the text *Hello World*, and wait with WAITKEY again… In short, we have made an infinite loop.

However, we can improve the example like this:

```
[[
  INK 5
  BRIGHT 1
  LABEL principio
  CLEAR
]]Hello World[[
  WAITKEY
  GOTO principio
]]
```

Now the label is declared just before the screen is cleared, and it will go there when the GOTO is reached, leaving the INK and BRIGHT unexecuted. Why? Because it is no longer necessary to execute them again, we have already set the text color and brightness at the beginning and executing them again is redundant!

The concept of labels and jumps is fundamental to understanding how to make a "Choose Your Own Adventure", since we will present options to the player, and depending on those options, we will go from one place to another in the text.

An important detail is the format of the label identifiers. These can only be a **sequence of numbers and letters or the underscore character in a row, and must start with a letter.**. That is, `LABEL 1` or `LABEL La Etiqueta` are not valid, but `LABEL l1` or `LABEL LaEtiqueta` are. They are also case sensitive, meaning that they are **case sensitive**, so `LABEL Etiqueta` and `LABEL Etiqueta` are not the same label. And obviously, you cannot declare a label with the same name twice.

On the other hand, commands are not case sensitive, meaning that `CLEAR`, `clear` or `Clear` are perfectly valid. However, I recommend capitalizing them to better distinguish them.

Starting with version 0.5, a shorthand way of declaring labels has been added, prefixing the label name with the character `#`, meaning that `LABEL Etiqueta` can be written as `#Etiqueta`. Thus, the previous example can be written like this:

```
[[
  INK 5
  BRIGHT 1
  #principio
  CLEAR
```

```
]]Hello World[[
  WAITKEY
  GOTO principio
]]
```

1.6 Options

Options are the most important feature of the engine. Again, let's see it with the example from the manual:

```
[[ /* Set screen colors and clear it */
   PAPER 0    /* Black background color  */
   INK   7    /* White text color */
   BORDER 0   /* Black border color  */
   CLEAR      /* Clear the screen*/
]][[ LABEL Localidad1]]You are in location 1. Where do you want to go?
[[ OPTION GOTO Localidad2 ]]Go to location 2
[[ OPTION GOTO Localidad3 ]]Go to location 3
[[ CHOOSE ]]
[[ LABEL Localidad2 ]]You did it!!!
[[ GOTO Final ]]
[[ LABEL Localidad3 ]]You are dead!!!
[[ GOTO Final]]
[[ LABEL Final : WAITKEY: END ]]
```

When compiling and running we have this:

We are shown two options that we can choose with the **P** and **Q** keys and select one with **Space** or **Enter**. If we choose the first option, we get this:

And if we choose the second:

With the command `OPTION GOTO label`, what we do is declare a selectable option. The place where the cursor is at that moment will be the point where the option icon appears. Let's rearrange the options a bit to illustrate this last point:

```
[[ /* Set screen colors and clear it */
   PAPER 0    /* Black background color  */
   INK   7    /* White text color */
   BORDER 0   /* Black border color  */
   CLEAR      /* Clear the screen*/
]][[ LABEL Localidad1]]You are in location 1.
Where do you want to go?

  [[ OPTION GOTO Localidad2 ]]Go to location 2

  [[ OPTION GOTO Localidad3 ]]Go to location 3
[[ CHOOSE ]]
[[ LABEL Localidad2 ]]You did it!!!
```

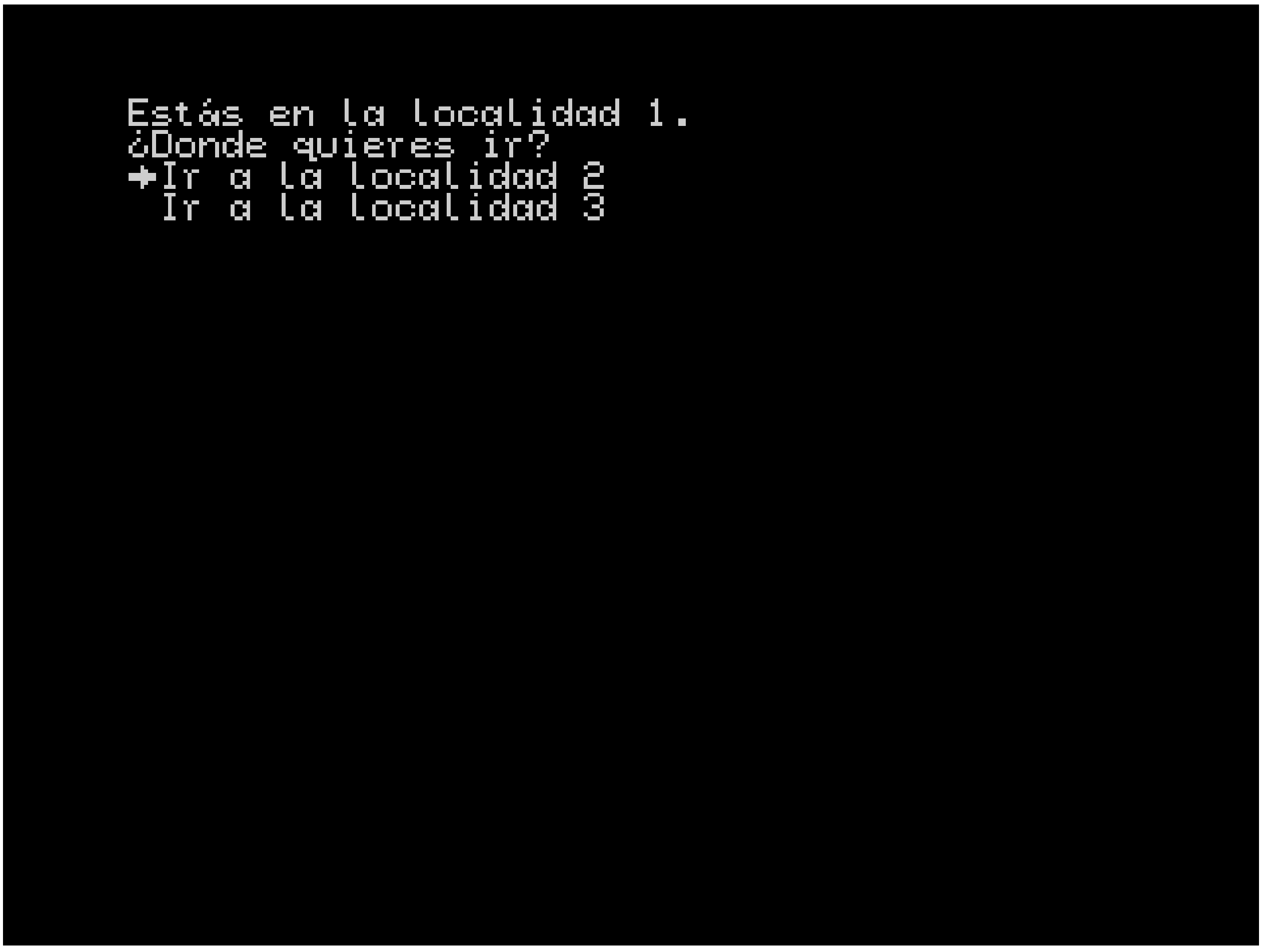


Figure 8: Options menu

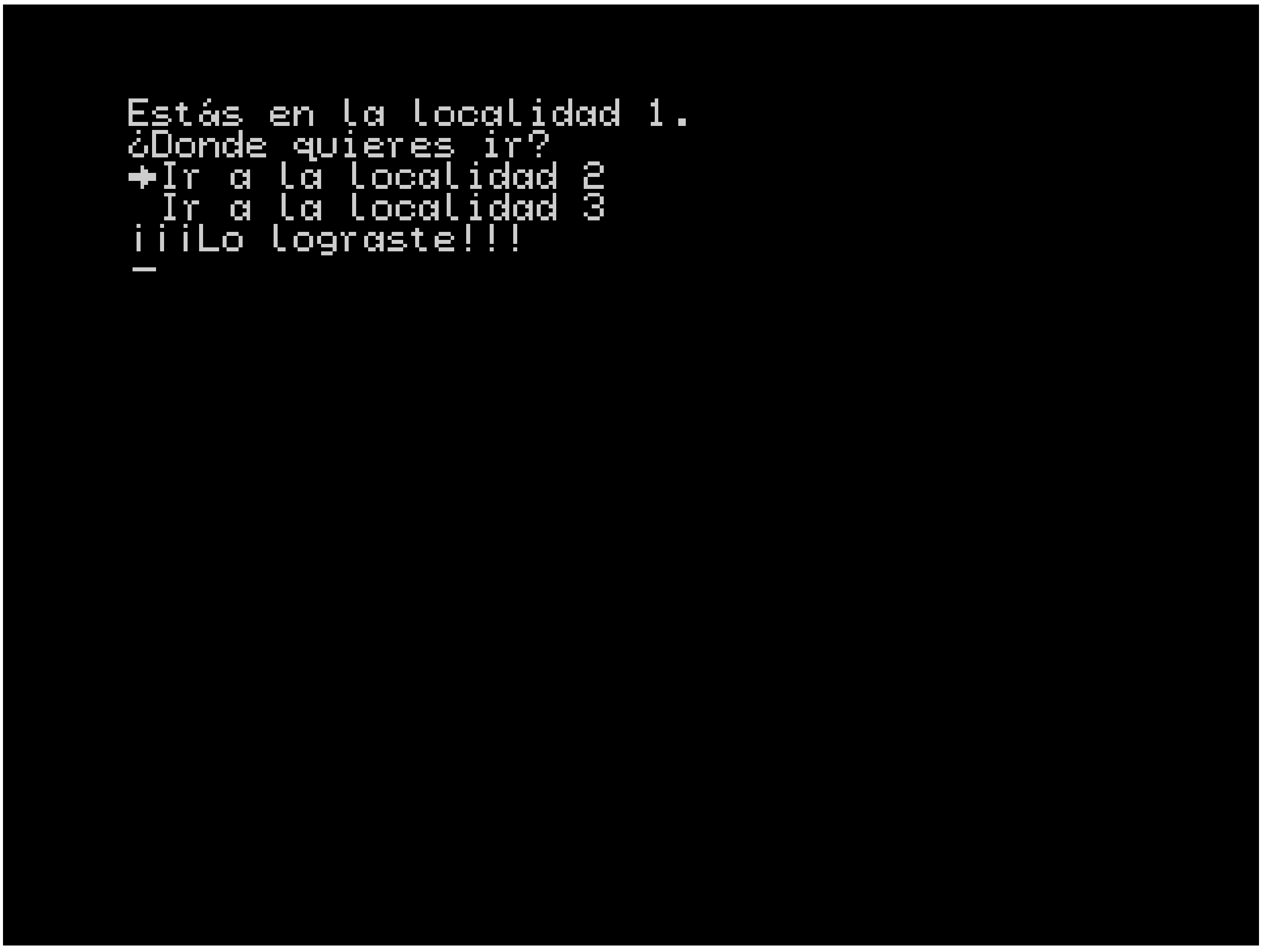


Figure 9: Select the first option

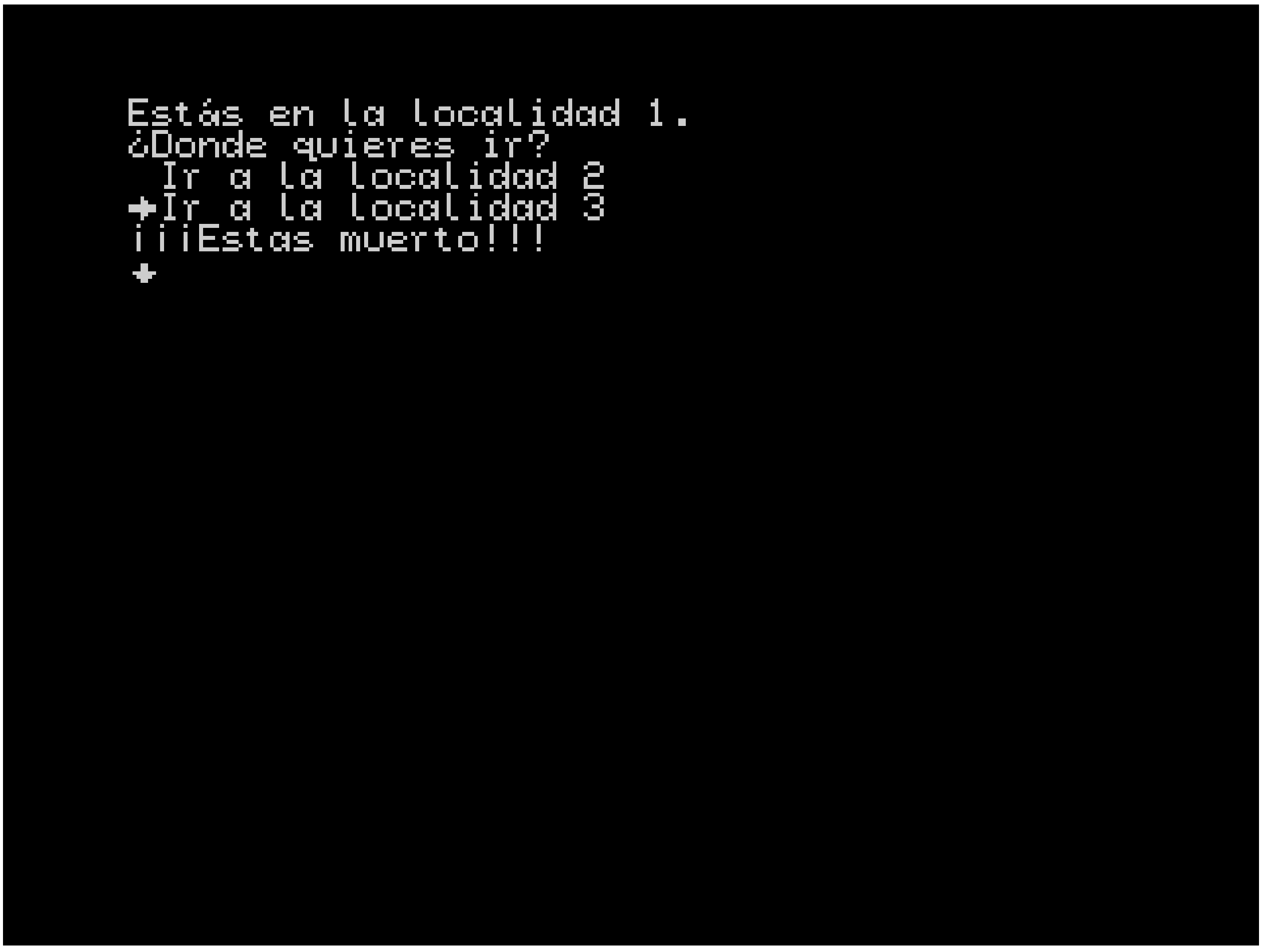


Figure 10: Select the second option

```
[[ GOTO Final ]]
[[ LABEL Localidad3 ]]You are dead!!!
[[ GOTO Final]]
[[ LABEL Final : WAITKEY: END ]]
```

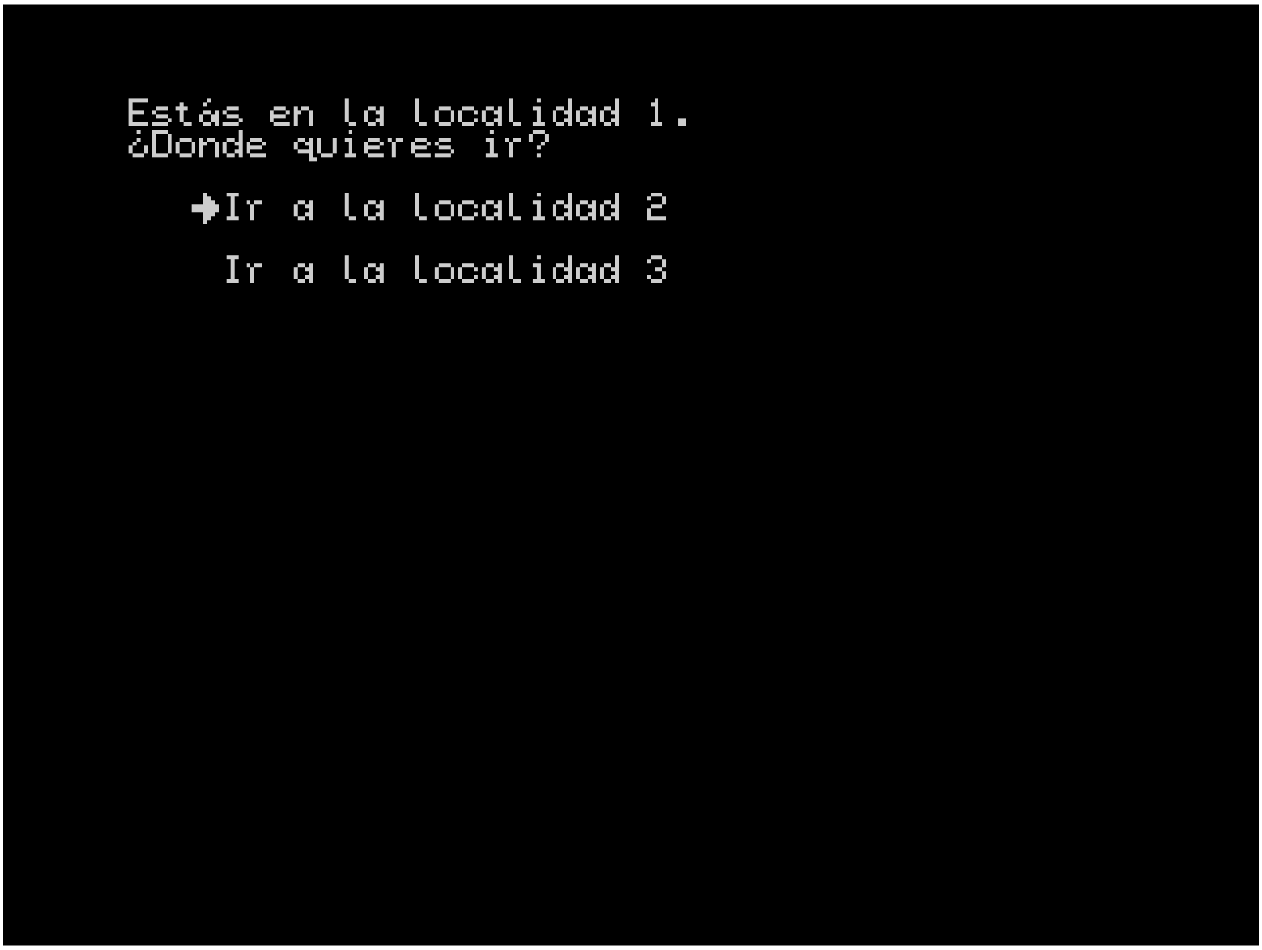


Figure 11: Tidy menu

As you can see, we have separated the options with line breaks and put two spaces of indentation before the `OPTION GOTO` command and this is reflected in the final result.

Once the options have been declared, with the `CHOOSE` command, we activate the menu, which will allow us to choose between one of the options that were already on the screen. When we select one, it will jump to the label indicated in the corresponding `OPTION GOTO`. In the example, if we select the first option, `OPTION GOTO Localidad2`, it will jump to the label `LABEL Localidad2` and print *You did it* and then jump to the label `LABEL Final`. The `GOTO Final` is necessary, because if not, it would print *You did it!!!* and then *You are dead!!!*; the `GOTO` is necessary, in this case, to avoid the result of the second option being executed.

As an additional note with `CHOOSE`, only a maximum of 32 options and a minimum of one are allowed (useless, but allowed). Outside that range the interpreter will give an error.

Also note that there is a timed variant of `CHOOSE`, compile and run this without selecting anything from the menu:

```
[[ /* Set screen colors and clear it */
   PAPER 0    /* Black background color  */
```

```
   INK   7    /* White text color */
   BORDER 0   /* Black border color  */
   CLEAR      /* Clear the screen*/
]][[ LABEL Localidad1]]You are in location 1.
Where do you want to go?

  [[ OPTION GOTO Localidad2 ]]Go to location 2

  [[ OPTION GOTO Localidad3 ]]Go to location 3
[[ CHOOSE IF WAIT 500 THEN GOTO Localidad3]]
[[ LABEL Localidad2 ]]You did it!!!
[[ GOTO Final ]]
[[ LABEL Localidad3 ]]You are dead!!!
[[ GOTO Final]]
[[ LABEL Final : WAITKEY: END ]]
```

You will notice that after about 10 seconds, it has displayed *You are dead!!!*.

What `CHOOSE IF WAIT 500 THEN GOTO Location3` does is the same as `CHOOSE`, activate the selection of options, but with the exception that it also performs a countdown, in this case from 500. If this countdown reaches zero without anything being selected, then the jump is made to the indicated label; in this case *Location3*. The counter works based on the Spectrum frames, that is, 1/50 of a second, so 500/50 = 10 seconds. We will see this in the next chapter.

Again, we could rewrite the previous code like this with the shortened form of the labels:

```
[[ /* Set screen colors and clear it */
   PAPER 0    /* Black background color  */
   INK   7    /* White text color */
   BORDER 0   /* Black border color  */
   CLEAR      /* Clear the screen*/
]][[ #Localidad1 ]]You are in location 1.
Where do you want to go?

  [[ OPTION GOTO Localidad2 ]]Go to location 2

  [[ OPTION GOTO Localidad3 ]]Go to location 3
[[ CHOOSE IF WAIT 500 THEN GOTO Localidad3]]
[[ #Localidad2 ]]You did it!!!
[[ GOTO Final ]]
[[ #Localidad3 ]]You are dead!!!
[[ GOTO Final]]
[[ #Final : WAITKEY: END ]]
```

From now on, I'll be using the shortened form for labels.

With this we already have the basis for making a basic "Choose Your Own Adventure". But we still have many more possibilities to explore…

1.7 Pauses and Waits

In the previous examples you will have already seen the `WAITKEY` command. This command generates a pause, with an animated icon, waiting for the confirmation key to be pressed. With this you can control the display of the text and avoid the user having to read a wall of text in one sitting, as well as allowing you to control the presentation.

An example would be when the "page" is ending and we want the user to press a key to move to the next one, which we can do like this:

```
Text at the end of the page.[[
  WAITKEY
  CLEAR
]]Text at the beginning of the next page.
```

With the `WAITKEY` we wait, and when we confirm, with the following `CLEAR` we erase the text on the screen and start writing from the beginning of what would be the next "page".

However, there is an option for CYD to do this on its own. The default behavior when we finish printing on the last line is to completely clear the screen and continue writing; but with the `PAGEPAUSE` command we can activate an alternative behavior. If we indicate `PAGEPAUSE 1`, for example, when the available space runs out, it will automatically generate a wait for the user to press the confirmation key before clearing the screen and continuing printing.

In addition, there are also "timed" waits, being `CHOOSE IF WAIT X THEN GOTO Y` from the previous chapter an example. When they are executed, a counter is loaded with the value passed as a parameter and a countdown is performed until the counter reaches zero. The counter is decremented once every Spectrum frame, that is, once every 1/50 of a second, or in other words, 50 times per second. So, if we want to wait a second, we have to set the counter to 50.

With this, we already have what is necessary to know the commands:

- With the `WAIT` command, an unconditional wait is performed, it is a stop until the counter runs out.

```
Wait three seconds[[WAIT 150]]
Done[[WAITKEY]]
```

- The `PAUSE` command is a combination of `WAITKEY` and `WAIT`, a wait is performed until the counter runs out or the user presses the confirmation key, we could consider it a `WAITKEY` with expiration.

```
Wait three seconds or press a key[[PAUSE 150]]
Done[[WAITKEY]]
```

- `CHOOSE IF WAIT X THEN GOTO Y` has already been explained in the previous chapter. If the counter runs out before a menu option is selected, the indicated jump is made.

Finally, let's talk about the `TYPERATE` command, which is a bit special compared to the rest of the wait commands. With this command we indicate the wait that occurs each time a character is printed. This wait is not adjusted to frames, but is a counter that depends on the speed of the processor (faster). The idea of this command is to write in a slower and more gradual way, for certain "dramatic" situations.

```
[[TYPERATE 100]]This prints slowly
[[TYPERATE 0]]This prints normally[[WAITKEY]]
```

Text layout on screen

One of the most important parts of designing a CYD adventure is adjusting the presentation of the text. CYD doesn't "know" how to present the text. As authors, we have to help it.

We can visualize its behavior by imagining that there is an invisible cursor on the screen that prints the text from left to right and from top to bottom. The engine always tries to ensure that the words do not break, Let's put a text like *Loren ipsum*, all in one go, without line breaks:

```
[[ /*
Set
screen
col-
ors
and
clear
it */
PAPER
0
/*
Black
back-
ground
color
*/
INK
7
/*
White
text
color
*/
BOR-
DER 0
/*
Black
bor-
der
color
*/
CLEAR
/*
Clear
the
screen*/
PAGEPAUSE
1
]]Lorem
ipsum
dolor
sit
amet,
con-
secte-
tur
adip-
isc-
ing
```

This is the result:

You can see that words that do not fit on their line continue on the next line without being cut off.

When the print cursor reaches the last line and must move to the next line, the screen is cleared and printing continues from the origin of the screen, at the top left; except if you use the `PAGEPAUSE` command, which gener-

We can layout the screen properly using spaces and line breaks as needed, but if we want to make an indent or tabs, you have the `TAB pos` command, which prints as many spaces as indicated in the parameter:

```
[[ /*
Set
screen
col-
ors
and
clear
it */
PAPER
0
/*
Black
back-
ground
color
*/
INK
7
/*
White
text
color
*/
BOR-
DER 0
/*
Black
bor-
der
color
*/
CLEAR
/*
Clear
the
screen*/
PAGEPAUSE
1
]]Nor-
mal
text
[[TAB
5]]Text
5
posi-
tions
to
the
right.[[
```

The `TAB` command allows us to save memory because, if we want to print 10 spaces, those 10 spaces will consume more than using the `TAB 10` command.

Similarly, we have the `REPCHAR` command to print any character repeatedly:

```
[[ /*
Set
screen
col-
ors
and
clear
it */
PAPER
0
/*
Black
back-
ground
color
*/
INK
7
/*
White
text
color
*/
BOR-
DER 0
/*
Black
bor-
der
color
*/
CLEAR
/*
Clear
the
screen*/
PAGEPAUSE
1
]]Nor-
mal
text
[[
TAB 5
]]Text
5
posi-
tions
to
the
```

This repeats the number 35 (the pad character), 10 times. And if we want to print it only once, we have the `CHAR` command:

```
[[ /*
Set
screen
col-
ors
and
clear
it */
PAPER
0
/*
Black
back-
ground
color
*/
INK
7
/*
White
text
color
*/
BOR-
DER 0
/*
Black
bor-
der
color
*/
CLEAR
/*
Clear
the
screen*/
PAGEPAUSE
1
]]Nor-
mal
text
[[
TAB 5
]]Text
5
posi-
tions
to
the
```

And to make a line break, the `NEWLINE` command:

```
[[ /*
Set
screen
col-
ors
and
clear
it */
PAPER
0
/*
Black
back-
ground
color
*/
INK
7
/*
White
text
color
*/
BOR-
DER 0
/*
Black
bor-
der
color
*/
CLEAR
/*
Clear
the
screen*/
PAGEPAUSE
1
]]Nor-
mal
text
[[
TAB 5
]]Text
5
posi-
tions
to
the
```

Let's now look at a command to place the print cursor anywhere on the screen. With the command `AT column,row`, we can indicate the coordinates where we want to place the cursor to continue printing. Let's see it with this

```
[[ /*
Set
screen
col-
ors
and
clear
it */
PAPER
0
/*
Black
back-
ground
color
*/
INK
7
/*
White
text
color
*/
BOR-
DER 0
/*
Black
bor-
der
color
*/
CLEAR
/*
Clear
the
screen*/
PAGEPAUSE
1 ]]
Text
below
[[AT
5,0]]Is
this
at
the
top?.[[
WAIT-
KEY:
END
]]
```

What happened here? If we examine the code, we see that before "Text below", there is a line break. So the cursor jumps to the next line and prints the "Text below", but after that we have `AT 5,0`, which means *move the cur-*

With this we can now place text wherever we want. But we are missing something to fully control the layout of the text on the screen, and that is to define some margins. By default CYD prints the text in full screen,

---

The format of the command is `MARGINS origin_col, origin_row, width, height`, where the parameters are the origin column and row of the print area and the corresponding width and height. By default, the engine starts as if th

Now let's go back to the initial example of this chapter and put it in the lower area of the screen:

```
[[ /*
Set
screen
col-
ors
and
clear
it */
PAPER
0
/*
Black
back-
ground
color
*/
INK
7
/*
White
text
color
*/
BOR-
DER 0
/*
Black
bor-
der
color
*/
CLEAR
/*
Clear
the
screen*/
PAGEPAUSE
1
MAR-
GINS
0,
10,
32,
14
]]Lorem
ipsum
dolor
sit
amet,
```

We have lowered the origin of the print area to row 10, and reduced its height to 14:

With this we can adjust the area where we want to print. Note the effect of `PAGEPAUSE 1`, which, since not everything fits, generates a confirmation icon in the center.

Now we are going to use the same thing in the second example of this chapter:

```
[[ /*
Set
screen
col-
ors
and
clear
it */
PAPER
0
/*
Black
back-
ground
color
*/
INK
7
/*
White
text
color
*/
BOR-
DER 0
/*
Black
bor-
der
color
*/
CLEAR
/*
Clear
the
screen*/
PAGEPAUSE
1
MAR-
GINS
0,
10,
32,
14 ]]
Text
below
[[AT
5,0]]Is
this
```

Do you notice anything strange?

If you haven't noticed, the coordinates of `AT` are not, in this case, the screen coordinates. The coordinates of `AT` are **always relative to the origin of the print area**. When we had the area defined as the full screen, The Spectrum screen has, naturally, 32 characters per line, which is somewhat insufficient for long texts. CYD supports variable width fonts and the default one uses 6x8 characters, which allows us to have 42 characters

For this reason, `AT` and `MARGINS` have a character-based coordinate system of 8x8 pixels, that is, 32 columns and 24 rows, counted from 0 to 31 and 0 to 23 respectively.

Finally, let's look at an attribute collision problem with this example:

```
[[ /*
Set
screen
col-
ors
and
clear
it */
PAPER
0
/*
Black
back-
ground
color
*/
BOR-
DER 0
/*
Black
bor-
der
color
*/
CLEAR
/*
Clear
the
screen*/
PAGEPAUSE
1 INK
7
/*
Blue
text
color
*/
]]LALALALALA[[INK
4 /*
Green
color
*/]]
LOLOLOLOLOLO[[
WAIT-
KEY:
END
]]
```

You can see that when you change the color to green with the `INK` command, because the next character to be printed (a space) is between two attribute cells, the previous character is painted white:

We can solve this by changing the color after the space:

```
[[ /*
Set
screen
col-
ors
and
clear
it */
PAPER
0
/*
Black
back-
ground
color
*/
BOR-
DER 0
/*
Black
bor-
der
color
*/
CLEAR
/*
Clear
the
screen*/
PAGEPAUSE
1 INK
7
/*
White
text
color
*/
]]LALALALALA
[[INK
4
/*Green
color
*/]]LOLOLOLOLOLO[[
WAIT-
KEY:
END
]]
```

And now it's correct:

The author will need to make these small adjustments to improve the presentation of the text. If you want to avoid them completely, do not mix different colors within the same line.

1.8 Images

To make the adventure more attractive, it is possible to add images in SCR format, of Spectrum screens. These images will be compressed with the `CSC` utility, creating files with the same extension, which must be included in the final disk.

The manual explains how `CSC` works, in case you want to do it manually. But the `MakeAdv` script automatically searches for and compresses the SCR files that are in the `\IMAGES` directory, so you will simply have to place the files there.

We are going to use an image that we have as an example, called `ORIGIN1.SCR`, inside the `\examples\test\IMAGES` directory. Copy it and rename it as `001.SCR`. And put the following code in `tutorial.txt`:

```
[[ /* Set screen colors and clear it */
   PAPER 0    /* Black background color  */
   BORDER 0   /* Black border color  */
   CLEAR      /* Clear the screen*/
   INK   7    /* White text color */
   PAGEPAUSE 1
]]I'm going to load screen 1.[[
   WAITKEY
   /* Load the image from file 001.CSC */
   PICTURE 1]]
I'm going to display the image.[[
   WAITKEY
   /* Display the loaded image  */
   DISPLAY 1
]]
Done[[ WAITKEY: END ]]
```

Before launching the emulator, look in the `\IMAGES` directory, where you will find `001.SCR` and `001.CSC`. The `MakeAdv` script looks for files named xxx.SCR, where the three x's are digits, and compresses them with `CSC`, generating the corresponding files with the extension `.CSC`. The engine will search the disk for files with this name when asked to load images.

Now we can launch the emulator. The first thing you will see is this:

When you press the confirm key, the `PICTURE 1` command will be launched. What this command will do is load and decompress the image file `001.CSC` into memory, but be careful, it is not displayed yet! You will notice that the disk has been accessed (this depends on the emulator).

With this we have the image loaded, but to display it, we have to use the `DISPLAY 1` command, and then the image is displayed:

We can now display images, but there are a few things to clear up first. The first thing you might be wondering is… what is the 1 in DISPLAY for? As stated in the reference, the `DISPLAY` command needs a parameter indicating whether to display the image or not; if the value is zero, it does not display it, and if it is non-zero, it does. This may seem pointless, but it makes sense if used with variables, to cause the image to be displayed conditionally based on the value of a variable.

Another thing you might be wondering is why are the commands to load the image and display it

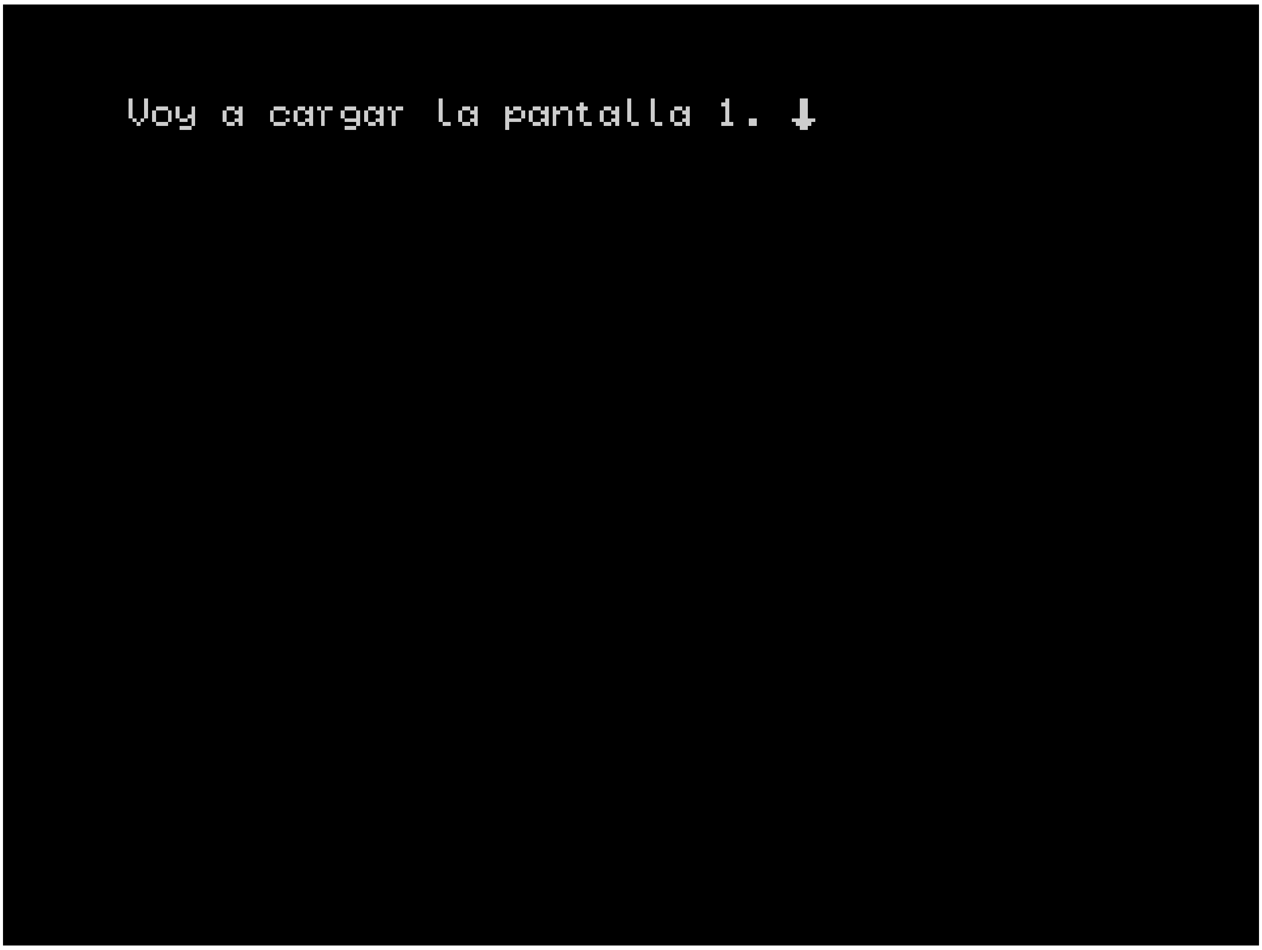


Figure 12: With image

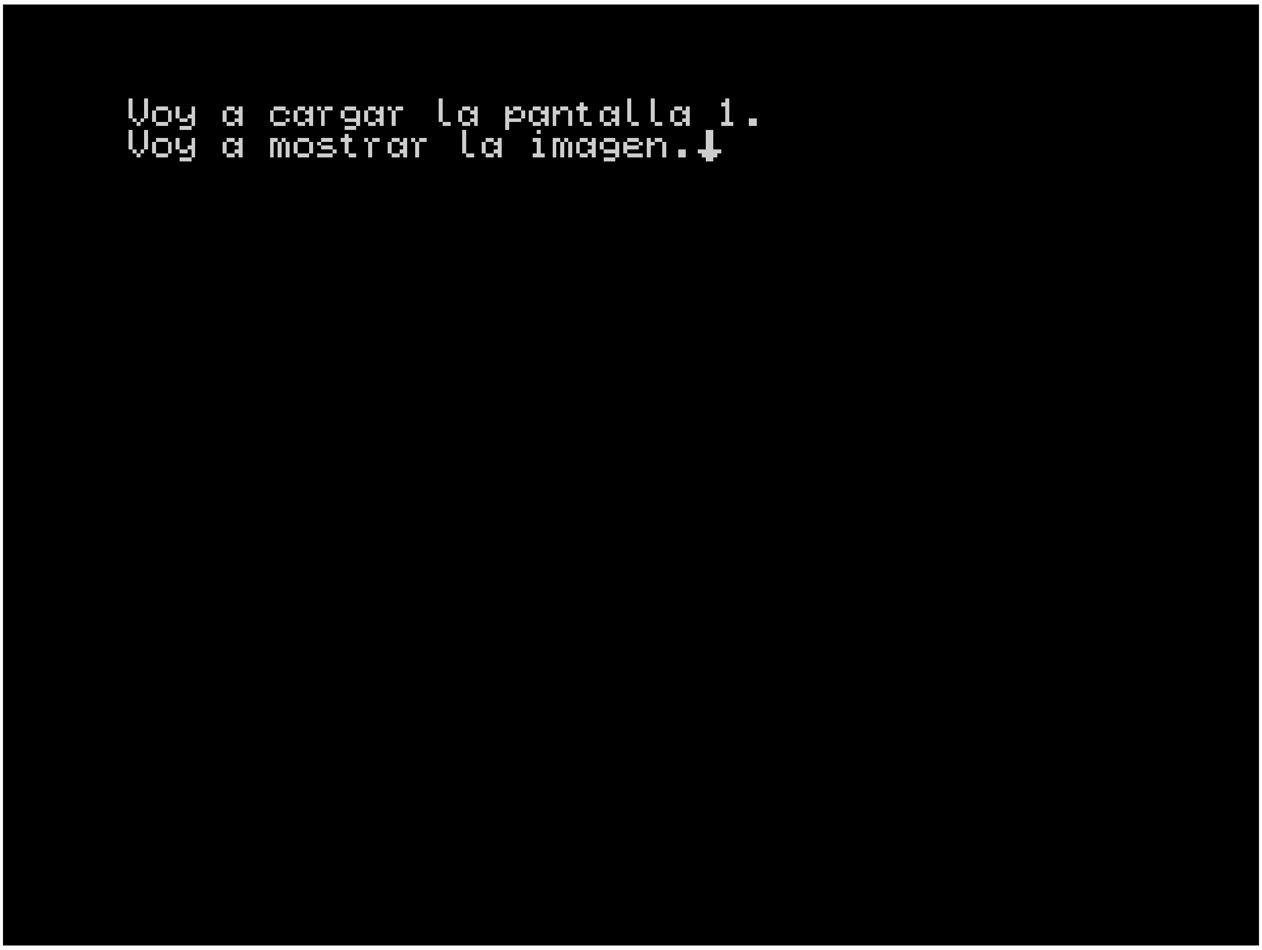


Figure 13: Loading image

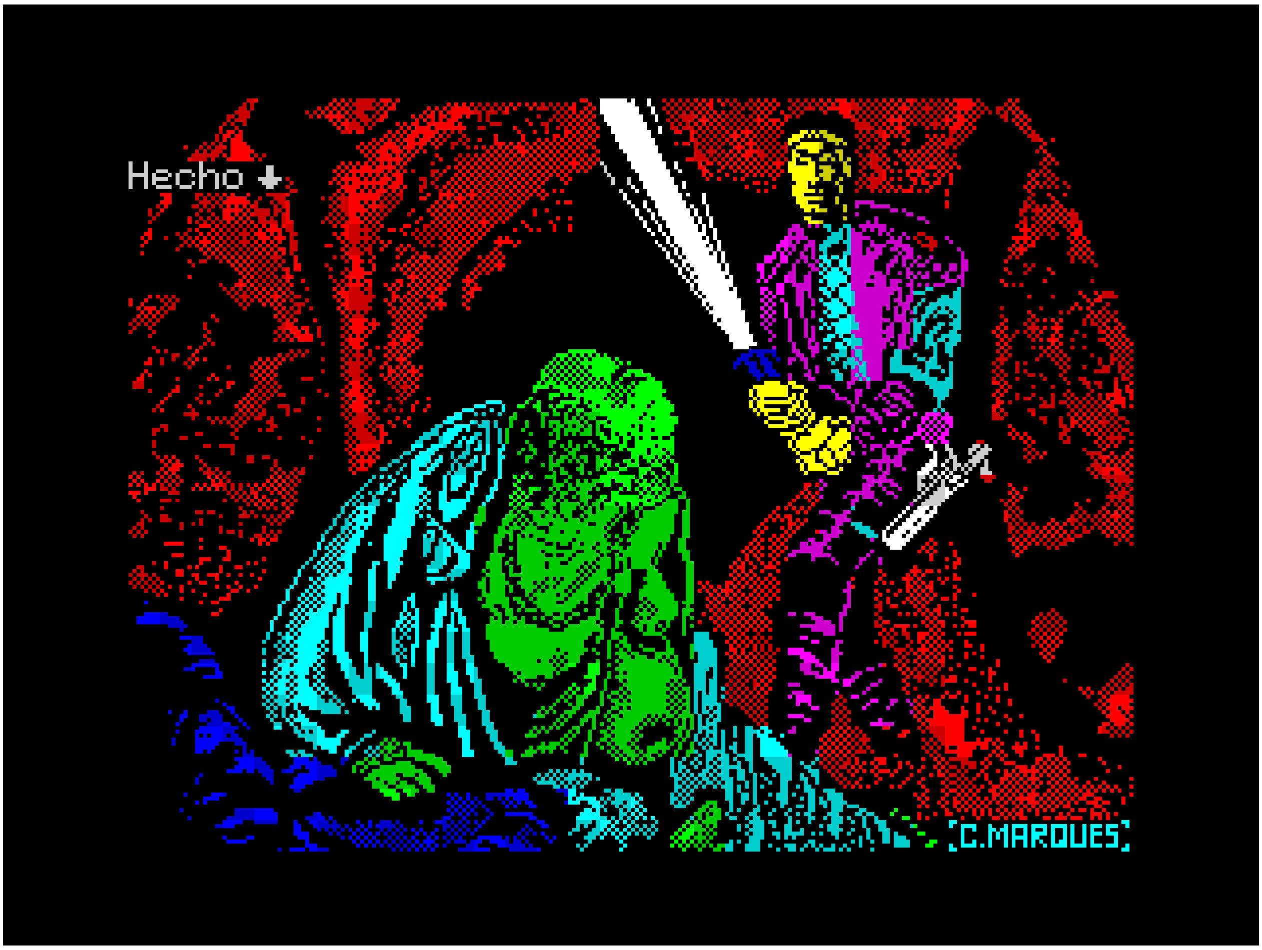


Figure 14: Showing image

separate, instead of using a single command to do both? Well, the answer is a design decision for the disk version, since by separating the loading into a separate operation, we can control when it is done to, for example, load it when a chapter starts, and then display the image at the most opportune moment, since loading will stop the engine and pause the reading at an undesirable moment.

For now, just remember that you first need `PICTURE 3`, to load the image `003.CSC`, for example, and then `DISPLAY 1` to display it. Note that we can only load one image at a time, so if we load another image, the one already loaded will be deleted, and a loaded image can be displayed as many times as we want. And you'll get a nice error if you try to load an image that doesn't exist on disk or in memory, or when displaying an image without loading it first.

When displaying images, keep in mind that whatever is already on the screen will always be overwritten. The default behavior is to load images full screen (192 lines), but you can edit the number of lines to load by modifying the value of the `IMGLINES` variable in the `make_adv.cmd` script:

```
REM Number of lines used on SCR files at compressing
SET IMGLINES=192
```

Due to the color limitations of the Spectrum, I recommend that it always be a multiple of 8.

When using images, we can adjust the size of the print area so that it does not overwrite the entire drawing using `MARGINS`. Let's combine two previous examples to see this:

```
[[ /* Set screen colors and clear it */
   PAPER 0    /* Black background color  */
   INK   7    /* White text color */
   BORDER 0   /* Black border color  */
   CLEAR      /* Clear the screen*/
   PAGEPAUSE 1
   PICTURE 1
   DISPLAY 1
   MARGINS 0, 10, 32, 14
]]Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nam eget pretium felis. Quisque tincidunt tortor
```

The first thing it will do is display the image, which will overwrite the entire screen, and then we will define a lower area where the text will be drawn:

Since the text does not fit, clicking continue will delete the printing area defined with `MARGINS` and continue printing the rest:

To make the behavior a little more consistent, let's put a `CLEAR` right after `MARGINS`:

```
[[ /* Set screen colors and clear it */
   PAPER 0    /* Black background color  */
   INK   7    /* White text color */
   BORDER 0   /* Black border color  */
   CLEAR      /* Clear the screen*/
   PAGEPAUSE 1
   PICTURE 1
   DISPLAY 1
```

Figure 15: Showing image and text

Figure 16: Showing image and text

```
   MARGINS 0, 10, 32, 14
   CLEAR
]]Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nam eget pretium felis. Quisque tincidunt tortor
```

And with this it's better:

Figure 17: Showing image and text

Here we've used a random image, more specifically the loading screen from the *La Adventura Original*. But if we're going to do this kind of cropping with all images, we can change the value of the `IMGLINES` variable, so that it crops out the lines we don't want. This will save space, and prevent the bottom text area from being overwritten when we load a new image.

For image management and taking into account these restrictions, it's important to plan in advance how and when we're going to display them.

1.9 Sound effects (Beeper)

To enhance the atmosphere of our adventure, the engine allows us to play sound effects through the Beeper. To do this we use the tool BeepFx from Shiru, a tool widely used in new developments for Spectrum.

In this tutorial we are not going to teach how to use the tool, but we are going to take one of the example files included in the package. From the menu `File -> Open project`, we open the file `demo.spj`, where there are a series of effects already created. We can reproduce them with the option

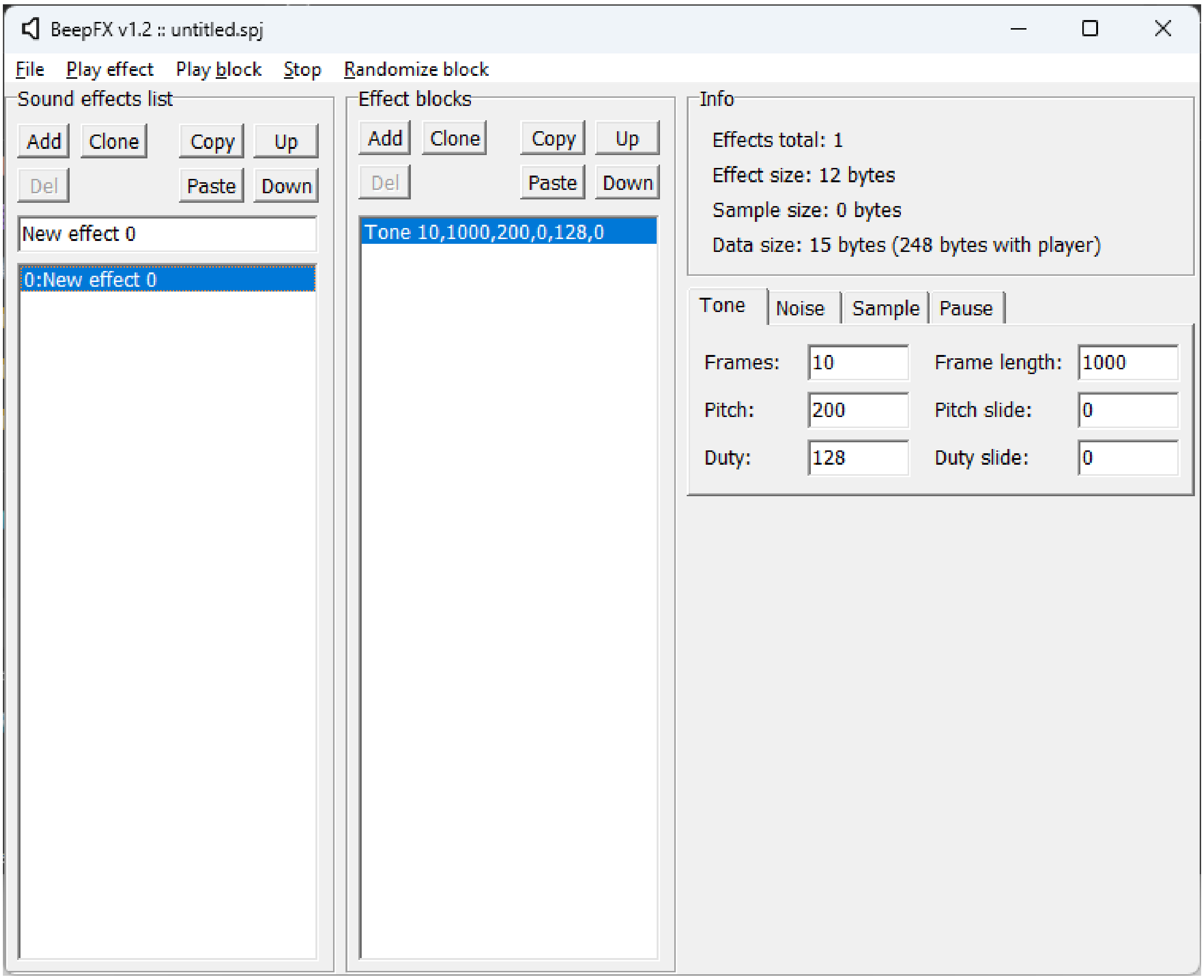


Figure 18: BeepFx

`Play Effect:`

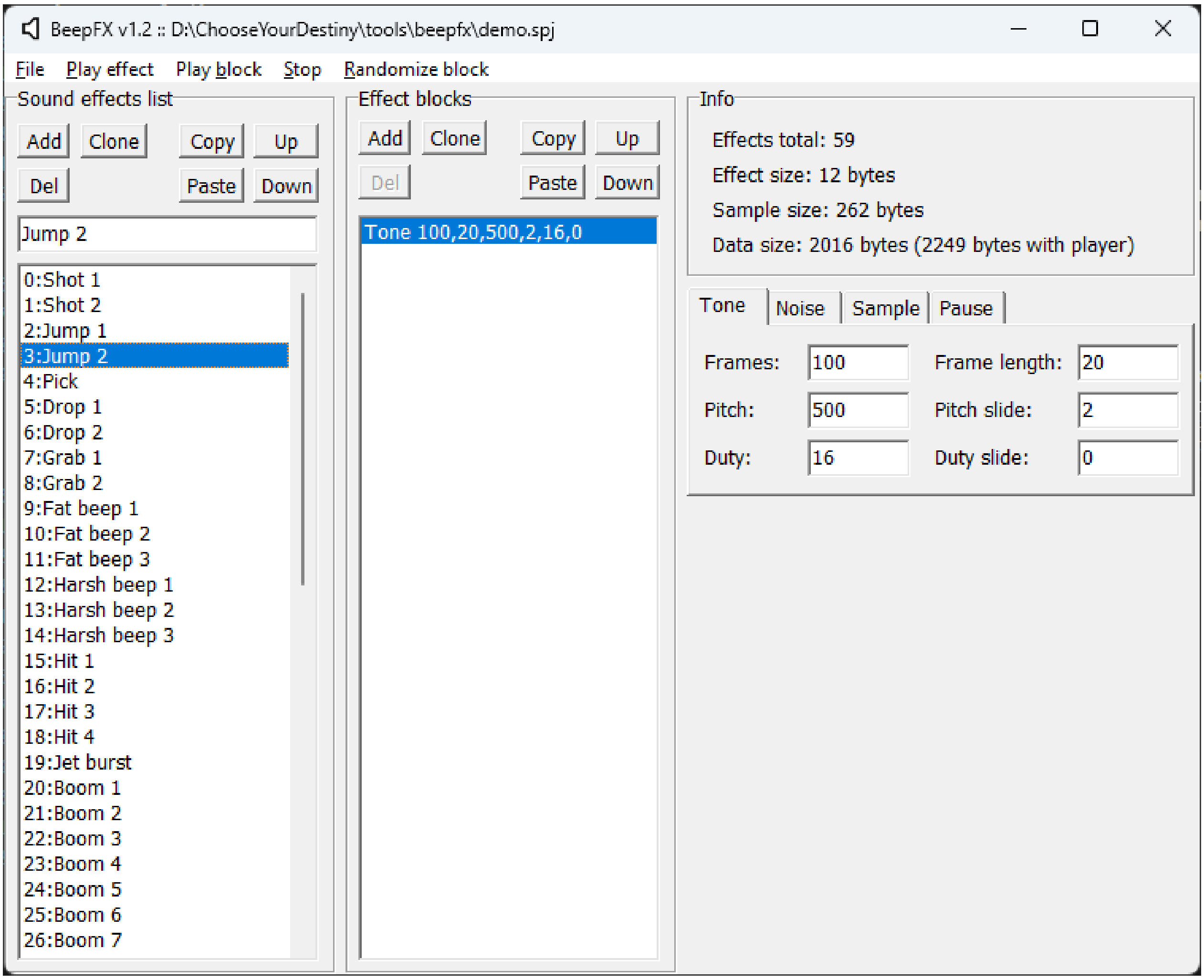


Figure 19: Example BeepFx

Now we are going to export it to a file with which we can use it with the engine. To do this, go to the menu `File -> Compile`, where this window will appear:

Now comes the important part, **we must always leave the `Assembly` and `Include player code` options checked**:

We click on the `Compile` button and a dialog box will appear to save the file. **We must call it `SFX.ASM`** and save it in the folder where we are developing our adventure. When we run the script `make_adv.cmd`, if it finds the file `SFX.ASM` in the same directory, it will automatically include it in the resulting file and we can use it from the engine.

Let's give an example, put this as the code for the adventure:

```
[[ /* Set screen colors and clear it */
   PAPER 0    /* Black background color  */
   BORDER 0   /* Black border color  */
   INK   7    /* White text color */
   PAGEPAUSE 1
   LABEL Menu
   CLEAR]]Select the effect to play:
```

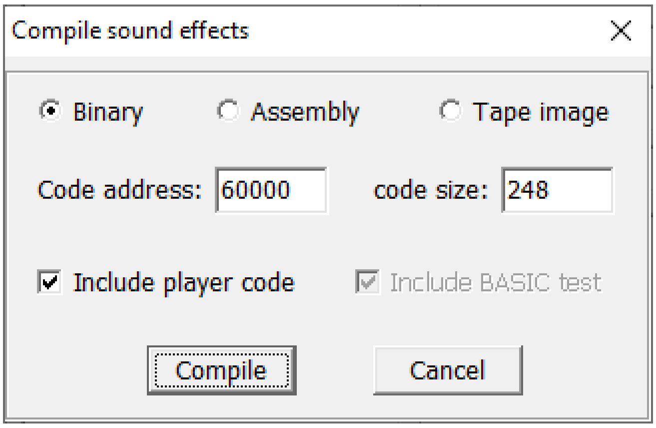


Figure 20: Export from BeepFx

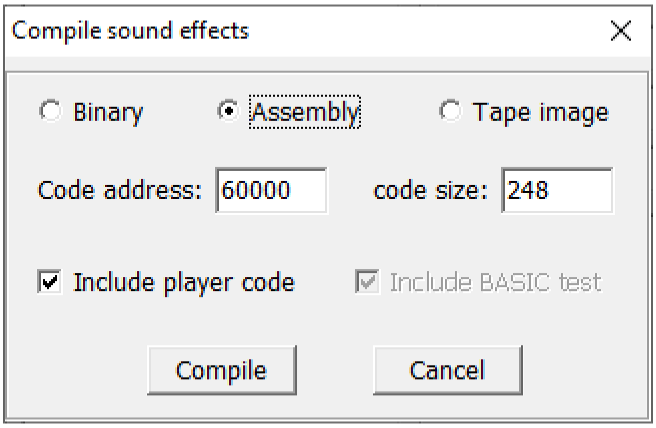


Figure 21: Options for BeepFx

```
  [[ OPTION GOTO Efecto0 ]]Effect 0
  [[ OPTION GOTO Efecto1 ]]Effect 1
  [[ OPTION GOTO Efecto2 ]]Effect 2
  [[ OPTION GOTO Efecto3 ]]Effect 3
  [[ OPTION GOTO Efecto4 ]]Effect 4
  [[ OPTION GOTO Final ]]Exit

[[ CHOOSE
   LABEL Efecto0 : SFX 0 : GOTO Menu
   LABEL Efecto1 : SFX 1 : GOTO Menu
   LABEL Efecto2 : SFX 2 : GOTO Menu
   LABEL Efecto3 : SFX 3 : GOTO Menu
   LABEL Efecto4 : SFX 4 : GOTO Menu
   LABEL Final]]Goodbye...[[WAITKEY: END ]]
```

As you can see, with the `SFX` command, we can play any of the effects in the file by specifying its number as a parameter.

A peculiarity of the SFX command is that, unlike images, if the SFX.BIN file is not on the disk, it will silently fail without giving an error and the sound effect will simply not be played.

1.10 Versions for the different Zx Spectrum models

So far, we have only developed adventures for the original 48k Zx Spectrum, however, the 128k models can also be played on tape or disk. The disk support has the peculiarity that the pictures and melodies are not stored in memory, but are loaded dynamically from disk when the `PICTURE` and `TRACK` commands are executed.

The available space will vary depending on the adventure and the extra resources to be used, since if music and sound effects are omitted, the size of the interpreter will be reduced, leaving more space for text and images. On average, you should expect to have about 96 Kb available on the 128K models and about 24 Kb on the 40K models. For this reason, the 48K version does not include support for AY melodies.

If you use the `make_adv.cmd` script, to change the model, you simply have to change the `TARGET` variable to the values `48k` or `128k` if you want to generate the TAPs for the corresponding models. For example, if we want to create a version of our tutorial for Spectrum 128k:

```
@echo off  &SETLOCAL

REM ---- Configuration variables

REM Name of the game
SET GAME=Tutorial
REM This name will be used as:
REM   - The file to compile will be test.txt with this example
REM   - The name of the TAP file or +3 disk image
```

```
REM Target for the compiler (48k, 128k for TAP, plus3 for DSK)
SET TARGET=128k

REM Number of lines used on SCR files at compressing
SET IMGLINES=192

REM Loading screen
SET LOAD_SCR=%~dp0\IMAGES\000.scr
```

With the `plus3` model, the output format will be a disk image in DSK format.

1.11 Music (AY)

CYD also allows playing music using the AY chip, using modules created with Vortex Tracker, in `PT3` format. AY music is not available for the 48K models, so we will need to change the `TARGET` variable in `made_adv.cmd`, as described in the previous chapter.

The operation is intentionally similar to that of the images. The module file names have to be 3-digit numbers with the extension `.PT3`, so that they are `000.PT3`, `001.PT3` and so on, and they must be included on the disk. To make things easier, the `make_adv.cmd` script will do it for us with all the files that follow this nomenclature and are inside the `.\TRACKS` folder.

The commands we have are also similar to the image management commands. With the `TRACK` command, we load a music module from disk into memory, so that with `TRACK 0`, we would load the module `000.PT3`, with `TRACK 1` we would load `001.PT3`, etc.

Once the module is loaded, we would play it with the `PLAY 1` command when we need to do so. The parameter of the `PLAY` command is a number that, if it is zero, stops the music (if it is already playing), and if it is not zero, plays it from the beginning (if it is already stopped).

Finally, with the `LOOP` command we indicate whether we want the module to play only once or we want it to play indefinitely. If the value of its parameter is zero, it will only play once, and if it is non-zero, it will start playing again when it finishes.

One detail to keep in mind is that the effect of the `LOOP` command is specific to the built-in player and is only valid when the module can finish. The `.PT3` format has repeat commands, making the module play endlessly by itself, so it is convenient to play it with an external player to see if this is the case.

1.12 Variables

With what we already know, we could already make a relatively simple adventure based on options, like “Choose Your Own Adventure”, but we can go further…

The engine has 256 variables or flags of a byte, that is, 256 warehouses where we can store values from 0 to 255. Each of these variables is identified in turn by a number from 0 to 255. Let’s see it with an example:

```
[[ /* Set screen colors and clear it */
   PAPER 0    /* Black background color  */
   BORDER 0   /* Black border color  */
   INK   7    /* White text color */
   PAGEPAUSE 1
   CLEAR]][[SET 1 TO 5]]You have [[PRINT @1]] gamusinos.
[[WAITKEY : END]]
```

This is the result:

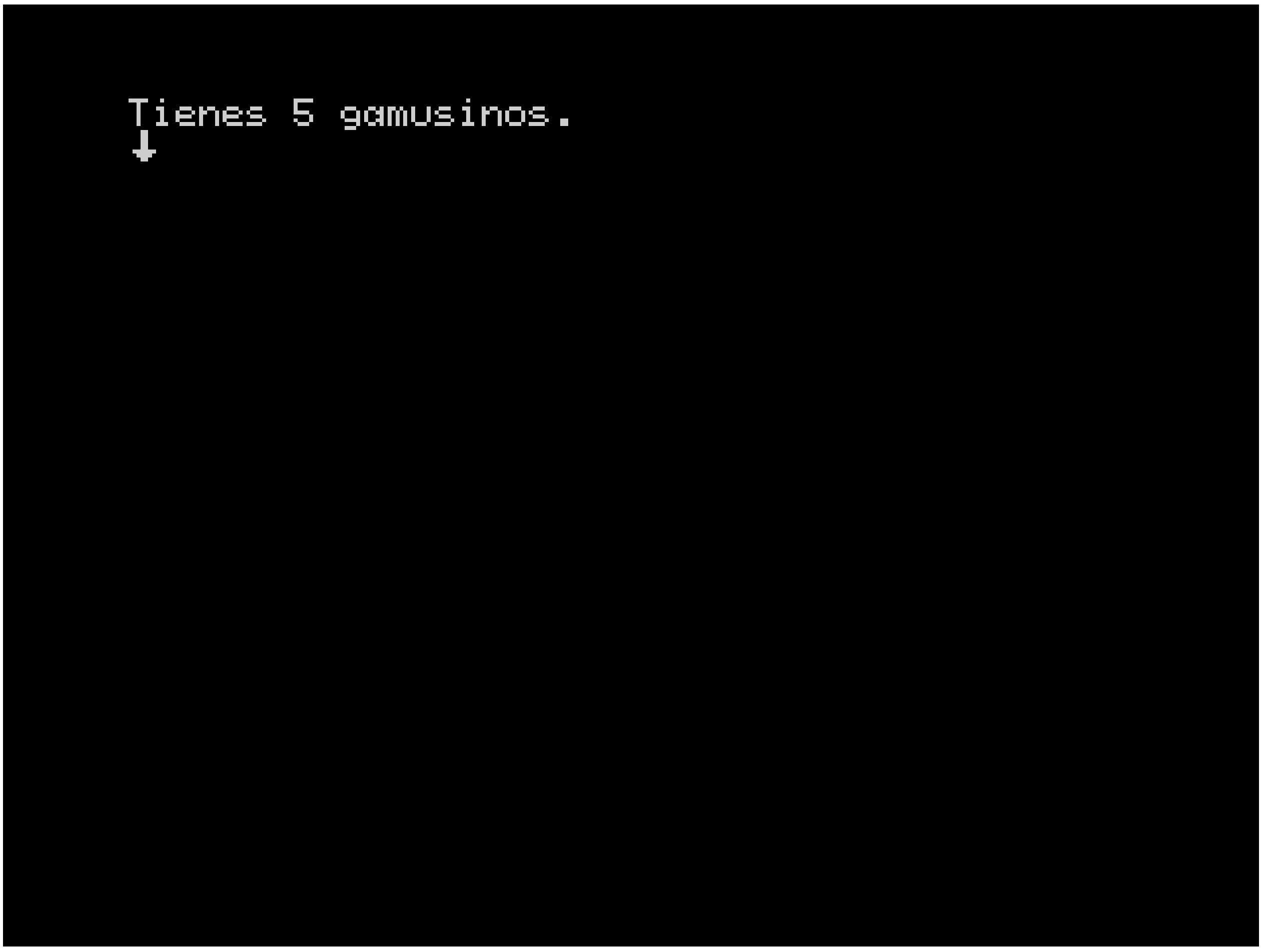


Figure 22: Variables

The first thing we see new is this `SET 1 TO 5`, with this we are telling the engine to save the value 5 inside the variable number 1. And as a consequence, with `PRINT @1` we are telling it to display the value of variable 1 on the screen.

One thing you might have noticed is that `PRINT` puts an at sign in front of the variable number. The at sign is the variable indicator. To explain it better, remove the at sign so that it looks like this:

```
[[ /* Set screen colors and clear it */
   PAPER 0    /* Black background color  */
   BORDER 0   /* Black border color  */
   INK   7    /* White text color */
   PAGEPAUSE 1
   CLEAR]][[SET 1 TO 5]]You have [[PRINT 1]] gamusinos.
```

```
[[WAITKEY : END]]
```

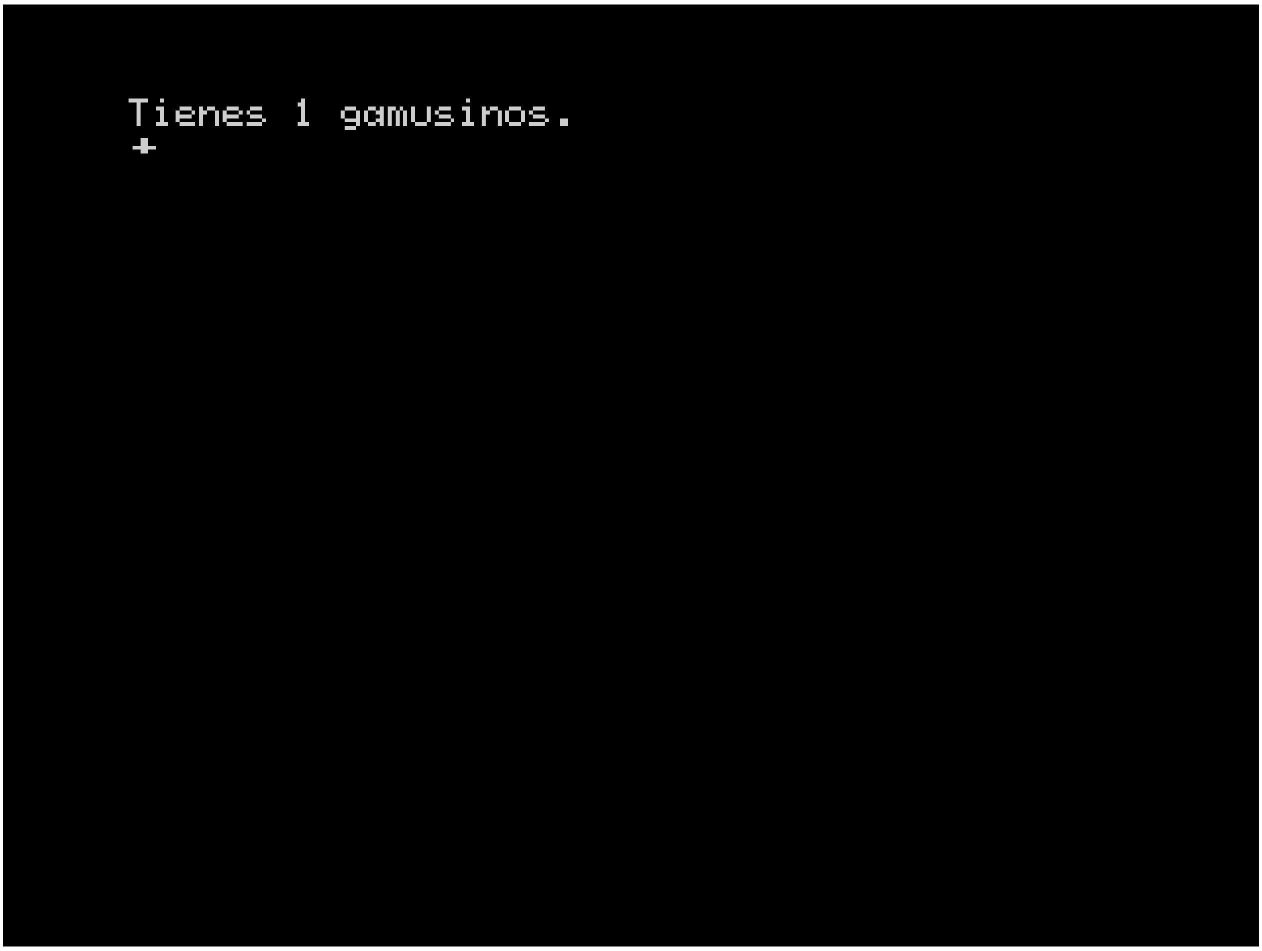


Figure 23: Variables2

Wow! The thing is that when you put an @ sign in front, it means **take the value of the variable whose number I indicate after it**. If you have consulted the manual, you will have seen that almost all commands allow expressions with variables.

Therefore, with `PRINT 1`, what you are indicating is "Print the value 1", but with `PRINT @1`, what is indicated is "Print the content of the variable 1".

Let's reinforce this concept with this example:

```
[[ /* Set screen colors and clear it */
   PAPER 0    /* Black background color  */
   BORDER 0   /* Black border color  */
   INK   7    /* White text color */
   PAGEPAUSE 1
   CLEAR
   SET 1 TO 5
   SET 0 TO @1
]]You have [[PRINT @0]] gamusinos.
[[WAITKEY : END]]
```

Again, we have 5 gamusinos:

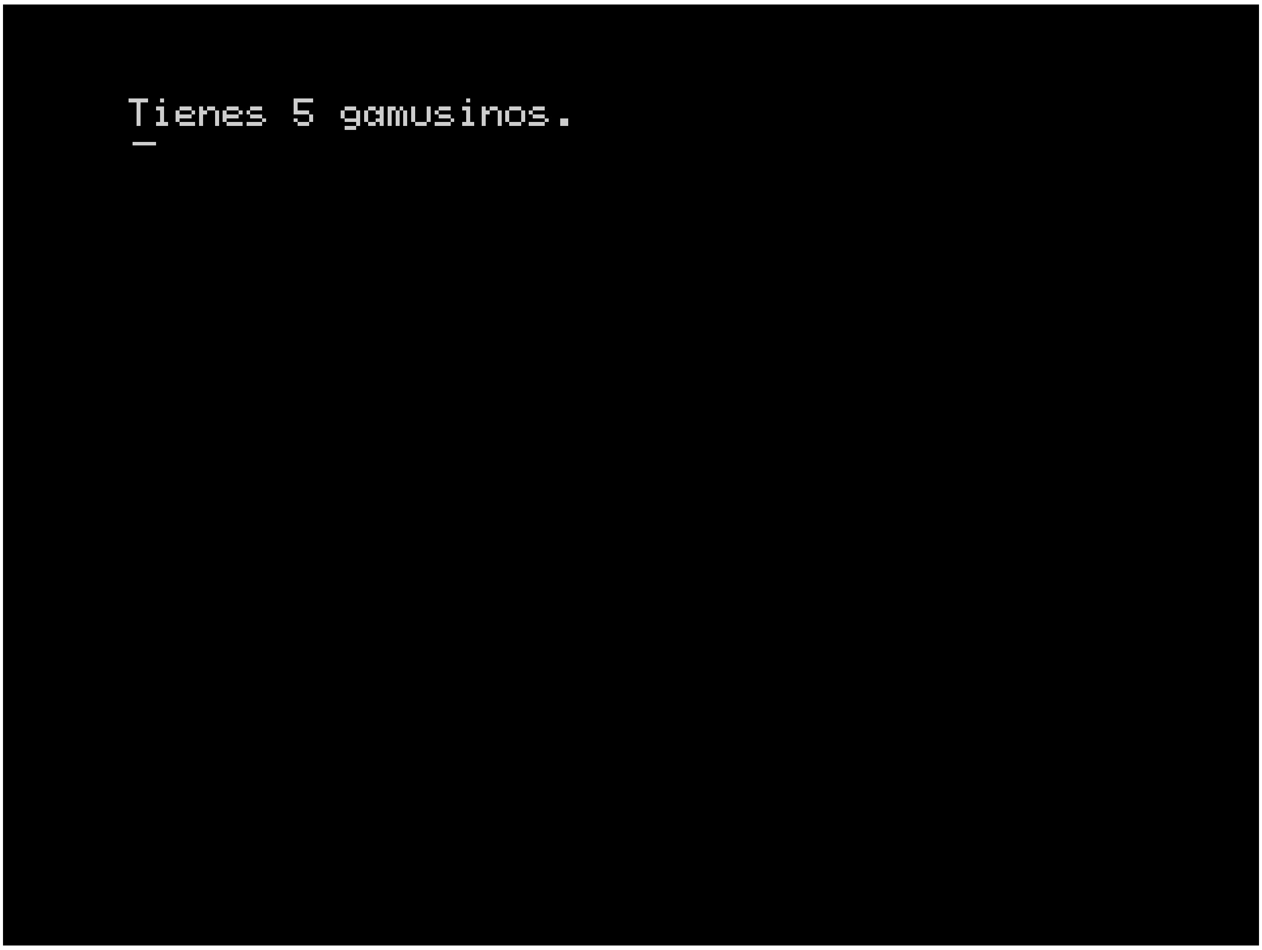


Figure 24: Variables3

With `SET 1 TO 5` we store 5 in the variable num. 1. Then with `SET 0 TO @1`, what we do is store in the variable number 0 the value that variable number 1 contains and finally, with `PRINT @0`, we display the content of variable number 0.

Now we are going to see a more practical example of variables, which will test our knowledge acquired in this tutorial:

```
[[ /* Set screen colors and clear it */
   PAPER 0    /* Black background color  */
   BORDER 0   /* Black border color  */
   INK   7    /* White text color */
   PAGEPAUSE 1
   SET 0 TO 0
   #Inicio
   CLEAR
]]You have [[PRINT @0]] gamusinos.
You need 10 gamusinos to be able to leave...
What do you do?

[[OPTION GOTO Suma1]]I take 1 gamusino.
[[OPTION GOTO Resta1]]I drop 1 gamusino.
[[
   IF @0 <> 10 THEN GOTO Escoger
   OPTION GOTO Final]]Exit.
[[
   #Escoger
   CHOOSE
   #Suma1
   SET 0 TO @0 + 1
   GOTO Inicio
   #Resta1
   SET 0 TO @0 - 1
   GOTO Inicio
   #Final]]Thanks for playing![[WAITKEY : END]]
```

It shows us this menu:

If we choose the first option, it adds 1 to the variable 0, which is done by the command `SET 0 TO @0 + 1`, and the second option subtracts, and it does so with `SET 0 TO @0 - 1`.

The first thing that may catch your attention is that if I hit subtract when the value is zero, it does nothing. This is correct, you cannot subtract below zero or add above 255 in the variables.

And the second, where is the exit option? We are going to catch gamusinos until we have 10, as indicated:

Now we can exit! The secret is in the conditionals. If we look at the exit option, we see that before it there is `IF 0 <> 10 THEN GOTO Choose`, which means “If the variable zero is not equal to 10, jump to the ‘Choose’ label”, which causes the “Exit” option to be skipped and not reflected in the menu until the value of the variable 0 is 10.

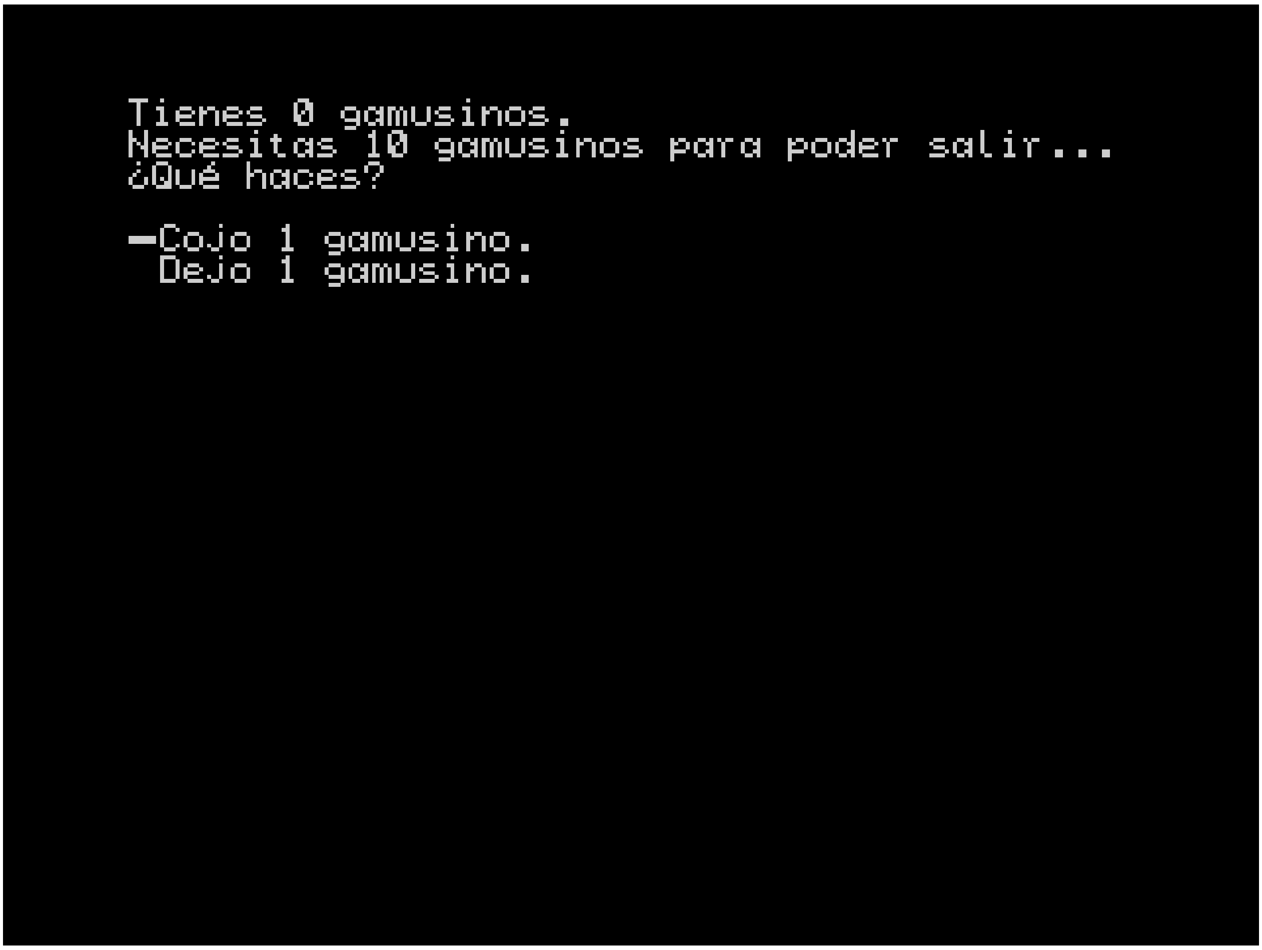


Figure 25: Add and subtract

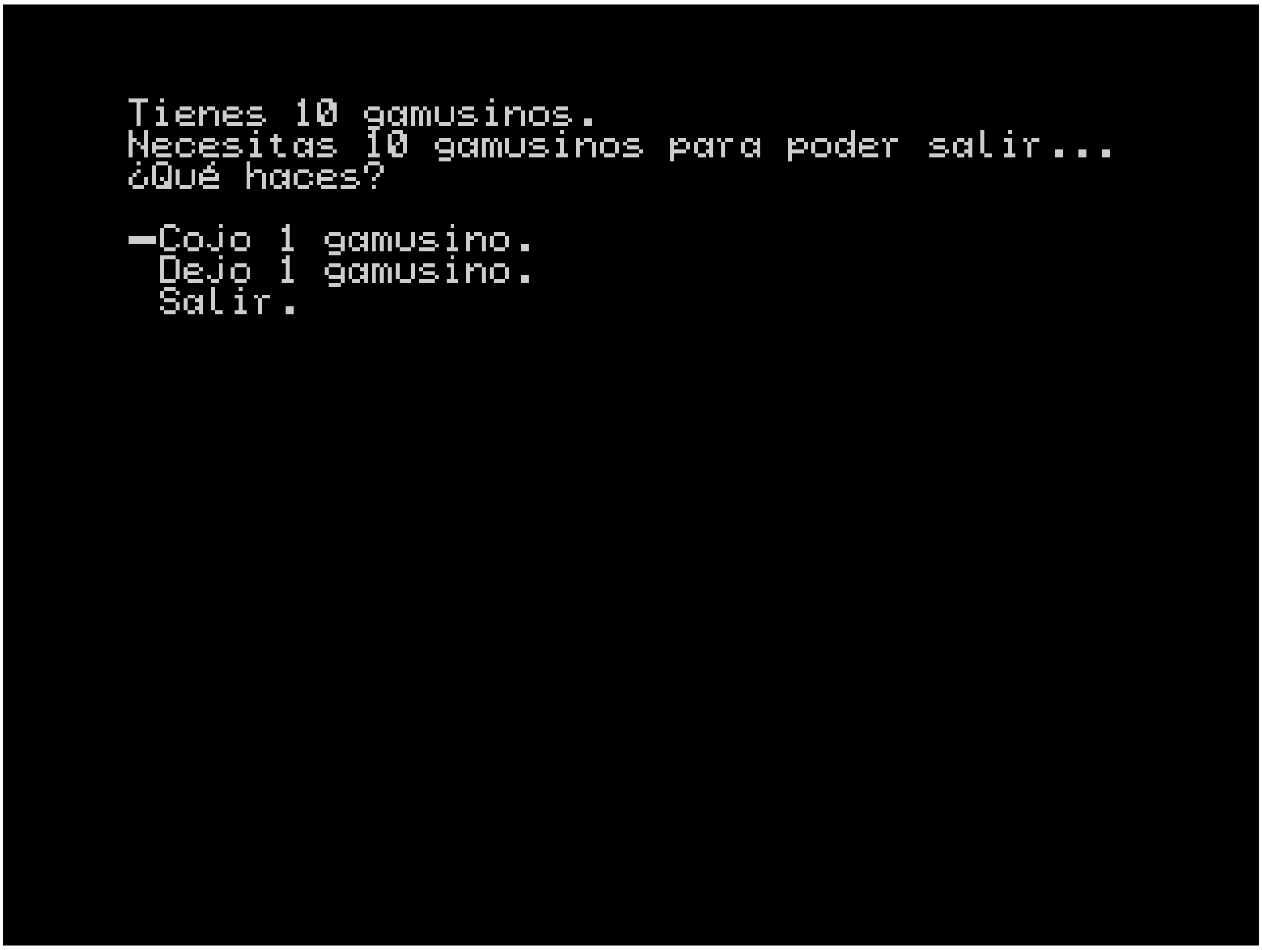


Figure 26: Conditionals

That is, with variables and conditions, we have the necessary tools to make menus of options or texts that vary depending on certain conditions and make our adventure more dynamic.

1.13 Variable declaration

Looking at this program, it can seem quite cryptic:

```
[[ /* Set screen colors and clear it */
   PAPER 0    /* Black background color  */
   BORDER 0   /* Black border color  */
   INK   7    /* White text color */
   PAGEPAUSE 1
   SET 0 TO 0
   #Inicio
   CLEAR
]]You have [[PRINT @0]] gamusinos.
You need 10 gamusinos to be able to leave...
What do you do?

[[OPTION GOTO Suma1]]I take 1 gamusino.
[[OPTION GOTO Resta1]]I drop 1 gamusino.
[[
   IF 0 <> 10 THEN GOTO Escoger
   OPTION GOTO Final]]Exit.
[[
   #Escoger
   CHOOSE
   #Suma1
   SET 0 TO @0 + 1
   GOTO Inicio
   #Resta1
   SET 0 TO @0 - 1
   GOTO Inicio
   #Final]]Thanks for playing![[WAITKEY : END]]
```

The first thing we are going to do is declare a variable, that is, give a name or alias to one of the flags so we can refer to it by its name. Let's take a look at it:

```
[[ /* Set screen colors and clear it */
   PAPER 0    /* Black background color  */
   BORDER 0   /* Black border color  */
   INK   7    /* White text color */
   PAGEPAUSE 1
   DECLARE 0 AS NumGamusinos
   SET NumGamusinos TO 0
   #Inicio
   CLEAR
```

```
]]You have [[PRINT @NumGamusinos]] gamusinos.
You need 10 gamusinos to be able to leave...
What do you do?

[[OPTION GOTO Suma1]]I take 1 gamusino.
[[OPTION GOTO Resta1]]I drop 1 gamusino.
[[
   IF 0 <> 10 THEN GOTO Escoger
   OPTION GOTO Final]]Exit.
[[
   #Escoger
   CHOOSE
   #Suma1
   SET NumGamusinos TO @NumGamusinos + 1
   GOTO Inicio
   #Resta1
   SET NumGamusinos TO @NumGamusinos - 1
   GOTO Inicio
   #Final]]Thanks for playing![[WAITKEY : END]]
```

This is much better, with `DECLARE 0 AS NumGamusinos` we are telling the compiler that, from now on, the variable 0 will be called NumGamusinos, and we can use that name instead. With this we can give a meaningful name to the flags. Also keep in mind the following:

- We can still refer to the variable by its number.
- We cannot declare a variable and a label with the same name.
- We cannot declare two different variables with the same name.
- We can give two different names to the same variable.

1.14 Subroutines

Subroutines are a special type of jump, which stores the position from which it was called, and which can be recovered later, allowing you to continue from the same point where you entered. This is a classic programming concept of “subroutines” or “subprograms”, to perform repetitive tasks and which surely sounds familiar to those who have programmed something in Basic.

As always, let’s see it with a simple example to understand it:

```
[[ /* Set screen colors and clear it */
   PAPER 0    /* Black background color  */
   BORDER 0   /* Black border color  */
   INK   7    /* White text color */
   PAGEPAUSE 1
   DECLARE 0 AS NumGamusinos
   SET NumGamusinos TO 0
   #Inicio
   CLEAR]]You have [[GOSUB ImprimirGamusimos]].
```

```
What do you do?

[[OPTION GOTO Suma1]]I take 1 gamusino.
[[OPTION GOTO Resta1]]I drop 1 gamusino.
[[OPTION GOTO Final]]Exit.
[[
   #Escoger
   CHOOSE
   #Suma1
   SET NumGamusinos TO @NumGamusinos + 1
   GOTO Inicio
   #Resta1
   SET NumGamusinos TO @NumGamusinos - 1
   GOTO Inicio
   #Final]]Thanks for playing!
In the end you keep [[GOSUB ImprimirGamusimos]].[[
   WAITKEY
   END
   /* Gamusino printing subroutine*/
   #ImprimirGamusimos : PRINT @NumGamusinos]] gamusinos[[RETURN]]
```

The Gamusinos example again… But this time we print the number of Gamusinos twice, when we choose an option and at the very end.

To do this we make a subroutine that performs this function:

```
[[LABEL ImprimirGamusimos]]You have [[PRINT @0]] gamusinos[[RETURN]]
```

We declare a label called `PrintGamusinos` that serves as a jump point. Then we have the print-out of the number of gamusinos, and then the `RETURN` command. We make the call with `GOSUB PrintGamusinos` at the two points where we want it to be executed.

As I have already indicated, `GOSUB PrintGamusinos`, what it does is make a jump to the label `PrintGamusinos`, but with the exception that the point of the call to the subroutine is saved in the *stack*. Every subroutine must have a return point, that is, a completion point with which the engine is told to return to the point where we had left it, and that is done by the `RETURN` command, which recovers the last return point from the stack and jumps to that position.

One thing to note is that the subroutine is at the end, after `END`. This is so that it is not executed without us calling it explicitly. Please note that the interpreter does not differentiate a subroutine from “normal” code, and if it reaches that point it will execute it and do an invalid `RETURN` at the end. Therefore, I recommend to avoid these situations, placing them at the end after an `END`, or using a `GOTO` before it to skip it in case you accidentally reach it.

And now, as usual, the clarifications and exceptions. Subroutines can be nested, that is, you can call a subroutine inside another. The return addresses will be stored in the stack in reverse order, but **BE CAREFUL, the stack has a limit**. This means you have a nesting limit. And as I explained in the previous paragraph, if you do a `RETURN` without a previous `GOSUB`, you will have at least one error, and at most, erroneous behavior.

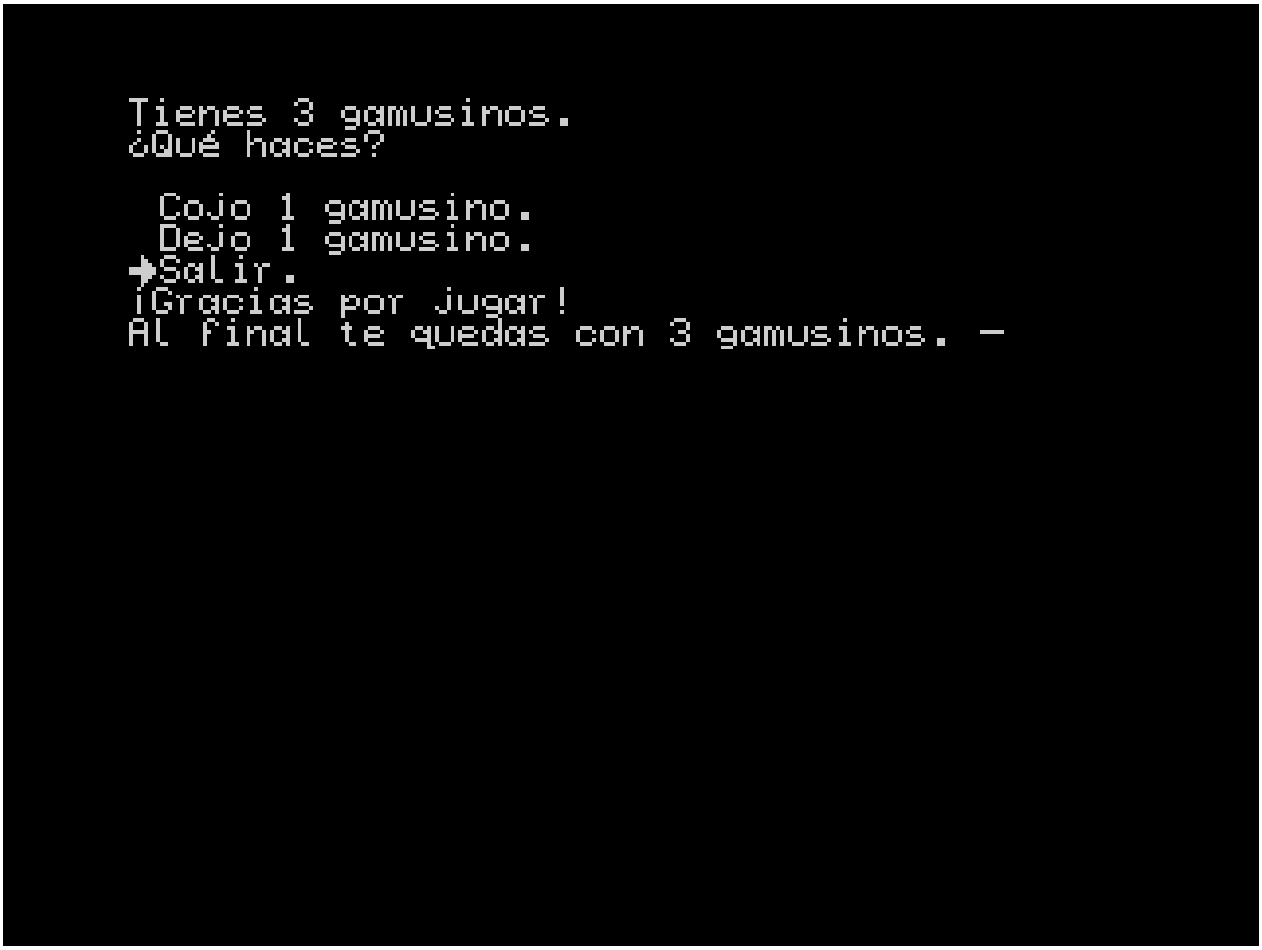


Figure 27: Subrutinas

Generally, the ideal time to call a subroutine is when you make a choice from a menu. So we've included the `OPTION GOSUB Label` variant, which will call the corresponding subroutine if that option is chosen. When it reaches the `RETURN` (always remember to end subroutines with it!), it will resume execution right after the `CHOOSE` of that option. With our new knowledge, let's fine-tune our code:

```
[[ /* Set screen colors and clear it */
   PAPER 0    /* Black background color  */
   BORDER 0   /* Black border color  */
   INK   7    /* White text color */
   PAGEPAUSE 1
   DECLARE 0 AS NumGamusinos
   SET NumGamusinos TO 0
   #Inicio
   CLEAR]]You have [[GOSUB ImprimirGamusimos]].
What do you do?

[[OPTION GOSUB Suma1]]I take 1 gamusino.
[[OPTION GOSUB Resta1]]I drop 1 gamusino.
[[OPTION GOTO Final]]Exit.
[[
   CHOOSE
   GOTO Inicio
   #Final]]Thanks for playing!
In the end you keep [[GOSUB ImprimirGamusimos]].[[
   WAITKEY
   END
   /* Gamusino printing subroutine*/
   #ImprimirGamusimos : PRINT @NumGamusinos]] gamusinos[[RETURN
   /* Addition subroutine*/
   #Suma1 : SET NumGamusinos TO @NumGamusinos + 1 : RETURN
   /* Subtraction subroutine */
   #Resta1 : SET NumGamusinos TO @NumGamusinos - 1 : RETURN
   ]]
```

1.15 Conditional Execution

In the following example we will use the `IF ... THEN ... ENDIF` structure to illustrate how it works: Copy the following into `Tutorial.cyd`:

```
[[ /* Set screen colors and clear it */
   PAPER 0    /* Black background color  */
   BORDER 0   /* Black border color  */
   INK   7    /* White text color */
   PAGEPAUSE 1
   DECLARE 0 AS NumGamusinos
   SET NumGamusinos TO 0
   #Inicio
```

```
   CLEAR]]You have [[GOSUB ImprimirGamusimos]].
What do you do?

[[OPTION GOSUB Suma1]]I take 1 gamusino.
[[OPTION GOSUB Resta1]]I drop 1 gamusino.
[[ IF @NumGamusinos = 10 THEN
   OPTION GOTO Final]]Exit.
[[
   ENDIF
   CHOOSE
   GOTO Inicio
   #Final]]Thanks for playing!
In the end you keep [[GOSUB ImprimirGamusimos]].[[
   WAITKEY
   END
   /* Gamusino printing subroutine*/
   #ImprimirGamusimos : PRINT @NumGamusinos]] gamusinos[[RETURN
   /* Addition subroutine*/
   #Suma1 : SET NumGamusinos TO @NumGamusinos + 1 : RETURN
   /* Subtraction subroutine */
   #Resta1 : SET NumGamusinos TO @NumGamusinos - 1 : RETURN
   ]]
```

Funny… now we can't get out…

…until we have 10 gamusinos! What happens?

The answer is the `IF @NumGamusinos = 10 THEN ... ENDIF`, if the condition that we have 10 gamusinos is met, whatever is between the `THEN` and the `ENDIF` is executed, and otherwise it will skip it.

Note that not only the option command to exit is skipped; it also skips the text after it and, therefore, it does not show it either.

Now I am going to change to `IF @NumGamusinos = 10 THEN ... ELSE ... ENDIF`. What we do is add another block of code, which is executed if the condition is *NOT* met. In this case we are going to add another IF that tells us what we have to do:

```
[[ /* Set screen colors and clear it */
   PAPER 0    /* Black background color  */
   BORDER 0   /* Black border color  */
   INK   7    /* White text color */
   PAGEPAUSE 1
   DECLARE 0 AS NumGamusinos
   SET NumGamusinos TO 0
   #Inicio
   CLEAR]]You have [[GOSUB ImprimirGamusimos]].
What do you do?

[[OPTION GOSUB Suma1]]I take 1 gamusino.
```

Figure 28: Ifs

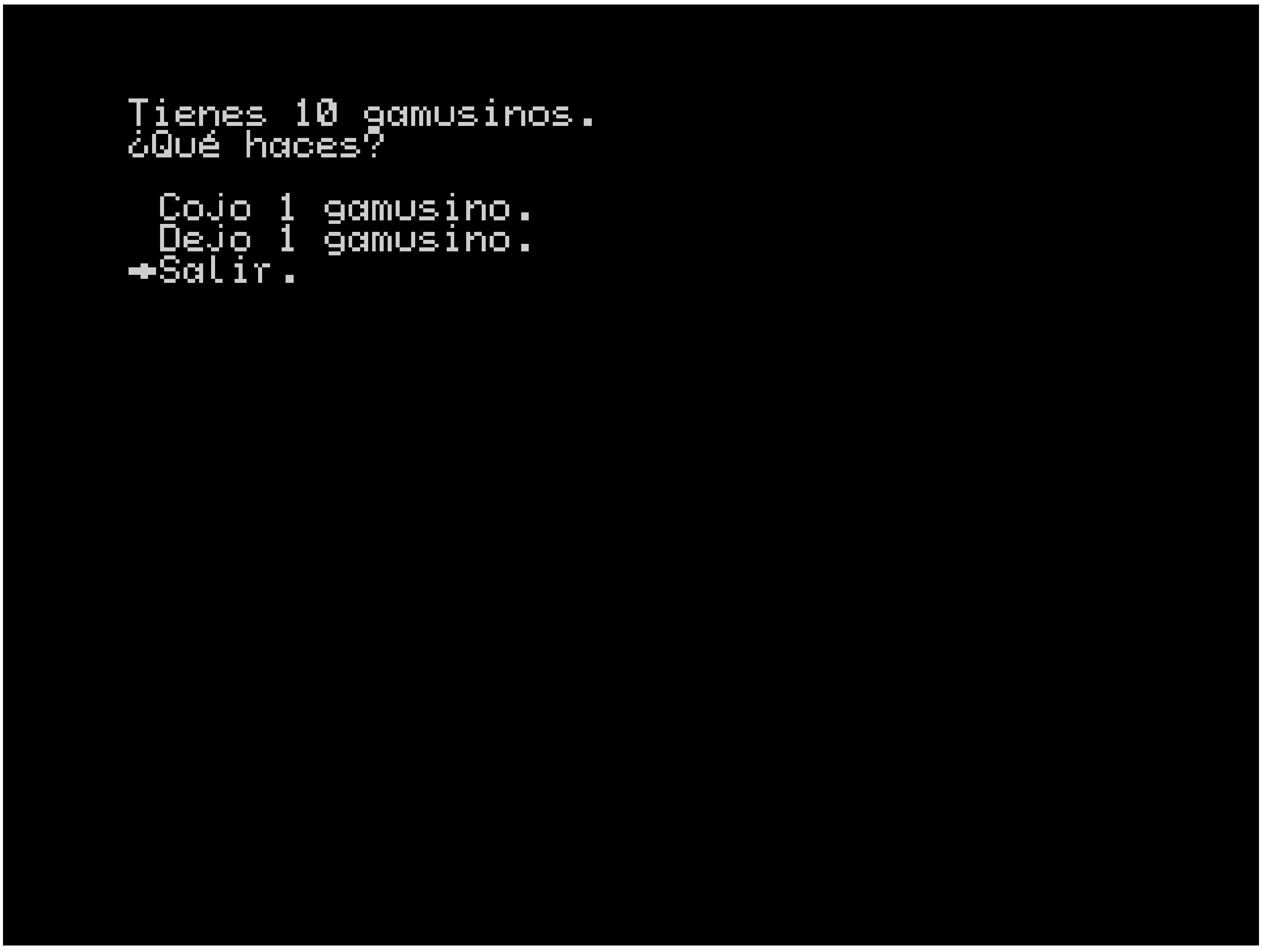


Figure 29: Ifs2

```
[[OPTION GOSUB Resta1]]I drop 1 gamusino.
[[ IF @NumGamusinos = 10 THEN
   OPTION GOTO Final]]Exit.
[[ ELSE IF @NumGamusinos < 10 THEN ]]
You need more gamusinos
[[ ELSE ]]
You have too many gamusinos
[[ ENDIF
   ENDIF
   CHOOSE
   GOTO Inicio
   #Final]]Thanks for playing!
In the end you keep [[GOSUB ImprimirGamusimos]].[[
   WAITKEY
   END
   /* Gamusino printing subroutine*/
   #ImprimirGamusimos : PRINT @NumGamusinos]] gamusinos[[RETURN
   /* Addition subroutine*/
   #Suma1 : SET NumGamusinos TO @NumGamusinos + 1 : RETURN
   /* Subtraction subroutine */
   #Resta1 : SET NumGamusinos TO @NumGamusinos - 1 : RETURN
   ]]
```

If we are below:

And if we are above:

This allows us, along with `GOTO` and `GOSUB`, to control the **flow of the program**.

1.16 Loops

To make certain parts of the code repeat while a condition is met we have `WHILE (cond) ... WEND`:

```
[[ /* Set screen colors and clear it */
   PAPER 0    /* Black background color  */
   BORDER 0   /* Black border color  */
   INK   7    /* White text color */
   PAGEPAUSE 1
]]We count up to 10:
[[
   DECLARE 0 AS cnt
   SET cnt TO 1
   WHILE (@cnt < 11)
      PRINT @cnt
      TAB 1
      SET cnt TO @cnt + 1
   WEND
]]
Done[[WAITKEY]]
```

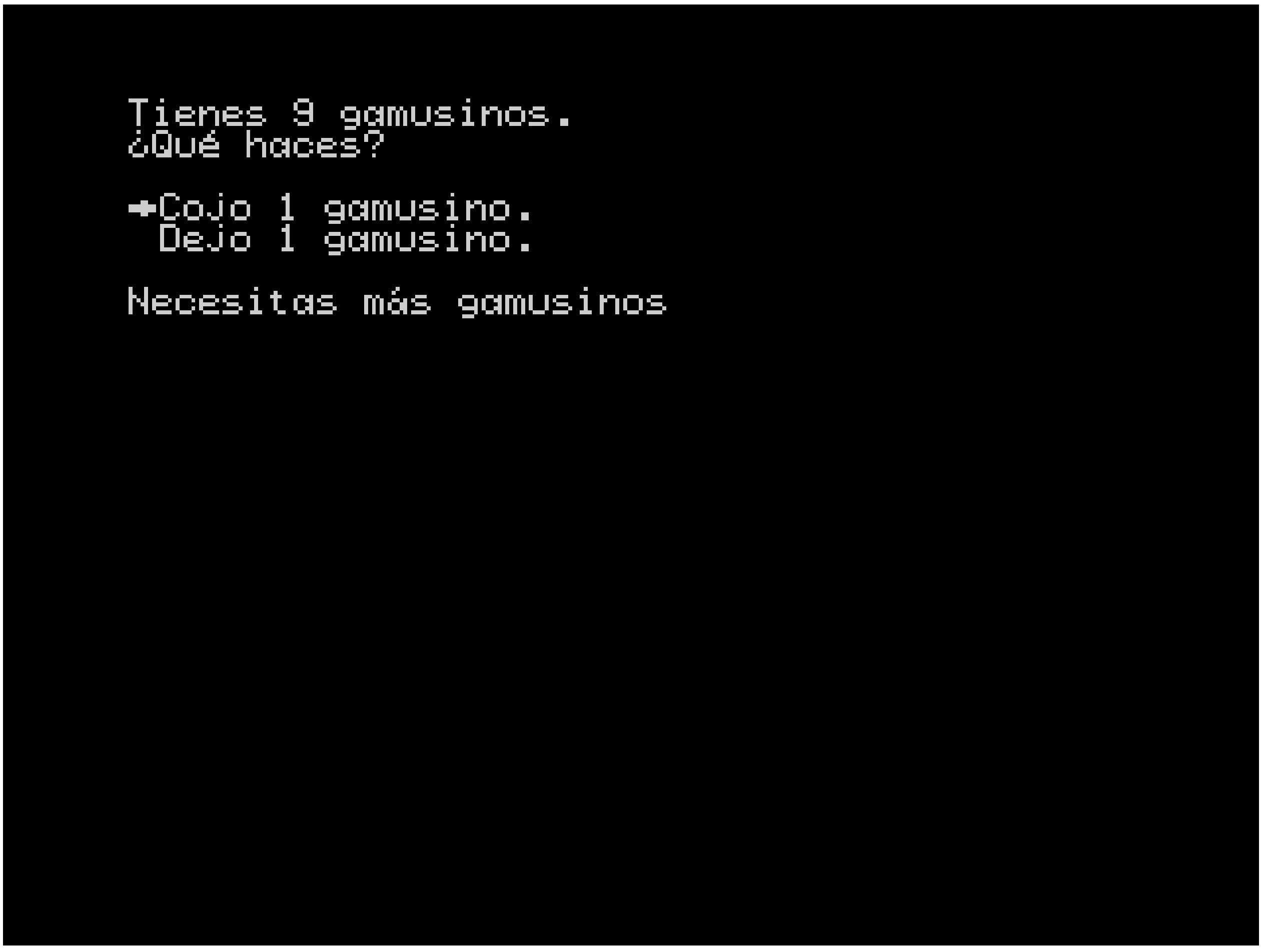


Figure 30: Ifs2

Figure 31: Ifs3

In this example, I set the variable *cnt* to 1, and when the loop starts, it checks the condition that the variable is less than 11, as long as it is true, everything up to `WEND` is executed. Each time the loop is executed, we print the value of the variable, put a space with `TAB 1` and increment its value by 1. When it reaches 11, it stops executing and continues with whatever is after `WEND`.

And now it counts from one to 10:

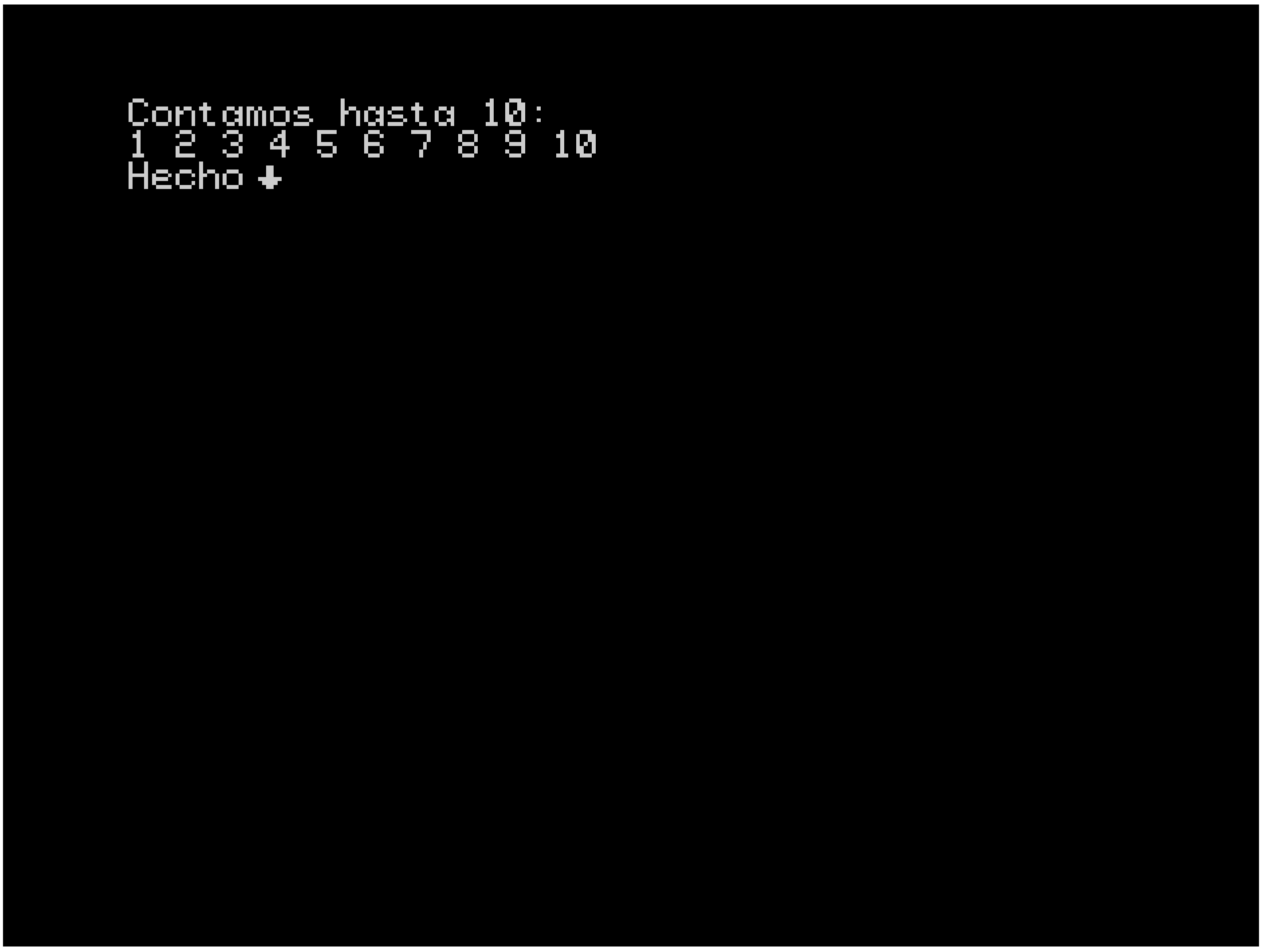


Figure 32: While

It is important to define the entry and exit conditions well, and to keep in mind that the verification of the loop condition is always done **at the beginning** of each iteration.

With the addition of the loop, we can implement almost anything with the CYD language.

1.17 Text compression and abbreviations

To save disk space, the compiler compresses the text, looking for the most common substrings in the text and replacing them with tokens or abbreviations.

The `make_adv.cmd` script will make the compiler look for abbreviations if it does not find a file named `tokens.json` in its folder, and will save the abbreviations found in a file with that name. On the other hand, if it finds such a file, it will pass it as a parameter to the compiler so that it uses the abbreviations contained in it. To make it look for abbreviations again, simply delete the `tokens.json` file before running it.

The process of looking for abbreviations can be very expensive as the amount of text in the adventure increases. My advice is to write the adventure script *before* you start programming and put it all in a text file that you can feed to the compiler, so that it can generate a suitable abbreviation file. Once you have that, you can start adding commands to the text to shape the adventure. Once you have it *almost* finished, you can generate abbreviations again to see if you can carve out some more space.

1.18 Organizing Code with INCLUDE

As your adventure grows larger, managing all the code in a single file can become challenging. The `INCLUDE` directive allows you to split your adventure into multiple files, making it easier to organize and maintain your project.

1.18.1 Basic Usage

The `INCLUDE` directive lets you insert the contents of one CYD file into another during compilation:

```
INCLUDE "filename.cyd"
```

For example, if you have a file called `variables.cyd` with all your variable declarations:

variables.cyd:

```
[[
DECLARE 0 AS PlayerHealth
DECLARE 1 AS PlayerGold
DECLARE 2 AS HasSword
DECLARE 3 AS HasKey
]]
```

You can include it in your main file:

main.cyd:

```
[[
INCLUDE "variables.cyd"
PAPER 0 : INK 7 : CLEAR
]]
You are in a dark dungeon...
```

When compiled, the preprocessor will automatically insert the contents of `variables.cyd` where the `INCLUDE` directive appears.

1.18.2 Organizing a Larger Project

Here's a recommended structure for a larger adventure:

```
my_adventure/
  ├── main.cyd              # Main entry point
  ├── config/
  │   ├── variables.cyd     # All variable declarations
  │   └── settings.cyd      # Initial settings (colors, etc.)
  ├── lib/
```

```
│   └── common.cyd        # Common subroutines
└── chapters/
    ├── intro.cyd         # Introduction
    ├── chapter1.cyd      # Chapter 1
    ├── chapter2.cyd      # Chapter 2
    └── ending.cyd        # Ending sequences
```

Example main.cyd:

```
[[
/* Load configuration */
INCLUDE "config/variables.cyd"
INCLUDE "config/settings.cyd"

/* Load common functions */
INCLUDE "lib/common.cyd"
]]

#Start
[[INCLUDE "chapters/intro.cyd"]]
[[GOTO Chapter1]]

#Chapter1
[[INCLUDE "chapters/chapter1.cyd"]]

#Chapter2
[[INCLUDE "chapters/chapter2.cyd"]]

#TheEnd
[[INCLUDE "chapters/ending.cyd"]]
[[END]]
```

1.18.3 Practical Example: Common Subroutines

Let's say you frequently show the player's status. Instead of repeating the code, create a subroutine file:

lib/common.cyd:

```
[[
#ShowStatus
INK 6
PRINT "Health: "
PRINT @PlayerHealth
PRINT " | Gold: "
PRINT @PlayerGold
INK 7
NEWLINE
RETURN
```

```
#GameOver
CENTER
INK 2
PRINT "GAME OVER"
INK 7
NEWLINE : NEWLINE
PRINT "Press any key to restart..."
WAITKEY
GOTO Start
]]
```

Then use it throughout your chapters:

chapters/chapter1.cyd:

```
[[CLEAR]]
You enter a dark corridor...
[[NEWLINE]]
[[GOSUB ShowStatus]]

[[OPTION GOTO Door1]]Open the left door
[[OPTION GOTO Door2]]Open the right door
[[CHOOSE]]

#Door1
You found treasure!
[[SET @PlayerGold TO @PlayerGold + 50]]
[[GOSUB ShowStatus]]
[[GOTO Chapter2]]

#Door2
A trap! You lose health.
[[SET @PlayerHealth TO @PlayerHealth - 20]]
[[IF @PlayerHealth <= 0 THEN
    GOSUB GameOver
ELSE
    GOSUB ShowStatus
    GOTO Chapter2
ENDIF]]
```

1.18.4 Important Features

- **Case-insensitive:** `INCLUDE`, `include`, and `Include` all work
- **Quotes:** Both `"file.cyd"` and `'file.cyd'` are supported
- **Relative paths:** Paths are resolved relative to the file containing the `INCLUDE`
- **Nested includes:** Files can include other files (up to 20 levels deep)
- **Comments in CYD:** Use `/* */` for comments (`/* Load utilities */`). While the `INCLUDE` pattern will ignore text after the filename, relying on this is not recommended.

- **Error detection:** Circular includes (A includes B, B includes A) are detected automatically

1.18.5 Benefits

1. **Organization:** Keep related code together (all variables in one place, each chapter in its own file)
2. **Reusability:** Share common subroutines across multiple adventures
3. **Maintainability:** Find and fix bugs more easily in smaller, focused files
4. **Collaboration:** Multiple people can work on different files
5. **Testing:** Test individual chapters or components separately

1.18.6 Tips for Using INCLUDE

1. **Start simple:** Begin with a single file, then split into modules as the adventure grows
2. **Logical grouping:** Group related functions, variables, or story sections together
3. **Consistent naming:** Use clear, descriptive filenames like `chapter1.cyd` or `combat_functions.cyd`
4. **Directory structure:** Use subdirectories (`chapters/`, `lib/`, `config/`) to keep things organized
5. **Document dependencies:** Add comments explaining what each included file provides

---

1.19 Using the bundled libraries

The `lib/` directory ships several **libraries written in the CYD language itself**: collections of ready-to-use subroutines that do not modify the engine. You add them with `INCLUDE` and call them with `GOSUB`. Each library **skips over itself** (an internal `GOTO`), so you just include it at the start of your program; execution continues with your code and the routines only run when you call them.

Each library reserves a small block of variables as its workspace, and the blocks do not overlap, so you can use several at once:

- `lib/math16_32.cyd` (variables 224..254): 16- and 32-bit arithmetic.
- `lib/strings.cyd` (variables 216..223): text-string input and handling.

The manual has the full reference for every routine; here we will look at a couple of practical cases.

1.19.1 A score that does not fit in a byte

CYD variables are 8-bit (0..255), so a large score overflows quickly. With `math16_32.cyd` we can compute it in 32 bits. Operands are loaded into "registers" (groups of consecutive variables); since literals are 8-bit, wide values are loaded byte by byte with the multiple assignment `SET reg TO {low_byte, ...}`.

```
[[
    INCLUDE "../../lib/math16_32.cyd"
    PAPER 0 : INK 7 : CLEAR

    /* enemies (1250) * points (800) = 1,000,000, without overflow */
    SET mlA0 TO {226, 4}        /* A = 1250 */
    SET mlB0 TO {32, 3}         /* B = 800  */
    GOSUB mul1632               /* C = A * B (32-bit result) */
```

```
    /* + bonus (234567) */
    SET mlA0 TO {@mlC0, @mlC1, @mlC2, @mlC3}   /* A = C */
    SET mlB0 TO {71, 148, 3, 0}                /* B = 234567 */
    GOSUB add32                                /* A = 1234567 */
]]Total score: [[ GOSUB print32 ]][[
    WAITKEY : END
]]
```

You can see this full example in `examples/math_library`.

1.19.2 Asking for the player's name

With `strings.cyd` reading a string from the keyboard is very simple. You only need to tell it where the buffer is (`stBase`) and its capacity (`stLen`); `strInput` handles the cursor, deletion with DELETE and ending with ENTER.

```
[[
    INCLUDE "../../lib/strings.cyd"
    PAPER 0 : INK 7 : CLEAR

    SET stBase TO 0 : SET stLen TO 16   /* buffer in variables 0..15 */
    GOSUB strInput
]]Hello, [[ GOSUB strPrint ]]!
[[
    GOSUB strLen
]]Your name has [[ PRINT @stRes ]] letters.[[
    NEWLINE : WAITKEY : END
]]
```

You can see this full example in `examples/strings_library`.

1.20 Workflow

From this point on, you should have enough knowledge to be able to tackle your own adventure. From here on, I cannot give you advice on how to write your adventure, that is up to your imagination. However, I am going to recommend a workflow based on the experience of developing the adventure *The Rings of Saturn*, which I submitted to the Radastán 2023 Adventure Contest.

1.20.1 GUI Compiler Alternative

Instead of manually editing and running the `make_adv` scripts, you can use the **make_adventure_gui** tool, which provides a graphical interface for compiling your adventures.

Launching the GUI:

Windows:

```
make_adventure_gui.cmd
```

Linux/macOS:

```
./make_adventure_gui.sh
```

The GUI allows you to configure all compilation options (target, paths, image lines, abbreviation settings, and more) without touching any scripts. Once configured, you can compile your adventure with a single button click. The GUI will display detailed compilation messages and automatically open the compiled file on your chosen emulator if configured.

This approach eliminates the need to understand batch or shell scripts, making the compilation process more accessible for users who prefer graphical interfaces.

My first recommendation is to write **before** the bulk of your adventure, as you will have already read in the previous chapter. With that, you will have a valid list of abbreviations to start working on, since the process of searching for abbreviations is **very expensive**. To give you an idea, compressing the entire *The Rings of Saturn* (a book of about 150 pages) takes up to three quarters of an hour. In addition, our workflow is essentially going to consist of writing, compiling and testing over and over again, so we are interested in making this process as fast as possible.

You can write your adventure with your favorite text editor, but always **in plain text**, since that is what the compiler understands.

An important issue when working with the editor is the encoding of the texts. A file on the disk is a collection of bits ordered with a name, the computer does not "know" what to do with those bits; it is the programs that interpret those bits.

A text file is a file whose bits represent characters. The way of interpreting those characters is called **encoding**. The most famous encoding used since the beginning of computing is **ASCII**, whose standard supports 128 characters, designed for the English language and with extensions to 256 for international and special characters. Today, ASCII has become too small, and the encoding used as standard by all current text editors by default is **UTF-8**.

All current editors support multiple encodings, so you will have to investigate the operation of your editor to select the correct encoding.

The **CYD** compiler supports UTF-8 text, but you have to take into account a number of limitations when writing your text or copying it from another source. The default character set of the engine is the following:

Those are the characters you have available, and UTF-8 has *millions* of different spellings, and I'm not exaggerating, so most of them will fail the compiler. In the manual I detail the only characters that will be translated from UTF-8 to the encoding used by the engine:

Character	Position
'ª'	16
'¡'	17
'¿'	18
'«'	19
'»'	20
'á'	21
'é'	22
'í'	23

Character	Position
‘ó’	24
‘ú’	25
‘ñ’	26
‘Ñ’	27
‘ü’	28
‘Ü’	29

As you can see, for example, the accented uppercase vowels are not there. The compiler will automatically transform the accented uppercase letters into unaccented uppercase letters.

Once you have written a good part of your adventure, and obtained an abbreviation file, we can start programming it.

My recommendation is to use the “divide and conquer” tactic, a technique that consists of dividing a problem to be solved into smaller subproblems that we will solve little by little. In our context, it means that we should divide the adventure into sections that we will program one by one. You have probably already instinctively done this step when writing your story, dividing it into chapters.

With this subdivision, we will now work with two source files, a *global file* with the text of the “complete” adventure, and another that we will pass to the compiler with the section that we are going to program, which we will call *working file*. The process consists of the following steps:

1. Copy one of the uncompleted sections from the global file to the working file.
2. Add the commands to the text of the working file and format it if necessary.
3. Compile the working file.
4. Run the resulting image in an emulator and test.
5. If you are not satisfied, modify the file and return to step 3.
6. When you have the complete section, copy the completed section in the working file and replace it in the global file.
7. If you still have sections to complete, return to step 1.
8. Move the entire global file to the working file.

However, once you have completed the adventure, you will probably have to fix things, compile and test again. And, depending on the length of the adventure, getting to the relevant part can be a hell. I will suggest several techniques to avoid this.

One tactic we can employ is to label the beginning of each and every section with `LABEL` (we would do this in the previous process). Then, we simply put a `GOTO` to the label of the section we want to test at the beginning of the adventure. When the section is executed, it would jump directly to the relevant section, saving us precious time. Remember to remove the initial `GOTO` afterwards!

Another technique I have employed is to cancel pauses. To do this, I use the compiler’s ability to use comments within code sections. Using our text editor, we can replace all occurrences of the `WAITKEY` command, for example, with the text `/*WAITKEY*/`. When we compile again, since those commands are commented out, they will not be executed and everything will be displayed non-stop. When we want to undo the change, we do the opposite process, replacing `/*WAITKEY*/` with `WAITKEY`.

Figure 33: Default character set

To speed up this process even further, we can even introduce the commands commented out in the previous phase.

Finally, almost all modern emulators offer the possibility of speeding up the execution. We can take advantage of this to display the text faster than usual.

I hope these techniques help you create your adventure in the most comfortable way possible. Programming is an iterative process of writing, compiling, running and testing, and it can be monotonous, but also very satisfying when we get the desired result.

1.21 Side-scrolling menus

Option menus also allow *side-scrolling* and the `MENUCONFIG` command allows you to configure this behavior. First, it should be noted that everything explained earlier in the tutorial is still applicable, but it has become a special case that we will detail. Without going into too many technical details of the engine, we will explain how the menu system works now.

The menu system is based on a list of options. Each time we declare an option, the screen position (adjusted to 8x8 coordinates) and the corresponding jump address are saved in the list, so that the options are saved in the same order in which we declared them. When the screen is cleared or an option is chosen from the menu, the list is cleared.

When you activate the menu with `CHOOSE`, a pointer corresponding to the selected option is placed in the first position of the list, and the corresponding option in the list shows the selection icon in the stored screen position. When you scroll through the menu with the keys, you move that pointer along the list, and at each step, the on-screen pointer moves to the corresponding position.

In previous versions, you were only allowed to move one position forward in the list with the *A* key and one position backward with the *Q* key. Because of this key layout, you were expected to have vertical menus.

Now you are allowed to use the *O* and *P* keys to scroll *"horizontally". In fact **CYD** has no concept of horizontality and verticality, it simply scrolls through the list in a certain way according to the keys pressed. Thanks to the command `MENUCONFIG x,y` you can configure this behavior. The command admits two parameters that mean the following:

- The first parameter (which we will call `X`) determines the increment or decrement of the selected option number when you press **P** and **O** respectively.
- The second parameter (which we will call `Y`) determines the increment or decrement of the selected option number when we press **A** and **Q** respectively.

That is, if the selected option number at a given time is `N`, then:

- When we press **O**, `N` becomes `N - X`.
- When we press **P**, `N` becomes `N + X`.
- When we press **Q**, `N` becomes `N - Y`.
- When we press **A**, `N` becomes `N + Y`.

With this we can make different types of movements, but it is our responsibility to place the options in the correct order and position on the screen so that it is consistent with the keystrokes.

Let's see it, as always, with the following example:

```
[[
    PAGEPAUSE 1
    BORDER 0
    INK 7
    BRIGHT 1
    FLASH 0
    PAPER 0
    CLEAR
]]Choose an option:

[[OPTION GOTO Opcion1]]First option.  [[OPTION GOTO Opcion2]]Second option.
[[OPTION GOTO Opcion3]]Third option.  [[OPTION GOTO Opcion4]]Fourth option.
[[OPTION GOTO Opcion5]]Fifth option.   [[OPTION GOTO Opcion6]]Sixth option.

[[
    MENUCONFIG 1,2 : CHOOSE
    LABEL Opcion1]]You chose option 1.[[
    GOTO Siguiente
    LABEL Opcion2]]You chose option 2.[[
    GOTO Siguiente
    LABEL Opcion3]]You chose option 3.[[
    GOTO Siguiente
    LABEL Opcion4]]You chose option 4.[[
    GOTO Siguiente
    LABEL Opcion5]]You chose option 5.[[
    GOTO Siguiente
    LABEL Opcion6]]You chose option 6.[[
    LABEL Siguiente
    WAITKEY : END]]
```

According to what we have explained, when using `MENUCONFIG 1,2`, then when we press **O** and **P**, the selected option is decremented and incremented by 1, and when we press **Q** and **A**, the selected option is decremented and incremented by 2. That is asking us for a menu with two elements per row, since when we press *down* or *up*, we jump two by two. With this, we must take care of placing the options in two columns and in order from left to right.

The default behavior, which simulates the *vertical* behavior of previous versions, is done with `MENUCONFIG 0,1`. That is, when we press **O** and **P**, the selected option is decremented and incremented by 0, that is, **it does nothing**, it does not move. And when you press **Q** and **A**, the selected option is decremented and incremented by 1, so we have a strictly vertical menu.

Thus, if with `MENUCONFIG 0,1` we will have a vertical menu in one column, with `MENUCONFIG 1,0` we will have a horizontal menu in one row. With `MENUCONFIG 1,3` we will have a three-column menu, etc. But remember that `MENUCONFIG` does not indicate the arrangement of the options, we define that when we declare them, but the behavior of the buttons.

In this case, it is better to try the example live and experiment on your own. Keep in mind that

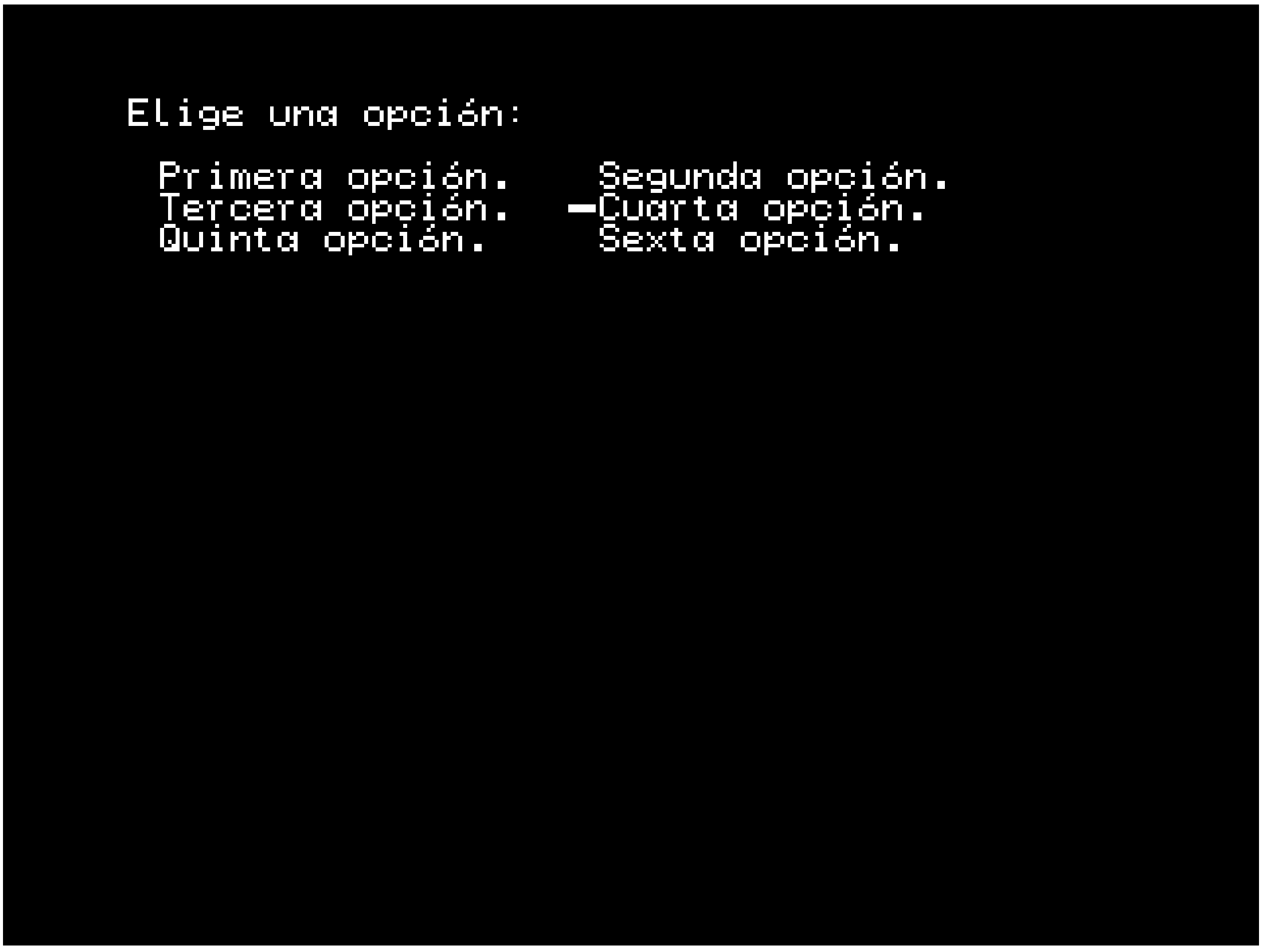


Figure 34: Side Menu

the pointer will never exceed the limits below zero, nor above the number of options in the list. Therefore, you should enter *reasonable* values or the menu will not work well.

1.22 Copying image fragments to screen

If you have followed the tutorial in order, you will already know that with the command `PICTURE` we load an image into the screen buffer, and with `DISPLAY` we display it, that is, it is copied from the buffer to the screen, but always in full screen. But it is also possible to copy a piece of the image loaded in the buffer to a certain defined position on the screen using the command `BLIT`.

But first, let's create the following SCR image with some graphical editor for Spectrum:

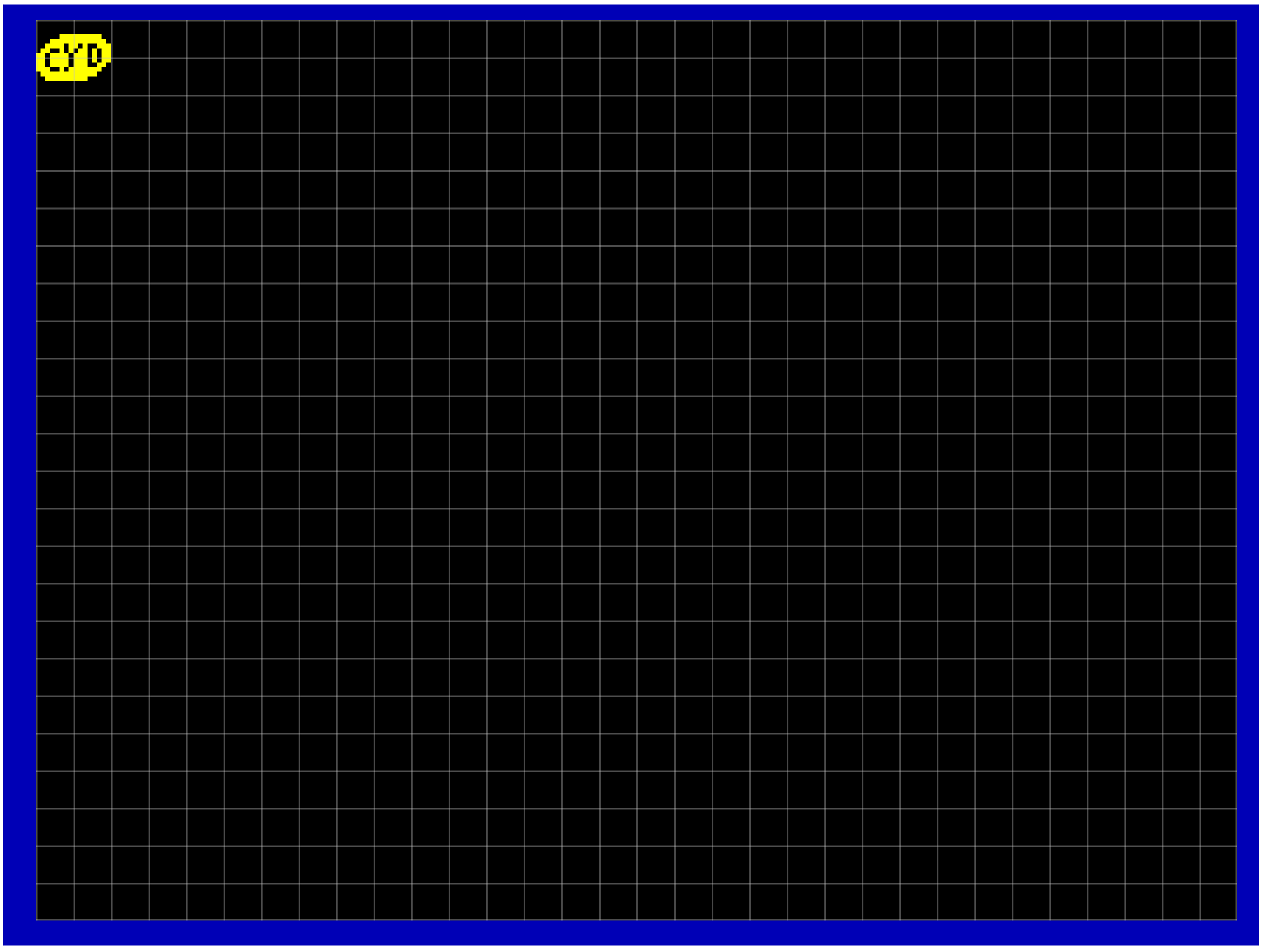


Figure 35: Example

And we leave it in the directory `IMAGES`. You can see that we have drawn in the first block of 16x16 pixels, or what is the same, 2x2 characters.

Now, let's use the following example:

```
[[
    INK 7
    PAPER 0
    BORDER 1
    CLEAR
    PICTURE 0             /* Loading image 0 into the buffer */
```

```
DECLARE 0 AS row     /* Variable for rows */
DECLARE 1 AS col     /* Variable for columns */
SET row TO 0
WHILE (@row < 24)
    SET col TO 0
    WHILE (@col < 32)
        BLIT 0, 0, 2, 2 AT @col, @row /* Copying to screen */
        SET col TO @col+2
    WEND
    SET row TO @row+2
WEND
AT 31, 23
WAITKEY
END]]
```

This example basically clears the screen and loads image 0 into the screen buffer (the one we created earlier), and then we go through the rows from 0 to 24 and the columns from 0 to 32, jumping two characters at a time. If you've followed this tutorial, you won't have any trouble understanding how it works.

The key component is the command `BLIT 0, 0, 2, 2 AT @col, @row`. What we're telling the engine is "take the rectangle from the buffer with origin (0,0) and size 2x2 characters and copy it to the screen at the position defined by the variables col and row". Since we are traversing the entire screen using two nested loops, the result is the following:

In general, with `BLIT x_orig, y_orig, width, height AT x_dest, y_dest`, what we do is copy from the image loaded in the buffer a piece of it defined by the first 4 parameters, and we copy it to the screen from the last two. This gives us many possibilities to build the image on the screen using "tiles", create animations, curtains, etc.

As final notes, three things to highlight:

- Variables can only be used in the last two parameters, as is the case in this example. The parameters that define the rectangle to be cut are always fixed.
- In all parameters, the unit is the 8x8 pixel character.
- The implementation is slow due to its genericity, so if you are tempted to use them as "sprites", the results will be disappointing due to flickering.

1.22.1 Strict Colon Mode (Syntax Enforcement)

As of version 1.2.x, the compiler enforces **Strict Colon Mode by default**. This means that when you write multiple statements on the same line, they **must** be separated by colons (`:`).

Correct Syntax:

```
SET x TO 10 : SET y TO 20 : PRINT "Values set"
```

Incorrect Syntax (will fail):

```
SET x TO 10 SET y TO 20 PRINT "Values set"
```

However, when statements are on separate lines, colons are **optional**:

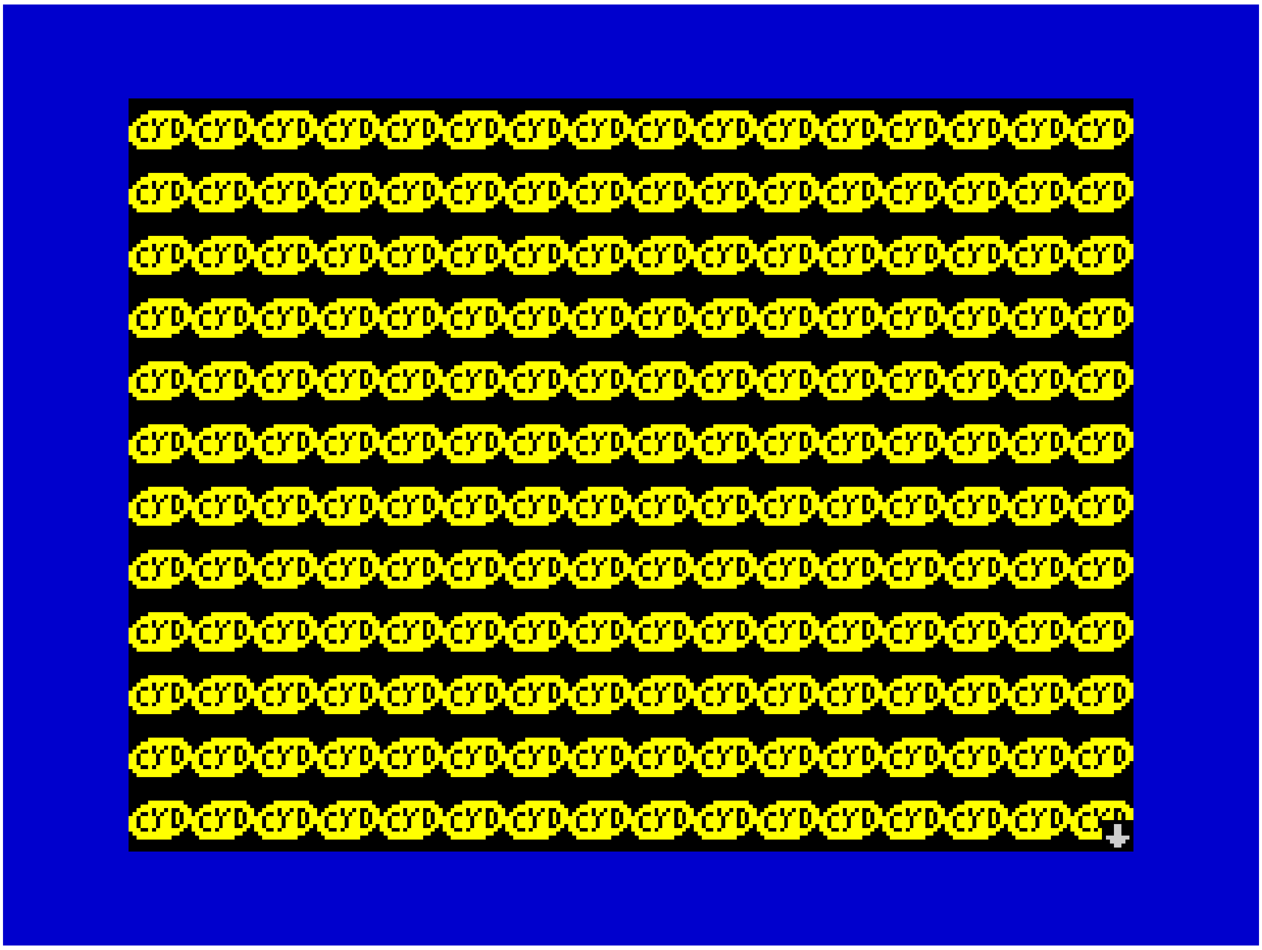


Figure 36: BLIT

```
SET x TO 10
SET y TO 20
PRINT "Values set"
```

Key points: - Line breaks automatically separate statements, making colons unnecessary when using new lines - If you prefer the old behavior (without strict colon mode), you can use the `--no-strict-colons` flag with the compiler - Strict Colon Mode improves code readability and prevents accidental statement concatenation - The compilation process enforces this rule consistently across all your adventure files

---

1.23 Reading the keyboard, variable arrays and indirections

First, I'll warn you that this is a fairly advanced chapter that introduces some rather advanced programming concepts if you've never programmed before. If you don't understand it, you can skip it without any problems.

There are a series of commands that allow reading characters from the keyboard and deleting them on the screen, and, together with indirection, they allow us to store text strings in variables and use them as arrays. However, **BE CAREFUL**, we only have 256 variables available... This means that we can't store very long text strings, but we can use it to store a short one, like the name of our protagonist or to create a simple command line.

Let's see it with the following example:

```
[[ /* Set screen colors and clear it */
   PAPER 0    /* Black background color  */
   INK   7    /* White text color */
   BORDER 0   /* Black border color  */
   CLEAR      /* Clear the screen*/
   PAGEPAUSE 1

   DECLARE 0 AS str        /* Start of the 16-character array */
   DECLARE 16 AS ptr       /* Current pointer over the string */
   DECLARE 17 AS chr       /* Character read from keyboard */

]]Enter your name:[[
   GOSUB inputStr]]
Welcome [[
   GOSUB printStr
   NEWLINE /* Line break */
   WAITKEY
   END

   /* Subroutine to print a 16-character string*/
   #printStr
   /* Initialize the pointer with the address of the string's first variable */
   SET ptr TO @@str
```

```
/*
While the pointer is less than its own address...
(The address serves as a marker to indicate the end of the string)
*/
WHILE (@ptr < @@ptr)
   /* If the content of the variable marked by the pointer is zero, we finish */
   IF [@ptr] = 0 THEN RETURN ENDIF
   /* Print the character using indirection on the pointer */
   CHAR [@ptr]
   /* Increment the pointer one position */
   SET ptr TO @ptr + 1
WEND
RETURN

#inputStr
/* Initialize the pointer with the address of the string's first variable */
SET ptr TO @@str
/* Fill the whole string with zeros */
WHILE (@ptr < @@ptr)
   SET [ptr] TO 0
   SET ptr TO @ptr + 1
WEND
/* Put the pointer back at the beginning */
SET ptr TO @@str
/* Infinite loop */
WHILE ()
   /* Put the '_' character as cursor */
   CHAR 95
   /* Read a pressed key and store its ASCII code in chr */
   SET chr TO INKEY()
   /* Erase the cursor */
   BACKSPACE
   /* If the key is ENTER, we exit */
   IF @chr = 13 THEN RETURN ENDIF
   /* If the key is DELETE... */
   IF @chr = 12 THEN
      /* Set the current pointer position to zero, if we are not at the end of the array */
      IF @ptr < @@ptr THEN
         SET [ptr] TO 0
      ENDIF
      /* If we are not at the beginning of the array... */
      IF @ptr > @@str THEN
         /* Erase the current position on screen and go back */
         BACKSPACE
         /* Move the pointer to the previous position */
```

```
            SET ptr TO @ptr - 1
            /* Fill it with zero as well */
            SET [ptr] TO 0
         ENDIF
      ELSE
         /*
         If the character is greater than 32 and less than 128 (printable ASCII characters)
         and we are not at the end of the array...
         */
         IF @chr >= 32 AND @chr < 128 AND @ptr < @@ptr THEN
            /* Print the character */
            CHAR @chr
            /* Store the character in the array */
            SET [ptr] TO @chr
            /* Advance the pointer */
            SET ptr TO @ptr + 1
         ENDIF
      ENDIF
   WEND
]]
```

As always, if we try it:

A "command line" appears where we can type; with **ENTER** we validate, and with **DELETE** we can delete the last character we have typed. You will notice, if you play with the program, that if we exceed 16 characters, it does not let us enter more, and only allows us to delete the last character or press **ENTER**. If we do the latter:

It prints the string we have entered! In this simple way, we already have a couple of subroutines that we can use to read and print small text strings. The comments included are enough to know what it is doing at each step, but there are things that we need to consolidate and with concepts that are already quite complex; so this section will have, exceptionally, subsections.

1.23.1 Indirection

In the example code you will have seen that variables are placed in square brackets, for example `SET [ptr] TO 0`. This is called indirection, and what it means is that **the result of the expression inside the square brackets is used as the pointer to the variable to be accessed**. This concept is called **indirection**, which allows us to create **pointers** and implement **arrays**.

The difference between `SET 1 TO 0` and `SET [1] TO 0` is that the first command means *"store zero in variable 1"*, and the second means *"store zero in the variable whose number corresponds to the content of variable 1"*. So, if in variable 1 we had a two, then it will store zero in variable 2. In this case, we are using variable number 1 as a *pointer*, since variable 1 *points* to another variable.

Let's look at an example. We want to fill variables 0 through 7 with zeros, using variable number 8 as a pointer. The code to do this would be as follows:

```
[[
SET 8 TO 0
```

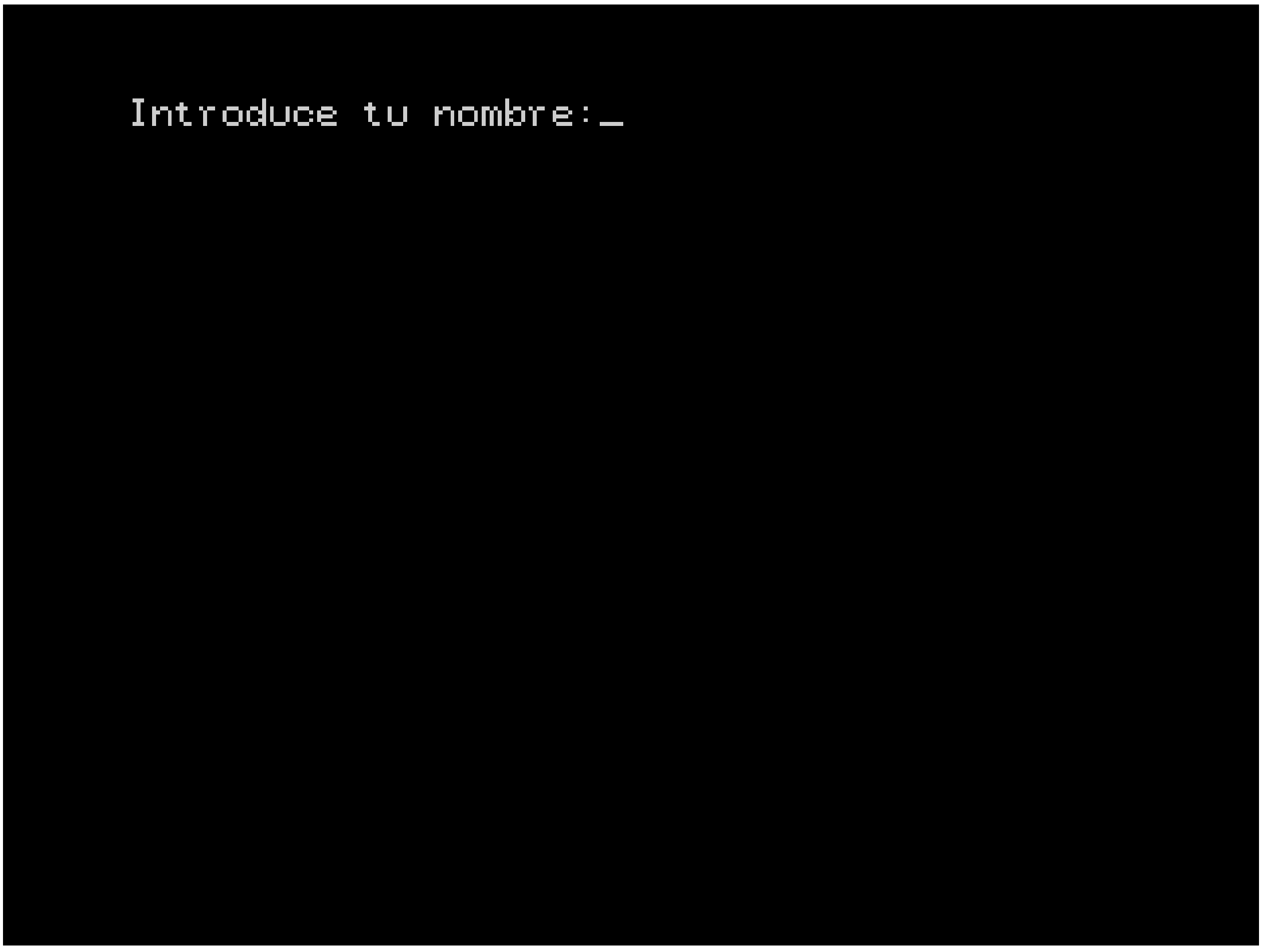


Figure 37: INPUT

Figure 38: OUTPUT

```
WHILE (@8 < 8)
   SET [8] TO 0
   SET 8 TO @8 + 1
WEND
]]
```

In the first line we initialize the pointer to 0 with `SET 8 TO 0`, which would give us this situation:

```
       0 1 2 3 4 5 6 7 8
      +-+-+-+-+-+-+-+-+-+-
      | | | | | | | | |0|...
      +-+-+-+-+-+-+-+-+-+-
```

Already inside the loop, we see that we have `SET [8] TO 0`, which indicates that in the variable whose index corresponds to the value of the content of variable 8, we will save the value zero:

```
       0 1 2 3 4 5 6 7 8
      +-+-+-+-+-+-+-+-+-+-
      |0| | | | | | | |0|...
      +-+-+-+-+-+-+-+-+-+-
       ^               |
       |               |
       +---------------+
```

The next step is to increment the pointer by one (`SET 8 TO @8 + 1`):

```
       0 1 2 3 4 5 6 7 8
      +-+-+-+-+-+-+-+-+-+-
      |0| | | | | | | |1|...
      +-+-+-+-+-+-+-+-+-+-
```

Since the value of variable 8 is still less than 8, we execute `SET [8] TO 0` again and save 0 in variable 1:

```
       0 1 2 3 4 5 6 7 8
      +-+-+-+-+-+-+-+-+-+-
      |0|0| | | | | | |1|...
      +-+-+-+-+-+-+-+-+-+-
         ^             |
         |             |
         +-------------+
```

We increase again:

```
       0 1 2 3 4 5 6 7 8
      +-+-+-+-+-+-+-+-+-+-
      |0|0| | | | | | |2|...
      +-+-+-+-+-+-+-+-+-+-
```

This process is executed until the loop condition is no longer met, which would result in this:

```
     0 1 2 3 4 5 6 7 8
    +-+-+-+-+-+-+-+-+-+-
    |0|0|0|0|0|0|0|0|8|...
    +-+-+-+-+-+-+-+-+-+-
```

So far I have used numerical names for variables. But how would we do it with variable name declarations?:

```
[[
DECLARE array AS 0
DECLARE ptr AS 8

SET ptr TO 0
WHILE (@ptr < 8)
   SET [ptr] TO 0
   SET ptr TO @ptr + 1
WEND
]]
```

The previous example could work… but if we decide to move the array and its pointer to other variables we will have a problem because in addition to the declarations, we would have to change `SET ptr TO 0` and `WHILE (@ptr < 8)` as well. But we have the following possibility:

```
[[
DECLARE array AS 0
DECLARE ptr AS 8

SET ptr TO @@array
WHILE (@ptr < @@ptr)
   SET [ptr] TO 0
   SET ptr TO @ptr + 1
WEND
]]
```

You will notice that instead, we are using the variable names with two @ signs `@@`. As you may know, when we find an @ sign inside an expression, it indicates that we are going to access the content of the corresponding variable. Well, the two @ signs indicate that we are going to use **the variable number** in the expression. So in the previous example `@@array` will be 0 and `@@ptr` will be 8. On the other hand, if we did the following, `@@array` would be 8 and `@@ptr` would be 16:

```
[[
DECLARE array AS 8
DECLARE ptr AS 16

SET ptr TO @@array
WHILE (@ptr < @@ptr)
   SET [ptr] TO 0
   SET ptr TO @ptr + 1
WEND
```

```
]]
```

It also allows us to access individual elements of the array. For example, to print the value of the fourth position of the array, such that if *array* is the variable zero:

```
      0 1 2 3 4 5 6 7
     +-+-+-+-+-+-+-+-+-
     |0|0| |4| | | | |...
     +-+-+-+-+-+-+-+-+-
            ^
```

We would do it like this with `PRINT [@@array + 3]`, and it would print 4. Remember that arrays are counted from zero, not from one. Another way to look at the `@@` operation is as the inverse of `[]`, so that `PRINT [@@array]` would be equivalent to `PRINT @array`.

This is a pretty advanced concept for someone who has never programmed before. I've tried to give a simple explanation, but if you don't understand it, it's normal and not necessary to make an interesting adventure, but if you do understand it, it can give you a powerful tool.

With these concepts in place and with the comments, you can understand how the first example works.

1.23.2 Reading the keyboard (INKEY)

To read a pressed key, we have the `INKEY()` function, which returns the code of the pressed key. This function works the same as its equivalent in Sinclair BASIC, but with the exception that the BASIC version returns zero if there is no valid key pressed, while the default behavior in CYD is to wait for a valid key to be pressed. If we want to reproduce the behavior of Sinclair BASIC, we can do so by using `INKEY(1)`.

The value returned when a key is pressed corresponds (normally) to its **ASCII** value, but adapted to the **Sinclair** version.

In the initial example, we check if the value of the returned key is 13 (ENTER) to validate and 12 (DELETE) to delete. If it is a value between 32 and 127, then it is a character that can be printed.

1.23.3 Deleting characters and making line breaks (BACKSPACE and NEWLINE)

To implement a command line, it is necessary to delete in case of a mistake. For this purpose, the `BACKSPACE` command has been added, which moves the cursor one position backwards and deletes the content of the new position. If it is on the left side of the defined border, it will jump one line backwards and will be placed at the end of the previous line (if possible).

It should be noted that the size used in the operation is that of character number 32 (the space). This is important if characters of sizes other than the space are used, the command will not delete them correctly, since CYD has no "memory" of what has already been printed.

The `NEWLINE` command is also introduced, which prints a line break without having to switch from "command" mode to "text" mode.

Both `NEWLINE` and `BACKSPACE` allow an optional parameter to indicate the number of times to print, such that `NEWLINE 3` would make 3 line breaks and `BACKSPACE 2` would delete two positions.

1.24 Using the alternative character set

CYD comes with two character sets for texts, the default one (which is 6 pixels wide) and an alternative one which is 4 pixels wide. To change from one to the other, use the `CHARSET` command as follows:

```
[[CHARSET 0]]6x8 character set
[[CHARSET 1]]4x8 character set
[[CHARSET 0]]Back to the default set[[
   WAITKEY
   END]]
```

With this result:

Figure 39: CHARSET

The default character set (`CHARSET 0`) is characters from zero to 127, which contain characters 6 pixels wide. With `CHARSET 1`, we indicate to use characters from 128 to 255, which contain characters 4 pixels wide.

1.25 Windows

Windows are up to 8 different independent text areas that we can define on the screen. In each one we can define the margins differently, and each one has its own cursor position and attributes. At any given time, we can only have one "active" window, that is, one window to write on and configure. You switch between them using the `WINDOW` command. The default active window is window zero.

As always, an example is more eloquent:

```
[[/* The default window is zero */
  MARGINS 0, 0, 16, 12
  INK 2
  CLEAR
  WINDOW 1
  MARGINS 16, 0, 16, 12
  INK 6
  CLEAR
  WINDOW 2
  MARGINS 0, 12, 16, 12
  INK 4
  CLEAR
  WINDOW 3
  MARGINS 16, 12, 16, 12
  INK 5
  CLEAR
  WINDOW 0
]]This is window 0
[[
  WINDOW 1
]]This is window 1
[[
  WINDOW 2
]]This is window 2
[[
  WINDOW 3
]]This is window 3
[[
  WINDOW 0
]]Back to window 0
[[
  WINDOW 1
]]Back to window 1
[[
  WINDOW 2
]]Back to window 2
[[
  WINDOW 3
]]Back to window 3
[[
  WAITKEY
  END]]
```

In the example, 4 windows are used, from 0 to 3. First we start with window zero, which is selected by default, and we define its position, size and ink color. Then we switch to window 1 and do the same, and so on with the rest. Finally we switch between windows to put text in each one.

And this is the result:

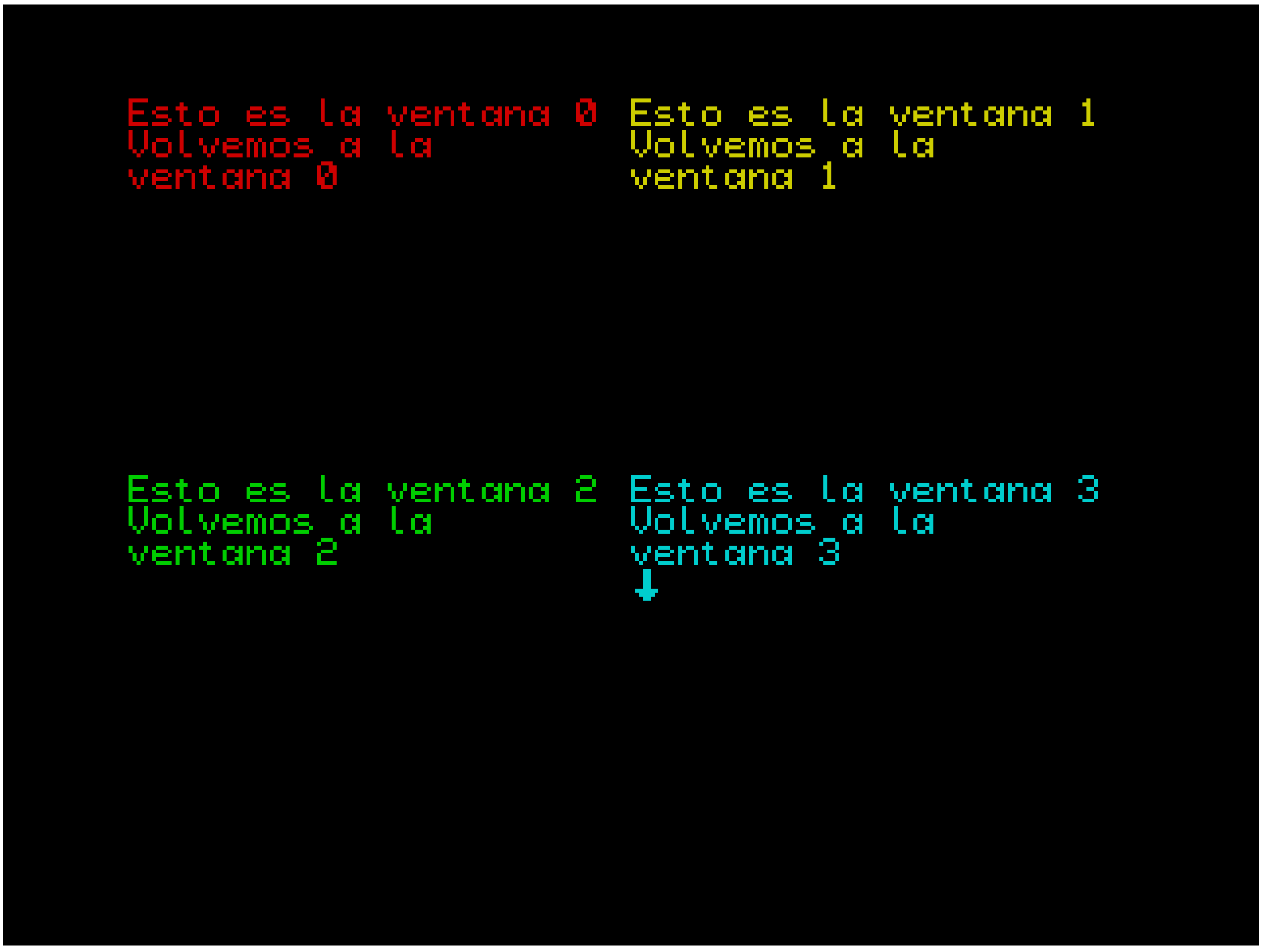


Figure 40: WINDOW

As you can see, the color, margins, and cursor position are preserved in each window, and we can easily switch between them. This allows us to have different text areas for menus, descriptions, bookmarks, etc.

Finally, it should be noted that these windows are not equivalent to the windows of a desktop environment, such as Windows, Mac, KDE, or GNOME. If two windows overlap, and you type in one of them, the content of the other window is not preserved. Think of them as text areas instead.

1.26 Arrays or sequences

Arrays are sequences of numbers that can be accessed by an index. Using the `DIM` command we can declare an "array", so that with `DIM name(size)` we declare an array with the name `name` and size `size`. The size of an array cannot be greater than 256 or be zero. We access each of the elements of the array with the nomenclature `name(pos)`, where `pos` is the position number of the element to be accessed within the array, starting from zero.

Let's see an example:

```
[[
  DIM miArray(3)          /* Declare an array of 3 elements from 0 to 2              */
  LET miArray(0) = 10     /* Assign values to each element                          */
  LET miArray(1) = 11
  LET miArray(2) = 12
  PRINT miArray(0)        /* Print the value stored at position 0            */
  NEWLINE
  PRINT miArray(1)        /* Print the value stored at position 1            */
  NEWLINE
  PRINT miArray(2)        /* Print the value stored at position 2            */
  NEWLINE : WAITKEY]]
```

Figure 41: ARRAY1

If you notice, we are treating `myArray(0)`, for example, as if it were a variable. It can be assigned and its values collected.

But **be careful!**, arrays start counting from zero to the size we have defined in the array declaration minus one. If we try to access a position outside its range, we will get a type 7 error:

```
[[
  DIM miArray(3)          /* Declare an array of 3 elements from 0 to 2              */
  LET miArray(0) = 10     /* Assign values to each element                          */
  LET miArray(1) = 11
```

```
  LET miArray(2) = 12
  PRINT miArray(0)        /* Print the value stored at position 0            */
  NEWLINE
  PRINT miArray(1)        /* Print the value stored at position 1            */
  NEWLINE
  PRINT miArray(2)        /* Print the value stored at position 2            */
  NEWLINE
  PRINT miArray(3)        /* THIS GIVES AN ERROR!                                          */
  NEWLINE : WAITKEY]]
```

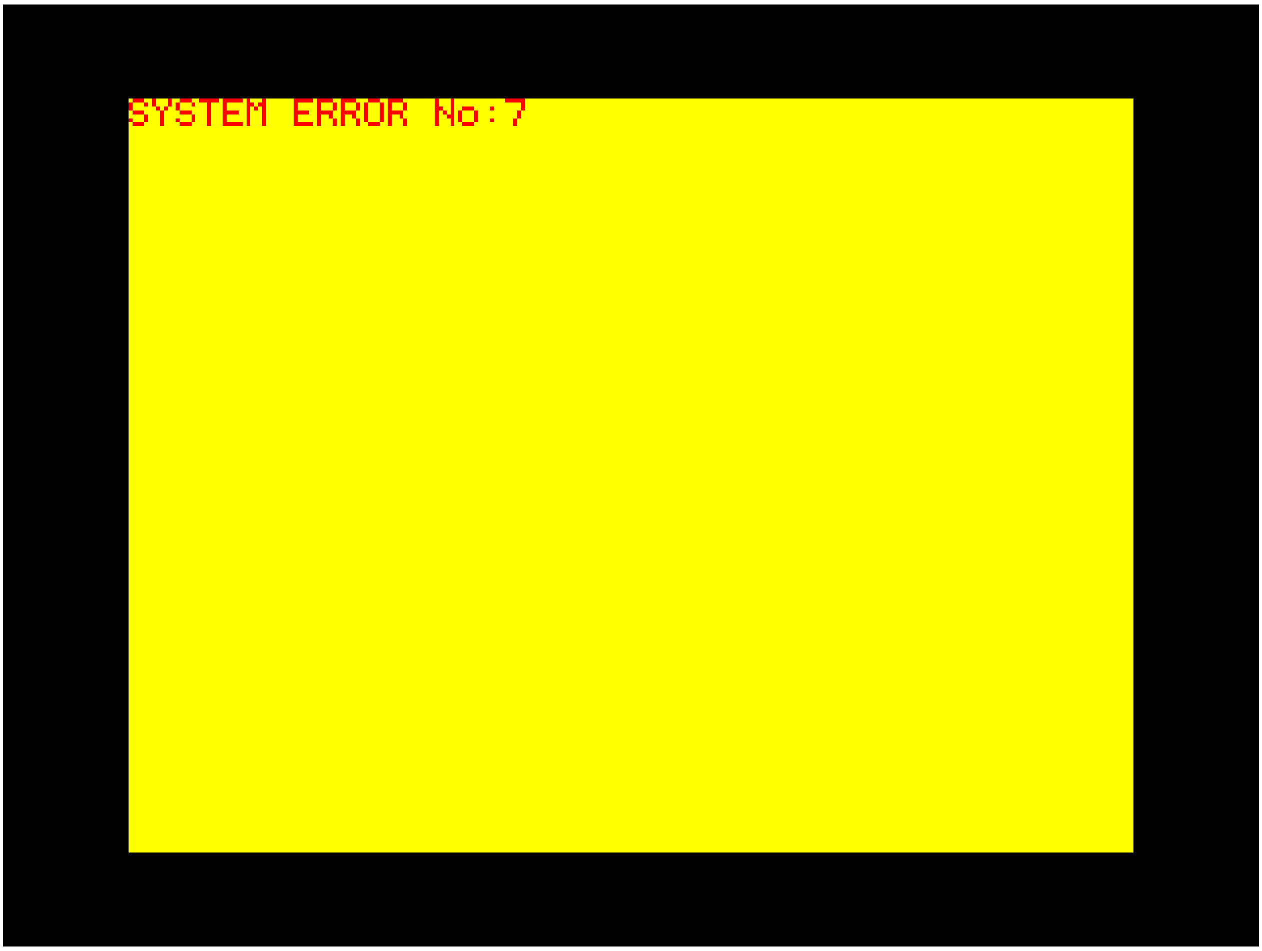


Figure 42: ARRAY2

To avoid these situations, you have the `LASTPOS(array_name)` function that returns the last allowed position for the given array so you can check before assigning:

```
[[
  DIM miArray(3)          /* Declare an array of 3 elements from 0 to 2              */
  LET miArray(0) = 10     /* Assign values to each element                          */
  LET miArray(1) = 11
  LET miArray(2) = 12
  PRINT miArray(0)        /* Print the value stored at position 0            */
  NEWLINE
  PRINT miArray(1)        /* Print the value stored at position 1            */
  NEWLINE
```

```
PRINT miArray(2)        /* Print the value stored at position 2            */
NEWLINE
]]Last position:[[
PRINT LASTPOS(miArray)  /* See the last allowed position     */
]]
Array size:[[
PRINT LASTPOS(miArray)+1  /* Adding one, we get its total size */
NEWLINE : WAITKEY]]
```

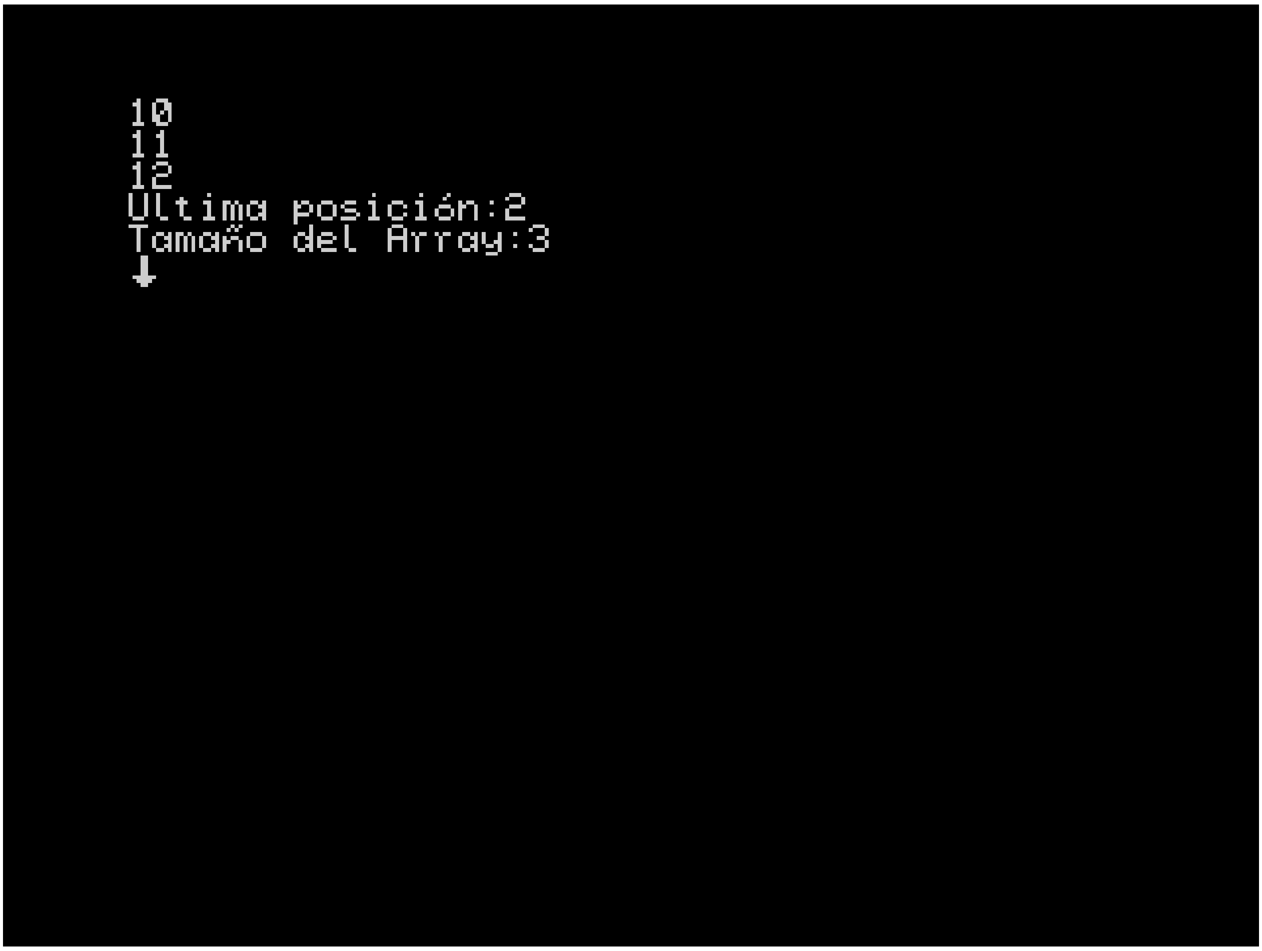


Figure 43: ARRAY3

Arrays allow us to have tables of values, but initializing elements one by one can be cumbersome and inelegant.

We can initialize array values by declaring them in the following way:

```
[[ DIM precios(5) = {10, 40, 100, 200, 250} ]]
```

In fact, we can save ourselves from indicating the size since it will be calculated from the number of elements indicated:

```
[[ DIM precios() = {10, 40, 100, 200, 250} ]]
```

Note that in this case the array will have the size of the number of elements that we have indicated **and no more**.

Arrays cannot be resized and only accept values between 0 and 255, in the same way as variables. The maximum size is 256 and the minimum is 0.

Let's look at one last example to consolidate concepts:

```
[[
  DECLARE 0 AS c
  DIM precios(5) = {10, 40, 100, 200, 250}
]] The [[ PRINT LASTPOS(precios)+1 ]] prices are:
[[
  LET c = 0
  WHILE(@c <= LASTPOS(precios))
      PRINT precios(@c)
      NEWLINE
      LET c = @c + 1
  WEND
  WAITKEY
]]
```

In the example, we use the variable `c` to iterate through the prices array until its last position and print the contents of each one:

Figure 44: ARRAY4

Finally, it should be noted that another limitation that arrays have is that they cannot be recorded directly onto tape or disk, unless their contents are dumped into variables beforehand.

1.27 Changing the character set

CYD has the following default character set:

Figure 45: Default character set

They are arranged in the image as a grid, starting from the top left, and going from left to right and up to down.

You can see both the 6x8 characters of the normal mode and the 4x8 characters of the alternate mode, as well as the characters used in the animated wait and selection icon:

Characters 127 through 143 (both inclusive and starting from zero) are special and have the following functions:

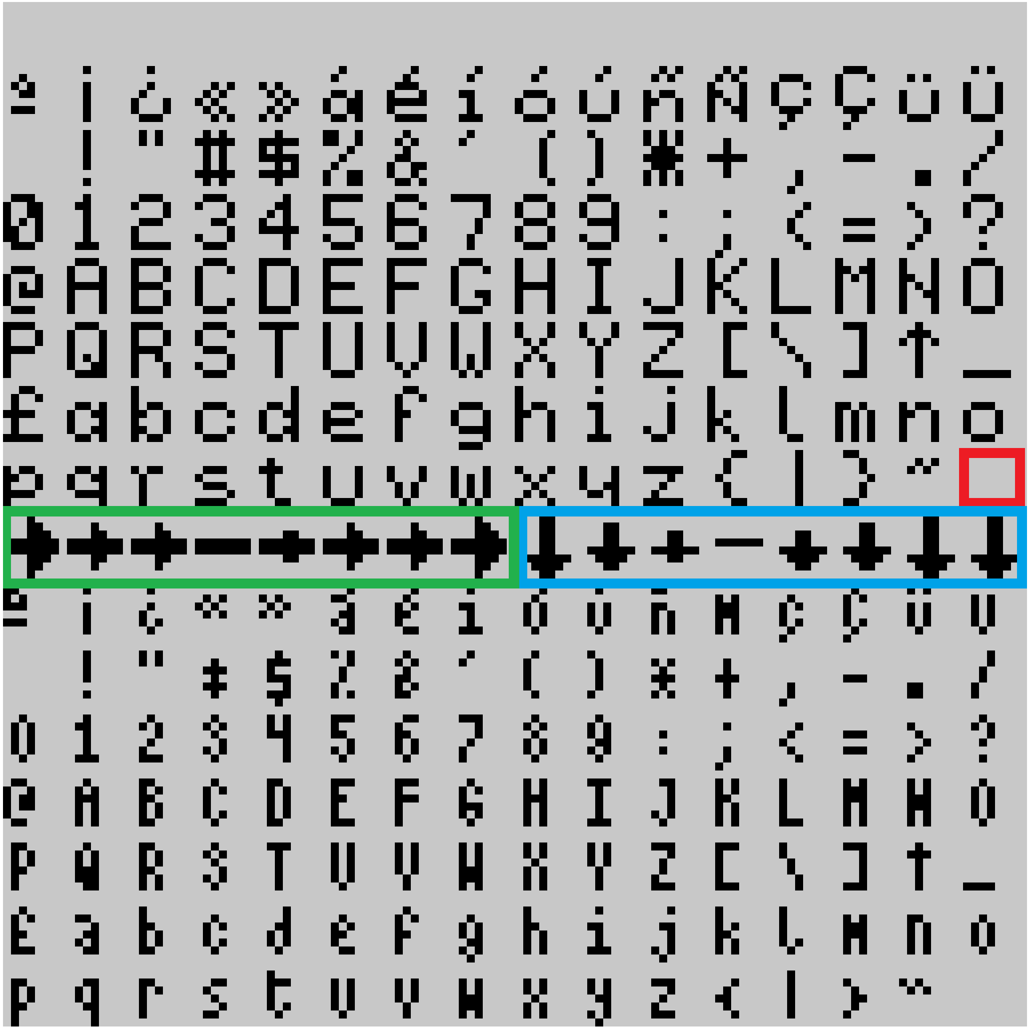


Figure 46: Special characters

- Character 127 is the character used when an option is not selected in a menu. (In red in the screenshot above)
- Characters 128 through 135 form the animation cycle of a selected option in a menu. (Green in the screenshot above)
- Characters 135 to 143 form the key wait indicator animation cycle. (Blue in the screenshot above)

Once we have this clear, what many of you will want is to change the animated icons to give them a personal touch. For this task we will need **ZxPaintbrush**. There is a copy included in the `Utiles` folder. We will also need the default font found in the `assets\default_charset.chr` file.

We will copy the file to the top directory, rename it as `charset.chr` and open it with ZxPaintbrush and with this we will have this:

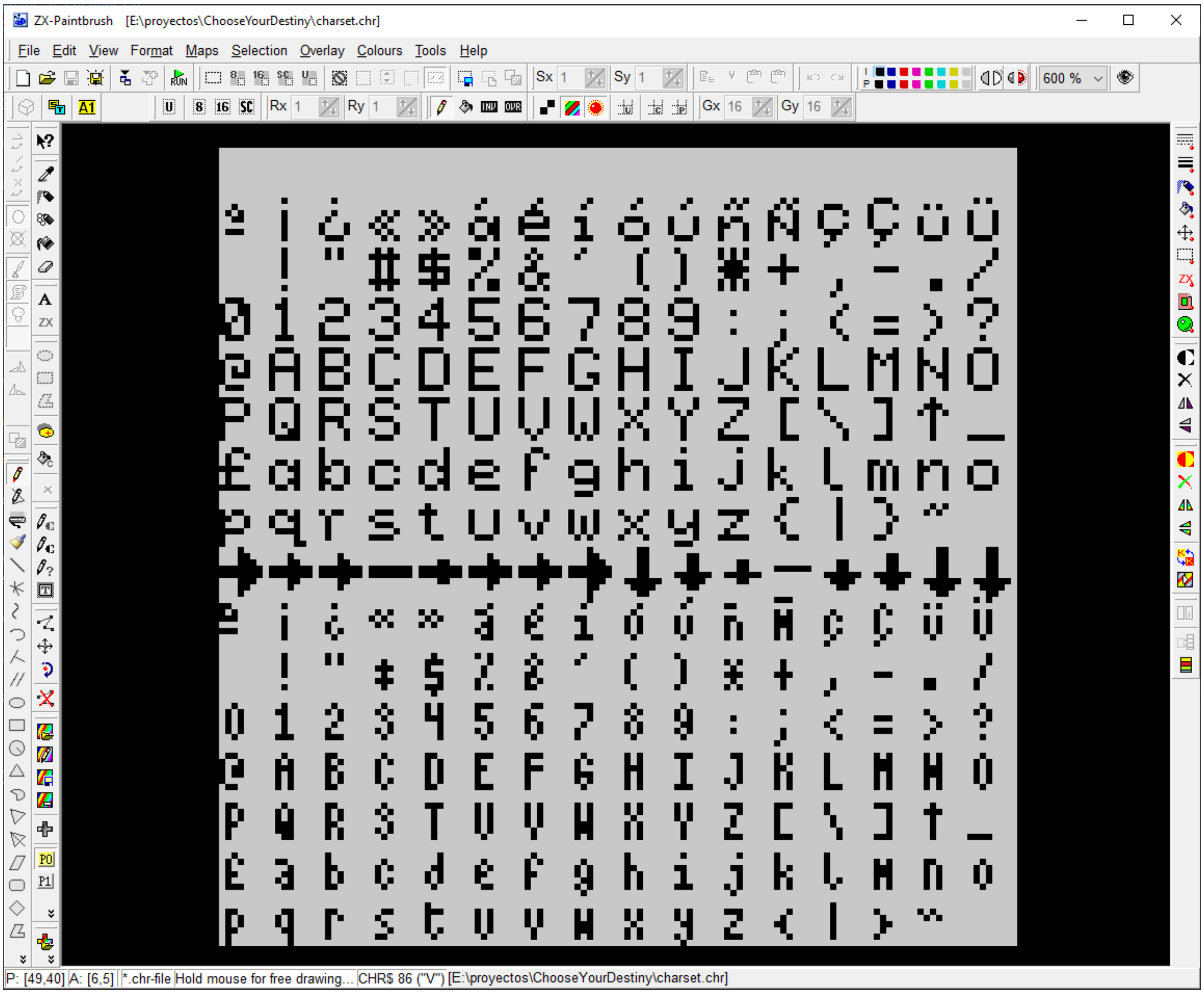


Figure 47: charset1

We are going to edit the key wait animation cycle (those marked in blue in the second screenshot). I'm going to put a clock, please excuse my poor artistic skills:

Now we need to convert them to a format that the compiler can digest. To do this we use the cyd_char_conv tool, which is located in `.\dist\cyd_chr_conv.cmd`. This tool is command line, so we will have to open a *Command Prompt* window in the project directory. If we run this command `.\dist\cyd_chr_conv.cmd --help`, we can see the options.

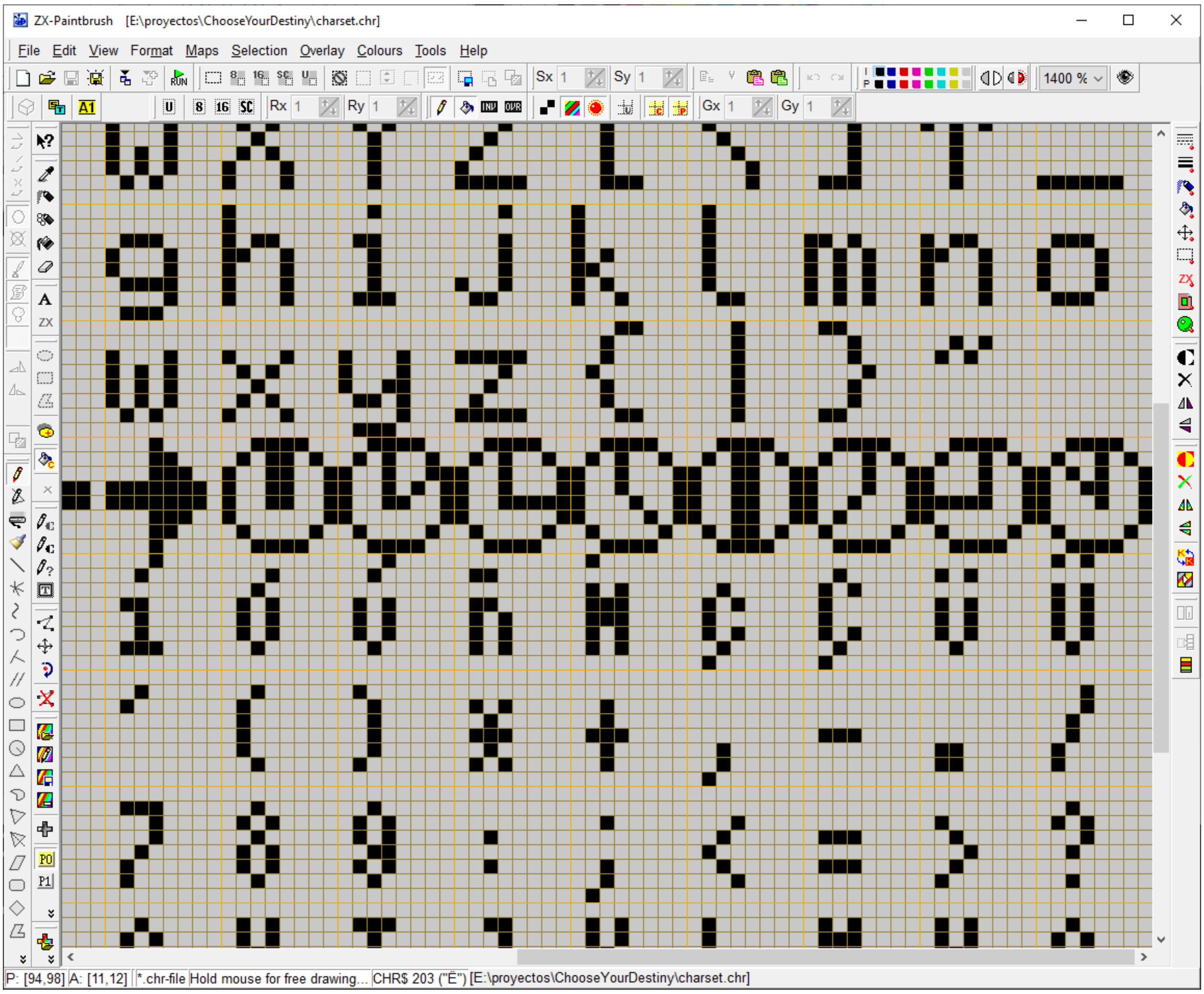
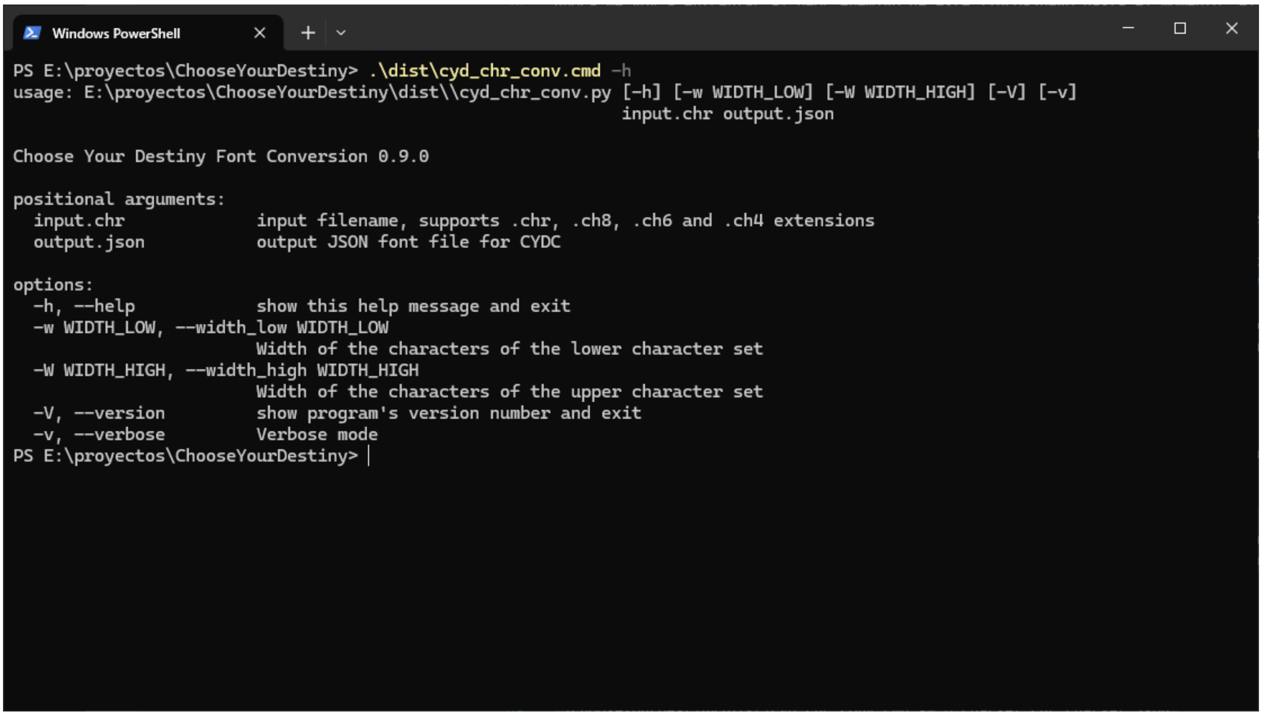


Figure 48: charset2




```
PS E:\proyectos\ChooseYourDestiny> .\dist\cyd_chr_conv.cmd -h
usage: E:\proyectos\ChooseYourDestiny\dist\\cyd_chr_conv.py [-h] [-w WIDTH_LOW] [-W WIDTH_HIGH] [-V] [-v]
                                                            input.chr output.json

Choose Your Destiny Font Conversion 0.9.0

positional arguments:
  input.chr             input filename, supports .chr, .ch8, .ch6 and .ch4 extensions
  output.json           output JSON font file for CYDC

options:
  -h, --help            show this help message and exit
  -w WIDTH_LOW, --width_low WIDTH_LOW
                        Width of the characters of the lower character set
  -W WIDTH_HIGH, --width_high WIDTH_HIGH
                        Width of the characters of the upper character set
  -V, --version         show program's version number and exit
  -v, --verbose         Verbose mode
PS E:\proyectos\ChooseYourDestiny> |
```

Figure 49: charset3

To do the conversion, we'll use it like this:

```
.\dist\cyd_chr_conv.cmd -w 6 -W 4 charset.chr charset.json
```

With the parameter `-w`, we indicate the width that the characters of the lower set will have, which in our case are 6 and with `-W` we indicate the size of the characters of the upper set, 4 in this case.

With this, we already have the file `charset.json` that we would have to pass to the compiler.

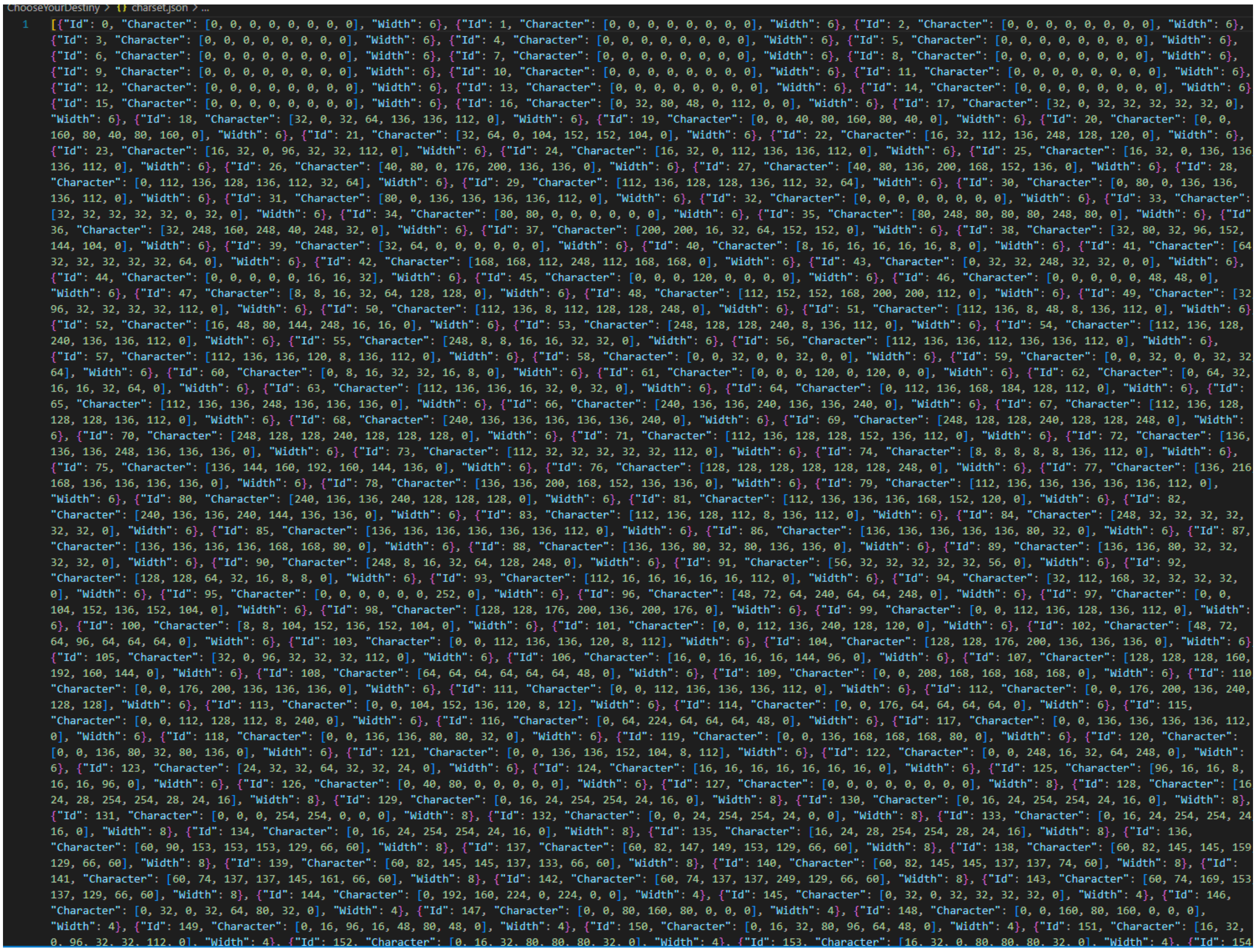


Figure 50: charset4

To tell the compiler that we are going to use the new character set, we have to indicate it with the parameter `-c`. Fortunately, the file `make_adv.cmd` will do this work for us.

To do this, we modify the header of that file, putting `-c charset.json` inside the `CYDC_EXTRA_PARAMS` variable, like this:

```
REM ---- Configuration variables ----------

REM Name of the game
SET GAME=test
REM This name will be used as:
REM   - The file to compile will be test.cyd with this example
REM   - The name of the TAP file or +3 disk image

REM Target for the compiler (48k, 128k for TAP, plus3 for DSK)
```

```
SET TARGET=48k

REM Number of lines used on SCR files at compressing
SET IMGLINES=192

REM Loading screen
SET LOAD_SCR="LOAD.scr"

REM Parameters for compiler
SET CYDC_EXTRA_PARAMS=-c charset.json

REM --------------------------------------
...
```

Running the `make_adv.cmd` file and launching the resulting tape file with an emulator, we see that the icon has changed:

Figure 51: charset4

In this way, we can change the rest of the animated icons, the fonts in general or even make special graphic characters, like "UDG", and display them with the `CHAR` command.

1.28 Native routines (IMPORT / CALL)

For jobs the virtual machine cannot do on its own —reading or writing a hardware port, touching an arbitrary memory address, or a tight bit of number-crunching that has to be fast— you can write a small routine in **Z80 assembly** and call it from your script.

> **Advanced, use at your own risk.** This runs your own native code with no safety net: a bug in the routine can hang or crash the machine, just like the BeepFX or WyzTracker code you already trust. If you are not comfortable with Z80 assembly, you do not need this feature.

Two keywords are involved:

```
[[
    IMPORT peek FROM "peek.asm"   /* declare the routine (compile-time) */
    SET 0 TO 0 : SET 1 TO 0       /* address 0000h in variables 0 and 1 */
    CALL peek                     /* run it; it leaves the result in variable 2 */
]]
```

- `IMPORT name FROM "file.asm"` registers a native routine. It is a compile-time declaration (like `DECLARE`), **not** an `INCLUDE`. A relative path is resolved relative to the **folder of your `.cyd` script**.
- `CALL name` runs it, the same way `GOSUB` calls a subroutine written in CYD.

You write **only the body** of the routine (no `ORG`, no directives); the compiler frames it, assembles it in isolation and places it in memory for you. The routine is entered with **`DE = FLAGS`**, the base of the variable array, so inputs and outputs travel through variables (variable `n` is the byte at `FLAGS + n`) and it must end with `RET`. For example, this `peek.asm` reads the byte at the address formed by variables 0 and 1 and leaves it in variable 2:

```
    ld a, (de)      ; DE = FLAGS -> A = variable 0 (address low byte)
    ld l, a
    inc de
    ld a, (de)      ; A = variable 1 (address high byte)
    ld h, a         ; HL = the address to read
    inc de          ; DE -> variable 2
    ld a, (hl)      ; A = the byte at that address
    ld (de), a      ; variable 2 = result
    ret
```

It works on 48K, 128K and +3. There is a full, runnable example in `examples/import_demo`, and the detailed contract (ABI) reference in the manual, section **Native routines (IMPORT / CALL)**.

1.28.1 Writing the assembler inline (`ASM … ENDASM`)

You don't always want a separate file. You can write the body **directly in the `.cyd`** between `ASM name` and `ENDASM`, and call it with `CALL` just as before:

```
[[
    ASM put42
        ld a, 42
        ld (de), a          ; DE = FLAGS -> variable 0 = 42
```

```
        ret
    ENDASM
    CALL put42
]]
```

It is the same ABI as with `IMPORT` (entered with `DE = FLAGS`, ends in `RET`): `ASM … ENDASM` is just an `IMPORT` with the body inline. You may indent the block to line it up with the `[[ ]]`; the compiler places the labels where the assembler needs them.

From there, a block can expose **several routines** that share code (`ASM lib EXPORTS a, b`), your routines can **read the script's arrays** by name (`ARR_<name>`, with services that do the paging for you on 128K/+3), and one routine can **call another in a different block** with `CYD_CALL` by declaring it in `USES`. Also, a routine nobody calls takes no memory: it is dropped automatically.

All of this is explained in detail in the manual (section **Native routines**), and there is a complete example that brings it together in `examples/inline_asm`. The idea is to be able to write comfortable native "libraries" when the virtual machine falls short on speed.